

EURSKEM AI · ENGINEERING DOCUMENTATION

VOLUME 2

Backend Code Reference

A module-by-module reference for the entire Python backend: every file, class, and public function.

Platform: Agentic Workflow Platform

Version: 0.1.0 · Generated: 2026-08-22

Built by Ayush Khandelwal

Typed workflows compiled to LangGraph · zero-token preflight · strict evidence lifecycle

1. Scope

This volume documents the complete backend package `app/` as it exists in source: **310 modules**, **756 classes**, and **2221 functions/methods** across **27 packages**. Every entry carries the signature and the first line of its docstring, so this volume doubles as a map from capability to file. Signatures are generated from the AST; nothing is hand-copied.

Package	Modules
<code>app.api</code>	26
<code>app.</code>	3
<code>app.db</code>	3
<code>app.evaluation</code>	6
<code>app.evidence</code>	6
<code>app.ingestion</code>	14
<code>app.integrations</code>	3
<code>app.integrations.email</code>	9
<code>app.integrations.files</code>	8
<code>app.knowledge</code>	6
<code>app.llm</code>	18
<code>app.mcp</code>	8
<code>app.mcp.business_records</code>	7
<code>app.mcp.d365_finance</code>	4
<code>app.mcp.dynamics</code>	6
<code>app.nodes</code>	61
<code>app.observability</code>	5
<code>app.proposal_graph</code>	9
<code>app.rag</code>	3
<code>app.research</code>	3
<code>app.retrieval</code>	16
<code>app.runtime</code>	25
<code>app.security</code>	17
<code>app.storage</code>	2

app.tools	10
app.workflow	14
app.workflow.business_view	18

api (26 modules)

app/api/__init__.py — 0 lines

(no summary)

app/api/audit.py — 38 lines

Audit module.

```
▶ async def get_session_audit(session_id: str, request: Request, run_id: str | None=None, user:
CurrentUser=Depends(require_consultant))
    Return the session audit.
```

app/api/auth.py — 151 lines

Auth module.

```
▶ class Token
    Pydantic model defining the Token shape.
▶ class Identity
    Pydantic model defining the Identity shape.
▶ def _set_auth_cookie(response: Response, token: str) -> None
    Attach the JWT as an HttpOnly cookie.
▶ async def login(request: Request, response: Response, form: OAuth2PasswordRequestForm=Depends())
    Compute the login.
▶ async def me(user: CurrentUser=Depends(require_consultant))
    Rehydrate identity from the auth cookie after a page refresh.
▶ async def logout(response: Response)
    Clear the auth cookie. Must match the path the cookie was set with.
```

app/api/builder.py — 1544 lines

Builder-support API — everything the visual authoring experience needs.

```
▶ class ContractRequest
    Pydantic model defining the ContractRequest shape.
▶ class ContractField
    Pydantic model defining the ContractField shape.
▶ class ContractNode
    Pydantic model defining the ContractNode shape.
▶ class SchemaPreviewRequest
    Pydantic model defining the SchemaPreviewRequest shape.
▶ class NodeTestRequest
    Pydantic model defining the NodeTestRequest shape.
▶ class SimulateRequest
    Pydantic model defining the SimulateRequest shape.
▶ class SchemaAssistRequest
    Pydantic model defining the SchemaAssistRequest shape.
▶ class ProposedField
    One suggested row. Mirrors FieldSpec, so the response drops straight into
▶ class ProposedSchema
    Pydantic model defining the ProposedSchema shape.
▶ class RuleAssistRequest
    Pydantic model defining the RuleAssistRequest shape.
▶ class ProposedRules
```

Pydantic model defining the ProposedRules shape.

- ▶ `class MCPToolTestRequest`
Pydantic model defining the MCPToolTestRequest shape.
- ▶ `def _services(request: Request) -> dict[str, Any]`
Internal helper for the services step.
- ▶ `def _friendly_validation_message(error: ValidationError) -> str`
One line per Pydantic error, without the "N validation errors for
- ▶ `def operators(user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Which operators apply to which field type, and how to render each one.
- ▶ `def output_contract(body: ContractRequest, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
What each step guarantees, as typed dotted paths (§40).
- ▶ `def schema_preview(body: SchemaPreviewRequest, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Compile visual schema rows and show what the model will be held to.
- ▶ `async def node_test(body: NodeTestRequest, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Run one node against sample data (§21).
- ▶ `async def simulate(body: SimulateRequest, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Run a workflow in memory and return a per-step trace (§23, §24, §47).
- ▶ `def _interrupt_values(interrupt: Any) -> list[dict[str, Any]]`
Pull the real payloads out of LangGraph's Interrupt objects.
- ▶ `def _waiting_steps(spec: WorkflowSpec, interrupt: Any) -> list[dict[str, Any]]`
One step per gate the run is parked at.
- ▶ `def _timeline(spec: WorkflowSpec, state: dict[str, Any], stubbed: set[str]) -> list[dict[str, Any]]`
Per-step trace in execution order, with each step's own explanation.
- ▶ `def _executed_path(state: dict[str, Any]) -> list[str]`
Internal helper for the executed path step.
- ▶ `def _explain(type_name: str, output: dict[str, Any]) -> dict[str, Any]`
Turn a node's output into the "why did this happen?" view (§24, §47).
- ▶ `async def assist_schema(body: SchemaAssistRequest, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Propose a structured-output schema from a description (§37).
- ▶ `async def assist_rules(body: RuleAssistRequest, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Turn a described business rule into deterministic configuration (§36).
- ▶ `def mcp_servers(request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Configured MCP servers for the Builder's connection panel.
- ▶ `async def mcp_tools(server_id: str, request: Request, refresh: bool=False, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Discover what a server can do (§5).
- ▶ `async def mcp_health(server_id: str, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Live connection check, for the panel's Test Connection button.
- ▶ `async def mcp_test_tool(body: MCPToolTestRequest, request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Run one MCP tool with literal arguments (§21).
- ▶ `def _require_mcp(request: Request) -> Any`
Internal helper for the require mcp step.
- ▶ `def email_connections(request: Request, user: CurrentUser=Depends(require_consultant)) -> dict[str, Any]`
Configured mailboxes for the Email node's connection picker.
- ▶ `def _contract_sort_key(field: Any) -> tuple[int, str]`
Order a step's contract so the author's own fields come first.

- ▶ `def _parse_workflow(yaml_text: str) -> WorkflowSpec`
Parse the workflow.
- ▶ `def _require_llm(request: Request) -> Any`
Internal helper for the require llm step.
- ▶ `def _label_of(node: Any) -> str`
The business label if the author set one, else the technical id (§17).
- ▶ `def _execution_kind(type_name: str) -> str`
Internal helper for the execution kind step.
- ▶ `def _is_external_write(type_name: str, config: dict[str, Any]) -> bool`
Return whether external write.
- ▶ `def _ancestors(spec: WorkflowSpec, node_id: str) -> set[str]`
Internal helper for the ancestors step.
- ▶ `def _slice_through(spec: WorkflowSpec, node_id: str) -> WorkflowSpec`
The smallest valid upstream slice ending at `node\_id` (§22).
- ▶ `def _short(value: Any, limit: int=80) -> str`
Internal helper for the short step.
- ▶ `def _previewable(value: Any, depth: int=0) -> Any`
Trim large values before they cross the wire.

app/api/candidates.py — 349 lines

Candidates viewer API: list a run's acquired evidence PDFs with open links,

- ▶ `class VerifyClaimRequest`
Pydantic model defining the VerifyClaimRequest shape.
- ▶ `def _discovery_candidate_url(candidate: dict[str, Any]) -> str | None`
Internal helper for the discovery candidate url step.
- ▶ `def _discovered_candidates_from_run(run: dict[str, Any]) -> list[dict[str, Any]]`
Every discovery/Deep Research/prior-project candidate this run ever
- ▶ `def _structured_dataset_query_url(record: dict[str, Any]) -> str | None`
Reconstruct the exact bounded Eurostat query that produced this row.
- ▶ `def _structured_dataset_candidates_from_run(run: dict[str, Any]) -> dict[str, dict[str, Any]]`
StructuredDatasetRetrieverAgent rows, reshaped into the same
- ▶ `def _internal_evidence_from_run(run: dict[str, Any]) -> list[dict[str, Any]]`
InternalProjectEvidenceRetrieverAgent facts: no URL exists for these,
- ▶ `async def list_candidates(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Every acquired evidence PDF for a run, plus discovered candidates.
- ▶ `async def verify_claim_endpoint(run_id: str, body: VerifyClaimRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Run the gpt-5.6-sol secondary check for one internal-evidence record.

app/api/cost.py — 57 lines

Cost module.

- ▶ `async def run_cost(run_id: str, request: Request, user: CurrentUser=Depends(get_current_user))`
Run the cost.
- ▶ `async def session_cost(session_id: str, request: Request, user: CurrentUser=Depends(get_current_user))`
Compute the session cost.

app/api/cost_admin.py — 634 lines

Account-wide cost management — the standalone admin page, distinct from app/api/cost.py's

- ▶ `class PricingOverrideRequest`
Pydantic model defining the PricingOverrideRequest shape.

- ▶ `class InfraAllocationRequest`
Pydantic model defining the `InfraAllocationRequest` shape.
- ▶ `class BudgetRequest`
Pydantic model defining the `BudgetRequest` shape.
- ▶ `def _db(request: Request)`
Internal helper for the db step.
- ▶ `def _cost_ledger_entries(db, *, since: datetime | None=None) -> list[dict]`
Internal helper for the cost ledger entries step.
- ▶ `async def list_pricing(request: Request, openrouter_q: str | None=Query(default=None, max_length=200), openrouter_limit: int=Query(default=25, ge=1, le=200), user: CurrentUser=Depends(require_admin))`
Direct/local catalog pricing (editable overrides layered on `MODEL_PRICING` defaults)
- ▶ `async def set_pricing_override(model: str, body: PricingOverrideRequest, request: Request, user: CurrentUser=Depends(require_admin))`
Set the pricing override.
- ▶ `async def clear_pricing_override(model: str, request: Request, user: CurrentUser=Depends(require_admin))`
Clear the pricing override.
- ▶ `def _effective_per_call_cost(allocation: dict[str, Any]) -> float | None`
Internal helper for the effective per call cost step.
- ▶ `async def list_infra_allocations(request: Request, user: CurrentUser=Depends(require_admin))`
List the infra allocations.
- ▶ `async def set_infra_allocation(model: str, body: InfraAllocationRequest, request: Request, user: CurrentUser=Depends(require_admin))`
Set the infra allocation.
- ▶ `def _allocated_infra_cost(entries: list[dict], allocations: dict[str, dict]) -> float`
Internal helper for the allocated infra cost step.
- ▶ `def _cache_savings_usd(entry: dict) -> float`
Mirrors `CostLedger.calculate()`'s own cache multipliers — what THIS entry's cached
- ▶ `async def cache_summary(request: Request, since_days: int=Query(default=30, ge=1, le=365), user: CurrentUser=Depends(require_admin))`
Compute the cache summary.
- ▶ `def _today_start() -> datetime`
Internal helper for the today start step.
- ▶ `async def get_budgets(request: Request, user: CurrentUser=Depends(require_admin))`
Return the budgets.
- ▶ `async def set_global_budget(body: BudgetRequest, request: Request, user: CurrentUser=Depends(require_admin))`
Set the global budget.
- ▶ `async def set_session_budget(session_id: str, body: BudgetRequest, request: Request, user: CurrentUser=Depends(require_admin))`
Set the session budget.
- ▶ `async def cost_overview(request: Request, days: int=Query(default=30, ge=1, le=365), user: CurrentUser=Depends(require_admin))`
Compute the cost overview.

`app/api/email_oauth.py` — 324 lines

Email OAuth connect flow — Outlook (Microsoft Graph) and Gmail.

- ▶ `class AllowSendUpdate`
Pydantic model defining the `AllowSendUpdate` shape.
- ▶ `async def ensure_indexes(db: Any) -> None`
TTL-expires an abandoned connect attempt's pending state — a user who
- ▶ `def _services(request: Request) -> dict[str, Any]`
Internal helper for the services step.
- ▶ `def _require_db(request: Request) -> Any`

```

    Internal helper for the require db step.
▶ def _require_email_service(request: Request) -> Any
    Internal helper for the require email service step.
▶ def _connection_id(provider: Provider, address: str) -> str
    Internal helper for the connection id step.
▶ def _html_result(*, ok: bool, message: str) -> HTMLResponse
    A minimal, self-contained page for the OAuth popup window — no
▶ async def connect(provider: Provider, request: Request, user:
CurrentUser=Depends(require_consultant)) -> RedirectResponse
    Starts the authorization-code flow: stash a one-time state (+ PKCE
▶ async def oauth_callback(provider: Provider, request: Request, code: str | None=None, state: str
| None=None, error: str | None=None, error_description: str | None=None) -> HTMLResponse
    The provider redirects the user's browser back here after consent.
▶ async def update_connection(connection_id: str, body: AllowSendUpdate, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    The only lever to actually let a connection send — every connection
▶ async def disconnect(connection_id: str, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    Compute the disconnect.

```

app/api/entity_registry.py — 169 lines

Manual entity-registry CRUD — Phase 1 confidential entity protection.

```

▶ class RegisterEntityRequest
    Pydantic model defining the RegisterEntityRequest shape.
▶ class DeleteEntityRequest
    Pydantic model defining the DeleteEntityRequest shape.
▶ def _scope(user: CurrentUser) -> str
    Internal helper for the scope step.
▶ def _service(request: Request)
    Internal helper for the service step.
▶ def list_entity_types() -> dict[str, list[str]]
    List the entity types.
▶ async def list_entities(request: Request, collection_id: str='default', user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    List the entities.
▶ async def register_entity(req: RegisterEntityRequest, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, str]
    Register the entity.
▶ async def delete_entity(req: DeleteEntityRequest, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, bool]
    Delete the entity.

```

app/api/eval.py — 283 lines

Eval Lab endpoints. Back the Phase 9 Eval Lab shell.

```

▶ class RunEvalRequest
    Pydantic model defining the RunEvalRequest shape.
▶ class WorkflowCompareRequest
    Pydantic model defining the WorkflowCompareRequest shape.
▶ class ScoreOutputRequest
    Pydantic model defining the ScoreOutputRequest shape.
▶ def _load_reference() -> str
    Extract text from the stored reference proposal (gitignored, private).

```


- ▶ `async def score_output(req: ScoreOutputRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
Score the output.
- ▶ `async def list_golden_set(name: str='document_qa', _user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
List the golden set.
- ▶ `async def run(req: RunEvalRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
Run the result.
- ▶ `async def history(request: Request, limit: int=20, _user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
Compute the history.
- ▶ `async def list_workflow_golden_set(name: str='verder_customer_triage', _user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
List the workflow golden set.
- ▶ `async def workflow_compare(req: WorkflowCompareRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run')))` `-> dict[str, Any]`
Run the same golden set of real customer messages through the flagship

app/api/health.py — 279 lines

Liveness and dependency-readiness endpoints.

- ▶ `async def _call_probe(probe: Probe)` `-> Any`
Internal helper for the call probe step.
- ▶ `async def _probe(name: str, probe: Probe | None)` `-> tuple[str, dict[str, Any]]`
Probe the result.
- ▶ `def _mongo_probe(services: dict[str, Any])` `-> Probe | None`
Internal helper for the mongo probe step.
- ▶ `def _weaviate_probe(services: dict[str, Any])` `-> Probe | None`
Internal helper for the weaviate probe step.
- ▶ `def _minio_probe(services: dict[str, Any])` `-> Probe | None`
Internal helper for the minio probe step.
- ▶ `def _redis_probe(services: dict[str, Any])` `-> Probe | None`
Internal helper for the redis probe step.
- ▶ `def _checkpointer_probe(services: dict[str, Any])` `-> Probe | None`
Internal helper for the checkpointer probe step.
- ▶ `async def probe_services(services: dict[str, Any])` `-> dict[str, dict[str, Any]]`
Probe every configured dependency concurrently.
- ▶ `async def _health_payload(request: Request)` `-> tuple[dict[str, Any], bool]`
Internal helper for the health payload step.
- ▶ `async def health(request: Request)`
Return live dependency status without failing the liveness request.
- ▶ `async def ready(request: Request)`
Return 503 until every required runtime dependency is reachable.

app/api/inspect.py — 396 lines

Read-only corpus & retrieval inspection endpoints.

- ▶ `class ChunkView`
Pydantic model defining the ChunkView shape.
- ▶ `class SourceGroup`
Pydantic model defining the SourceGroup shape.
- ▶ `class ChunksResponse`
Pydantic model defining the ChunksResponse shape.
- ▶ `class RetrieveRequest`

```

Pydantic model defining the RetrieveRequest shape.
▶ class RetrievedView
    Pydantic model defining the RetrievedView shape.
▶ class RetrieveResponse
    Pydantic model defining the RetrieveResponse shape.
▶ def _get_weaviate(request: Request)
    Return the WeaviateClient wrapper.
▶ def _native_collection(wv)
    Return the v4 native collection handle for DocumentChunk.
▶ def list_chunks(request: Request, collection_id: Optional[str]=Query(None), session_id:
Optional[str]=Query(None), limit: int=Query(2000, le=10000), user:
CurrentUser=Depends(require_permission('workflow:read'))) -> ChunksResponse
    List ingested chunks grouped by source document.
▶ async def _run_retrieval(request: Request, req: RetrieveRequest, user: CurrentUser)
    Run the query through the canonical retrieval path.
▶ async def retrieve_preview(request: Request, req: RetrieveRequest, user:
CurrentUser=Depends(require_permission('workflow:read'))) -> RetrieveResponse
    Run a query through hybrid search + rerank and show the ranked chunks.

```

app/api/integration_oauth.py — 399 lines

Integration OAuth connect flow — Google Drive and OneDrive.

```

▶ async def ensure_indexes(db: Any) -> None
    TTL-expires an abandoned connect attempt's pending state.
▶ def _services(request: Request) -> dict[str, Any]
    Internal helper for the services step.
▶ def _require_db(request: Request) -> Any
    Internal helper for the require db step.
▶ def _require_integration_service(request: Request) -> Any
    Internal helper for the require integration service step.
▶ def _connection_id(provider: Provider, address: str) -> str
    Internal helper for the connection id step.
▶ def _html_result(*, ok: bool, message: str) -> HTMLResponse
    A minimal, self-contained page for the OAuth popup window.
▶ async def connect(provider: Provider, request: Request, user:
CurrentUser=Depends(require_consultant)) -> RedirectResponse
    Starts the authorization-code flow: stash a one-time state (+ PKCE)
▶ async def oauth_callback(provider: Provider, request: Request, code: str | None=None, state: str
| None=None, error: str | None=None, error_description: str | None=None) -> HTMLResponse
    The provider redirects the user's browser back here after consent.
▶ async def list_connections(request: Request, user: CurrentUser=Depends(require_consultant)) ->
dict[str, Any]
    List the connections.
▶ async def disconnect(connection_id: str, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    Compute the disconnect.
▶ async def browse_files(connection_id: str, request: Request, folder_id: str | None=None, query:
str='', page_size: int=25, page_token: str | None=None, user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    Live folder/search browsing for the node config panel's file picker
▶ async def browse_path(connection_id: str, file_id: str, request: Request, user:
CurrentUser=Depends(require_consultant)) -> dict[str, Any]
    Breadcrumb chain for the file browser: walk `parent_id` up from
▶ async def download_file(connection_id: str, file_id: str, request: Request, user:
CurrentUser=Depends(require_consultant)) -> Response
    Streams one file's real bytes straight to the browser — a plain

```

Knowledge Studio collection, profile, index, document and ingestion APIs.

```

▶ class CollectionCreate
    Pydantic model defining the CollectionCreate shape.
▶ class ProfileCreate
    Pydantic model defining the ProfileCreate shape.
▶ class IndexCreate
    Pydantic model defining the IndexCreate shape.
▶ def _scope(user: CurrentUser) -> str
    Internal helper for the scope step.
▶ def _service(request: Request)
    Internal helper for the service step.
▶ def _repository(request: Request)
    Internal helper for the repository step.
▶ async def create_collection(payload: CollectionCreate, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write')))
    Create the collection.
▶ async def list_collections(request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read')))
    List the collections.
▶ async def get_collection(collection_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read')))
    Return the collection.
▶ async def create_profile(payload: ProfileCreate, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write')))
    Create the profile.
▶ async def list_profiles(request: Request, profile_type: ProfileType | None=Query(None), user:
CurrentUser=Depends(require_permission('knowledge:read')))
    List the profiles.
▶ async def get_profile(profile_id: str, request: Request, version: int | None=Query(None, ge=1),
user: CurrentUser=Depends(require_permission('knowledge:read')))
    Return the profile.
▶ async def create_default_profiles(request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write')))
    Create the default profiles.
▶ async def ingestion_presets(user: CurrentUser=Depends(require_permission('knowledge:read')))
    Compute the ingestion presets.
▶ async def doc_types(user: CurrentUser=Depends(require_permission('knowledge:read')))
    Document types a Collection may declare.
▶ async def embedding_models(user: CurrentUser=Depends(require_permission('knowledge:read')))
    Embedding models selectable when building an Index.
▶ async def create_index(collection_id: str, payload: IndexCreate, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write')))
    Create the index.
▶ async def list_indexes(collection_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read')))
    List the indexes.
▶ async def activate_index(collection_id: str, index_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write')))
    Compute the activate index.
▶ async def start_ingestion(collection_id: str, request: Request, files:
list[UploadFile]=File(...), parser_profile_id: str | None=Form(None), parser_profile_version: int |
None=Form(None), chunking_profile_id: str | None=Form(None), chunking_profile_version: int |
None=Form(None), embedding_profile_id: str | None=Form(None), embedding_profile_version: int |

```

```

None=Form(None), embedding_model: str | None=Form(None), metadata_json: str=Form('{}'), user:
CurrentUser=Depends(require_permission('knowledge:write'))
    Start the ingestion.
▶ async def list_ingestions(request: Request, collection_id: str | None=Query(None), user:
CurrentUser=Depends(require_permission('knowledge:read'))
    List the ingestions.
▶ async def get_ingestion(ingestion_job_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read'))
    Return the ingestion.
▶ async def cancel_ingestion(ingestion_job_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:write'))
    Cancel the ingestion.
▶ async def list_documents(collection_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read'))
    List the documents.
▶ async def get_document(document_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read'))
    Return the document.
▶ async def document_source_url(document_id: str, request: Request, user:
CurrentUser=Depends(require_permission('knowledge:read'))
    Compute the document source url.

```

app/api/llm_providers.py — 135 lines

Safe model-catalog and local-provider status endpoints.

```

▶ def _configured(model: str, provider: str) -> bool
    Internal helper for the configured step.
▶ async def list_models(user: CurrentUser=Depends(require_consultant))
    Return public capability metadata without URLs or credentials.
▶ async def list_openrouter_models(q: str | None=Query(default=None, max_length=200), limit:
int=Query(default=50, ge=1, le=200), user: CurrentUser=Depends(require_consultant))
    Live OpenRouter model catalog (via OpenRouter's own `GET /v1/models`), cached with a
▶ async def probe_model(model: str, user: CurrentUser=Depends(require_admin))
    Probe an enabled local endpoint; never return its private address.

```

app/api/node_types_chat.py — 637 lines

Ask AI about node types — explains what a node type does, when to use it,

```

▶ class ChatMessage
    Pydantic model defining the ChatMessage shape.
▶ class AskContext
    Compact, structured context for a scoped Ask AI question — what the
▶ class AskAboutNodeTypesRequest
    Pydantic model defining the AskAboutNodeTypesRequest shape.
▶ class DraftPromptRequest
    Pydantic model defining the DraftPromptRequest shape.
▶ class DraftInstructionsFieldSpec
    Pydantic model defining the DraftInstructionsFieldSpec shape.
▶ class DraftInstructionsRequest
    Pydantic model defining the DraftInstructionsRequest shape.
▶ class DraftCodeRequest
    Pydantic model defining the DraftCodeRequest shape.
▶ def _schema_field_names(schema: dict[str, Any] | None) -> list[str]
    Internal helper for the schema field names step.
▶ def _build_node_type_catalog() -> str

```

Always reads `NodeRegistry.manifest()` fresh — the same live source

- ▶ `def _manifest_entry(type_name: str) -> dict[str, Any] | None`
Live lookup — same freshness guarantee as `_build_node_type_catalog`.
- ▶ `def _compact_catalog(type_names: list[str]) -> str`
Same one-line-per-type shape as `_build_node_type_catalog()`, but for an
- ▶ `async def ask_about_node_types(req: AskAboutNodeTypesRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the ask about node types.
- ▶ `async def draft_prompt(req: DraftPromptRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Draft the prompt.
- ▶ `def _describe_fields(fields: list[DraftInstructionsFieldSpec]) -> str`
Internal helper for the describe fields step.
- ▶ `async def draft_instructions(req: DraftInstructionsRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Draft the instructions.
- ▶ `async def draft_code(req: DraftCodeRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Draft the code.

`app/api/pipelines.py` — 347 lines

Pipelines chain saved workflows so one workflow's node outputs become

- ▶ `class ValidatePipelineRequest`
Pydantic model defining the `ValidatePipelineRequest` shape.
- ▶ `class RunPipelineRequest`
Pydantic model defining the `RunPipelineRequest` shape.
- ▶ `class AdvancePipelineRequest`
Pydantic model defining the `AdvancePipelineRequest` shape.
- ▶ `class AbandonPipelineRequest`
Pydantic model defining the `AbandonPipelineRequest` shape.
- ▶ `class SavePipelineRequest`
Pydantic model defining the `SavePipelineRequest` shape.
- ▶ `def validate_pipeline(req: ValidatePipelineRequest)`
Validate a pipeline without running a stage or spending tokens.
- ▶ `def _preflight_http_detail(report) -> dict[str, Any]`
Internal helper for the preflight http detail step.
- ▶ `async def run(req: RunPipelineRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Run the result.
- ▶ `async def advance(pipeline_run_id: str, req: AdvancePipelineRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the advance.
- ▶ `async def my_pipeline_runs(request: Request, limit: int=50, user: CurrentUser=Depends(require_consultant))`
Compute the my pipeline runs.
- ▶ `async def my_pipeline_run_detail(pipeline_run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the my pipeline run detail.
- ▶ `async def abandon(pipeline_run_id: str, req: AbandonPipelineRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Manually move a stuck pipeline out of running/gated so any run that is
- ▶ `def list_pipelines()`
Library view: saved pipeline files on disk, mirroring GET `/workflows`.
- ▶ `def get_pipeline(name: str)`
Return the pipeline.

- ▶ `def save_pipeline(req: SavePipelineRequest)`
Save the pipeline.

`app/api/proposals.py` — 729 lines

EU Proposal Evidence and Reasoning System API.

- ▶ `class GraphRequest`
Pydantic model defining the GraphRequest shape.
- ▶ `class SourceVersionRequest`
Pydantic model defining the SourceVersionRequest shape.
- ▶ `class VerifyClaimsRequest`
Pydantic model defining the VerifyClaimsRequest shape.
- ▶ `class ConceptRequest`
Pydantic model defining the ConceptRequest shape.
- ▶ `class ApprovalRequest`
Pydantic model defining the ApprovalRequest shape.
- ▶ `class ApprovalDecisionRequest`
Pydantic model defining the ApprovalDecisionRequest shape.
- ▶ `class HorizonEvaluationRequest`
Pydantic model defining the HorizonEvaluationRequest shape.
- ▶ `class ProposalEvidenceFactoryRequest`
Pydantic model defining the ProposalEvidenceFactoryRequest shape.
- ▶ `class RenderProposalRequest`
Pydantic model defining the RenderProposalRequest shape.
- ▶ `class RenderDOCXProposalRequest`
Provides the RenderDOCXProposalRequest behaviour.
- ▶ `def _scope(user: CurrentUser) -> str`
Internal helper for the scope step.
- ▶ `def _services(request: Request) -> dict[str, Any]`
Internal helper for the services step.
- ▶ `def _store(request: Request, *, require_objects: bool=False)`
Internal helper for the store step.
- ▶ `def _graph_from_checkpoint(checkpoint: dict[str, Any]) -> ProposalGraph`
Internal helper for the graph from checkpoint step.
- ▶ `async def coverage(body: GraphRequest, user: CurrentUser=Depends(require_consultant))`
Compute the coverage.
- ▶ `async def review_run(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the review run.
- ▶ `async def register_source_version(proposal_id: str, source_id: str, body: SourceVersionRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Register the source version.
- ▶ `async def list_source_versions(proposal_id: str, request: Request, source_id: str | None=None, user: CurrentUser=Depends(require_consultant))`
List the source versions.
- ▶ `async def verify_claims(proposal_id: str, body: VerifyClaimsRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Verify the claims.
- ▶ `async def concept_alternatives(proposal_id: str, body: ConceptRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the concept alternatives.
- ▶ `async def request_approval(proposal_id: str, body: ApprovalRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the request approval.
- ▶ `async def decide_approval(proposal_id: str, approval_id: str, body: ApprovalDecisionRequest, request: Request, user: CurrentUser=Depends(require_consultant))`

Compute the decide approval.

- ▶ `async def list_approvals(proposal_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
List the approvals.
- ▶ `async def horizon_evaluation(proposal_id: str, body: HorizonEvaluationRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the horizon evaluation.
- ▶ `async def run_proposal_evidence_factory(proposal_id: str, body: ProposalEvidenceFactoryRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Verify already-fetched full text and return a citable evidence package.
- ▶ `async def render_proposal(proposal_id: str, body: RenderProposalRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Render a proposal and persist both sanitised HTML and final PDF.
- ▶ `async def render_proposal_docx(proposal_id: str, body: RenderDOCXProposalRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Render a proposal as editable DOCX and persist its sanitised source.

app/api/rag_agents.py — 159 lines

Saved RAG Agent management and direct test query API.

- ▶ `class RAGAgentCreate`
Pydantic model defining the RAGAgentCreate shape.
- ▶ `class RAGQueryRequest`
Pydantic model defining the RAGQueryRequest shape.
- ▶ `def _scope(user: CurrentUser) -> str`
Internal helper for the scope step.
- ▶ `async def create_agent(payload: RAGAgentCreate, request: Request, user: CurrentUser=Depends(require_permission('rag:write')))`
Create the agent.
- ▶ `async def list_agents(request: Request, search: str | None=Query(default=None), user: CurrentUser=Depends(require_permission('knowledge:read')))`
List the agents.
- ▶ `async def get_agent(rag_agent_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read')))`
Return the agent.
- ▶ `async def query_agent(rag_agent_id: str, payload: RAGQueryRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query')))`
Query the agent.

app/api/research.py — 32 lines

Research-feature discovery endpoints.

- ▶ `async def list_scientific_skills(request: Request, user: CurrentUser=Depends(require_consultant))`
List the scientific skills.

app/api/retrieval.py — 272 lines

Retrieval Playground search, comparison and trace APIs.

- ▶ `class RetrievalSearchRequest`
Pydantic model defining the RetrievalSearchRequest shape.
`def to_query(self, owner_scope_id: str) -> RetrievalQuery`
Compute the to query.
- ▶ `class RetrievalCompareRequest`
Pydantic model defining the RetrievalCompareRequest shape.
- ▶ `async def presets(user: CurrentUser=Depends(require_permission('knowledge:read')))`

Return editable starting configurations; saving creates a profile.

- `def _scope(user: CurrentUser) -> str`
Internal helper for the scope step.
- `def _service(request: Request)`
Internal helper for the service step.
- `def _llm(request: Request, scope: str, node_id: str, collection_id: str | None=None)`
Internal helper for the llm step.
- `def _safe_query(value: str) -> str`
Internal helper for the safe query step.
- `async def search(payload: RetrievalSearchRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query')))`
Search the result.
- `async def compare(payload: RetrievalCompareRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query')))`
Compare the result.
- `async def list_traces(request: Request, limit: int=Query(default=100, ge=1, le=500), user: CurrentUser=Depends(require_permission('knowledge:read')))`
List the traces.
- `async def get_trace(retrieval_request_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read')))`
Return the trace.

app/api/run_chat.py — 286 lines

Ask AI about a run — a grounded Q&A chat scoped to one run's own data

- `class ChatMessage`
Pydantic model defining the ChatMessage shape.
- `class AskAboutRunRequest`
Pydantic model defining the AskAboutRunRequest shape.
- `def starter_questions(run_id: str) -> list[str]`
Compute the starter questions.
- `def _summarize_value(value: Any, limit: int=800) -> str`
Summarize the value.
- `def _node_configs_from_workflow_yaml(workflow_yaml: str | None) -> dict[str, dict[str, Any]]`
Internal helper for the node configs from workflow yaml step.
- `def _node_type_catalog() -> dict[str, dict[str, Any]]`
Internal helper for the node type catalog step.
- `def _build_run_context(run: dict[str, Any]) -> str`
Build the run context.
- `def _scope(user: CurrentUser) -> str`
Internal helper for the scope step.
- `async def get_run_chat(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Return the run chat.
- `async def ask_about_run(run_id: str, req: AskAboutRunRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the ask about run.

app/api/runs.py — 1219 lines

Runs module.

- `class BusinessActionRequest`
Pydantic model defining the BusinessActionRequest shape.
- `class FactCorrectionRequest`
Pydantic model defining the FactCorrectionRequest shape.

- ▶ `class AssignRunRequest`
Pydantic model defining the AssignRunRequest shape.
- ▶ `class RetryRunRequest`
Pydantic model defining the RetryRunRequest shape.
- ▶ `def _scope(user: CurrentUser) -> str`
Internal helper for the scope step.
- ▶ `async def my_runs(request: Request, limit: int=50, user: CurrentUser=Depends(require_consultant))`
Compute the my runs.
- ▶ `async def my_run_detail(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Compute the my run detail.
- ▶ `async def pending_gate(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
The human-review gate a paused run is currently waiting on, if any.
- ▶ `async def _load_run_and_spec(db, scope: str, run_id: str)`
The run document plus its parsed workflow spec, or a 404/400.
- ▶ `async def _pending_gate(db, scope: str, run_id: str, run: dict[str, Any])`
Internal helper for the pending gate step.
- ▶ `def _run_cost_entries(services: dict[str, Any], run_id: str, scope: str) -> list[dict[str, Any]]`
Per-call model/cost/latency for this run, or [] when unavailable.
- ▶ `async def run_business_projection(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Business View's data source — this run reshaped into business language.
- ▶ `async def run_business_narration(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Rephrase this work item's already-decided status in business language.
- ▶ `async def run_business_explanation(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
"Why?" — the facts and rules behind the handling decision (§20, §48).
- ▶ `async def run_business_technical_detail(run_id: str, activity_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
The technical layer behind one business activity (§5, §47).
- ▶ `async def run_business_action(run_id: str, body: BusinessActionRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Perform one typed Business View action (§53, §54).
- ▶ `async def correct_fact(run_id: str, body: FactCorrectionRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Overwrite one extracted fact on this run and mark what it fed as stale.
- ▶ `async def assign_run(run_id: str, body: AssignRunRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Record who a work item is assigned to — a plain annotation on the
- ▶ `def _hitl_config_fallback(checkpoint: dict[str, Any], node_id: str | None) -> dict[str, Any]`
Best-effort HITL gate fields straight from the workflow's own node
- ▶ `async def retry_failed_run(source_run_id: str, req: RetryRunRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Create a new attempt that reuses every completed node checkpoint.
- ▶ `async def pause_run(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Ask a running workflow to pause at its next node boundary.
- ▶ `async def resume_paused_run(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Resume a run paused by the run-history "pause" action.
- ▶ `async def restart_run(source_run_id: str, req: RetryRunRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Create a brand-new run of the same workflow and original inputs.
- ▶ `async def delete_run_endpoint(run_id: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Delete the run endpoint.

app/api/workflow_files.py — 217 lines

Authenticated multipart upload/download API for workflow input files.

- ▶ class ExtractWorkflowFileRequest
Pydantic model defining the ExtractWorkflowFileRequest shape.
- ▶ def _scope(user: CurrentUser) -> str
Internal helper for the scope step.
- ▶ async def capabilities(_user: CurrentUser=Depends(require_consultant))
Return the authoritative UI picker limits and accepted extensions.
- ▶ async def upload_workflow_input_files(files: Annotated[list[UploadFile], File(...)], request: Request, user: CurrentUser=Depends(require_consultant))
Store files and return stable references suitable for workflow state.
- ▶ async def extract_workflow_input_file(body: ExtractWorkflowFileRequest, request: Request, user: CurrentUser=Depends(require_consultant))
Convert one uploaded document into plain text for the HITL editor.
- ▶ async def download_workflow_input_file(key: str, request: Request, user: CurrentUser=Depends(require_consultant))
Download one uploaded input after enforcing the authenticated scope.

app/api/workflow_generation.py — 976 lines

Auto-create a workflow from a natural-language prompt, tested before it's

- ▶ class GenerationAttempt
Provides the GenerationAttempt behaviour.
- ▶ class GenerationResult
Provides the GenerationResult behaviour.
- ▶ class GenerateWorkflowRequest
Pydantic model defining the GenerateWorkflowRequest shape.
- ▶ def _field_type_label(schema: dict) -> str
A short type descriptor for one JSON-schema property, e.g. "string",
- ▶ def _node_type_catalog(type_names: list[str] | None=None) -> str
The catalog text embedded in the generation system prompt. With
- ▶ def _real_usage_snippet(type_name: str, manifest_entry: dict) -> str | None
Step 2's grounding for one node type: its own config, exactly as
- ▶ def _real_usage_examples(type_names: list[str], manifest: list[dict]) -> str
Step 2's text block: one real config snippet per candidate type that
- ▶ def _strip_code_fence(text: str) -> str
Internal helper for the strip code fence step.
- ▶ def _synthesize_sample_inputs(specs: dict[str, WorkflowInputSpec], provided: dict[str, Any] | None) -> tuple[dict[str, Any] | None, str | None]
Fill in placeholder values for a workflow's declared inputs so it can
- ▶ async def run_generation_pipeline(*, prompt: str, sample_inputs: dict[str, Any] | None, generate_yaml: Callable[[str, str | None, str | None], Awaitable[str]], static_check: Callable[[str], Awaitable[tuple[WorkflowPreflightReport, dict[str, WorkflowInputSpec]]]], execute: Callable[[str, dict[str, Any]], Awaitable[dict[str, Any]]]) -> GenerationResult
The repair-loop control flow, dependency-injected so it's testable
- ▶ def build_llm_yaml_generator(llm: Any, services: dict[str, Any], scope: str, *, mode: str='generate') -> Callable[[str, str | None, str | None], Awaitable[str]]
Build a `generate\_yaml(base\_prompt, prior\_yaml, feedback) -> yaml text`
- ▶ async def generate_workflow_endpoint(req: GenerateWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant))
Generate the workflow endpoint.

app/api/workflows.py — 1129 lines

Workflows module.

- ▶ `class RunRequest`
Pydantic model defining the RunRequest shape.
- ▶ `class ValidateWorkflowRequest`
Pydantic model defining the ValidateWorkflowRequest shape.
- ▶ `class AutofixWorkflowRequest`
Pydantic model defining the AutofixWorkflowRequest shape.
- ▶ `class AutofixWorkflowResponse`
Pydantic model defining the AutofixWorkflowResponse shape.
- ▶ `class ResumeRequest`
Pydantic model defining the ResumeRequest shape.
- ▶ `class SubprocessCallbackRequest`
Pydantic model defining the SubprocessCallbackRequest shape.
- ▶ `class SaveWorkflowRequest`
Pydantic model defining the SaveWorkflowRequest shape.
- ▶ `class SaveDraftRequest`
Pydantic model defining the SaveDraftRequest shape.
- ▶ `async def _reserve_run_id(services: dict[str, Any], *, run_id: str, session_id: str) -> str`
Claim a client-supplied run_id for one tenant, idempotently.
- ▶ `def _scope(user: CurrentUser, body_session: str | None) -> str`
The value a run is scoped to for storage + retrieval.
- ▶ `async def list_workflows()`
List the workflows.
- ▶ `def list_node_types(user: CurrentUser=Depends(require_admin))`
The Workflow Builder UI calls this to populate the palette and forms.
- ▶ `async def allowed_models(node_type: str)`
Returns the default allowed model list for a node type.
- ▶ `def _preflight_http_detail(report: WorkflowPreflightReport) -> dict[str, Any]`
Internal helper for the preflight http detail step.
- ▶ `async def validate_workflow(req: ValidateWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Validate a workflow without running a node or consuming LLM tokens.
- ▶ `async def _run_preflight(yaml_text: str, req: AutofixWorkflowRequest, services: dict[str, Any]) -> WorkflowPreflightReport`
Same branching `validate\_workflow` uses, so Auto-fix and Preflight
- ▶ `async def autofix_workflow(req: AutofixWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Best-effort repair of a workflow's preflight errors: deterministic
- ▶ `async def get_preflight_stats(request: Request, since_hours: int | None=None, user: CurrentUser=Depends(require_admin))`
Platform-wide preflight reliability rollout across every /validate,
- ▶ `async def run(req: RunRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Run the result.
- ▶ `async def resume(run_id: str, req: ResumeRequest, request: Request, user: CurrentUser=Depends(require_consultant))`
Resume doesn't need services for execution — the cached compiled graph in
- ▶ `async def subprocess_callback(token: str, req: SubprocessCallbackRequest, request: Request)`
Deliver a Subprocess child's terminal result to its waiting parent.
- ▶ `def list_workflows()`
Library view in the UI calls this to render the workflow collection.
- ▶ `def get_workflow(name: str)`
Builder calls this to load an existing workflow for editing.
- ▶ `def get_workflow_detail(name: str)`
Workflow Library details panel calls this to refresh `library` and
- ▶ `async def get_workflow_stats(name: str, request: Request, user: CurrentUser=Depends(require_consultant))`
Library "Runs and performance" tab. Session-scoped like Run History

- ▶ `def save_workflow(req: SaveWorkflowRequest)`
Builder calls this on Save. Validates the YAML parses to a WorkflowSpec
- ▶ `def _pipelines_referencing_workflow(name: str) -> list[str]`
Names of every pipeline whose stages reference this workflow. Deleting
- ▶ `def delete_workflow(name: str, user: CurrentUser=Depends(require_consultant))`
Permanently removes a workflow's YAML together with its autosave
- ▶ `def _builder_name(name: str) -> str`
Internal helper for the builder name step.
- ▶ `def _builder_version_id(version_id: str) -> str`
Internal helper for the builder version id step.
- ▶ `def save_workflow_draft(name: str, req: SaveDraftRequest)`
Autosave a recoverable draft. Never compiled/preflighted — a draft may
- ▶ `def get_workflow_draft(name: str)`
Builder calls this on load to offer 'Resume draft' for a newer autosave.
- ▶ `def delete_workflow_draft(name: str)`
Delete the workflow draft.
- ▶ `def list_workflow_versions(name: str)`
List the workflow versions.
- ▶ `def get_workflow_version(name: str, version_id: str)`
Return the workflow version.
- ▶ `def restore_workflow_version(name: str, version_id: str)`
Restoring an old version is Save-equivalent: it must pass the same
- ▶ `def _sse_message(*, event: str, data: dict[str, Any], event_id: int | None=None) -> str`
Internal helper for the sse message step.
- ▶ `async def stream_run_events(run_id: str, request: Request, last_event_id: int | None=None, default=None, alias='Last-Event-ID'), user: CurrentUser=Depends(require_consultant))`
Authenticated Server-Sent Events stream for one workflow run.
- ▶ `def get_file(key: str, request: Request, download: bool=False)`
Stream a file from object storage by key (used by the Output Viewer).

app (3 modules)

app/__init__.py — 0 lines

(no summary)

app/config.py — 656 lines

Config module.

► class Settings

Provides the Settings behaviour.

def workflow_file_max_bytes(self) -> int

The workflow file max bytes.

def workflow_file_max_total_bytes(self) -> int

The workflow file max total bytes.

def max_request_body_bytes(self) -> int

The max request body bytes.

def validate_production_security(self) -> 'Settings'

Refuse to boot production with development security settings.

def moonshot_api_key(self) -> str

Kimi web search and vision (app/tools/web_io.py, vision_io.py)

def kimi_api_base_url(self) -> str

The kimi api base url.

def required_readiness_services(self) -> tuple[str, ...]

The required readiness services.

def resolved_paper_search_mcp_path(self) -> Path | None

The resolved paper search mcp path.

def resolved_scientific_skills_path(self) -> Path

The resolved scientific skills path.

def allowed_scientific_skills(self) -> tuple[str, ...]

The allowed scientific skills.

def allowed_cors_origins(self) -> tuple[str, ...]

The allowed cors origins.

def allowed_hosts(self) -> tuple[str, ...]

The allowed hosts.

► def _csv_values(value: str) -> tuple[str, ...]

Internal helper for the csv values step.

► def _redis_url_has_password(url: str) -> bool

Return whether a Redis URL contains a non-empty password component.

► def _valid_http_url(value: str) -> bool

Internal helper for the valid http url step.

► def _is_placeholder(value: str) -> bool

Return whether placeholder.

app/main.py — 725 lines

FastAPI application entry point for Eurskem AI — Agentic Workflow Platform.

► async def lifespan(app: FastAPI)

Compute the lifespan.

db (3 modules)

app/db/__init__.py — 0 lines

(no summary)

app/db/migrations.py — 266 lines

Versioned, idempotent MongoDB data migrations.

- ▶ `class MigrationError`
Raised when the database cannot be brought to the required schema.
- ▶ `class Migration`
Provides the Migration behaviour.
- ▶ `async def _set_missing(collection: Any, field: str, value: Any) -> None`
Set the missing.
- ▶ `async def _backfill_run_documents_v1(db: Any) -> None`
Give legacy run and checkpoint documents the explicit v1 shape.
- ▶ `async def _backfill_knowledge_resources_v2(db: Any) -> None`
Expand legacy ``collections`` (CollectionConfig) records with the
- ▶ `async def _acquire_lock(collection: Any, *, owner: str, wait_timeout_seconds: float) -> None`
Acquire the lock.
- ▶ `async def _renew_lock(collection: Any, *, owner: str) -> None`
Renew the lock.
- ▶ `async def run_migrations(db: Any, *, owner: str | None=None, wait_timeout_seconds: float=60.0) -> list[str]`
Run every pending migration and return the applied migration ids.

app/db/mongo.py — 253 lines

Async MongoDB client and manifest collection.

- ▶ `class MongoClient`
Async Mongo wrapper. One per app instance.
 - `def _ensure_client(self) -> AsyncIOMotorClient`
Ensure the client.
 - `def manifests(self) -> AsyncIOMotorCollection`
The manifests collection, opened lazily.
 - `async def close(self) -> None`
Close the result.
 - `def scorecards(self) -> AsyncIOMotorCollection`
The eval scorecards collection, opened lazily.
 - `def collections(self)`
The collections.
 - `async def ensure_indexes(self) -> None`
Create the indexes the pipeline relies on. Idempotent.
 - `async def upsert_manifest(self, manifest: dict[str, Any]) -> str`
Insert or replace a manifest by `_id`. Returns the `_id`.
 - `async def get_manifest(self, manifest_id: str) -> dict[str, Any] | None`
Fetch a single manifest by `_id`. Returns None if not found.
 - `async def list_manifests(self, status: str | None=None, limit: int=100) -> list[dict[str, Any]]`
List manifests, optionally filtered by status, newest first.
 - `async def mark_status(self, manifest_id: str, status: str) -> None`
Update only the status field of an existing manifest.
 - `async def delete_manifest(self, manifest_id: str) -> bool`

Delete a manifest. Returns True if a record was deleted.

```
async def save_scorecard(self, scorecard: dict[str, Any]) -> str
```

Insert one eval scorecard. Returns the inserted id as a string.

```
async def list_scorecards(self, workflow_name: str | None=None, limit: int=20) ->
```

```
list[dict[str, Any]]
```

List scorecards newest-first, optionally for one workflow.

```
▶ def build_manifest(minio_key: str, original_filename: str, source_format: str, byte_size: int,  
chunk_ids: list[str], metadata: dict[str, Any] | None=None, status: str='completed') -> dict[str,  
Any]
```

Construct a manifest dict in the canonical shape.

```
▶ def get_mongo_client() -> MongoClient
```

Lazily-constructed module-level client.

evaluation (6 modules)

app/evaluation/__init__.py — 29 lines

Evaluation package: LLM-as-a-Judge harness.

app/evaluation/golden_set.py — 50 lines

Golden set loading.

- ▶ class GoldenExample
Pydantic model defining the GoldenExample shape.
- ▶ def load_golden_set(path: str | Path) -> list[GoldenExample]
Load a JSONL golden set. Skips blank lines; fails loud on malformed rows.

app/evaluation/judge.py — 198 lines

LLM-as-a-Judge.

- ▶ class _JudgeVerdict
Provider-facing schema for one criterion.
- ▶ class LLMJudge
Scores an answer on the four RAG criteria via the LLM gateway.
async def _score_one(self, *, criterion: str, question: str, answer: str, context: str, reference: str) -> CriterionScore
Score the one.
async def score_one(self, criterion: str, *, question: str, answer: str, context: str, reference: str='') -> CriterionScore
Public single-criterion API used by focused evaluation runs.
async def score_all(self, *, question: str, answer: str, context: str, reference: str='') -> list[CriterionScore]
Score one answer on all four criteria, one independent call each.

app/evaluation/models.py — 87 lines

Evaluation data models.

- ▶ class CriterionScore
One judged criterion for one answer.
- ▶ class ExampleResult
All criterion scores for a single golden example.
def mean(self) -> float
The mean.
- ▶ class Scorecard
Aggregate result of evaluating one workflow over one golden set.
def per_criterion_mean(self) -> dict[str, float]
Backward-compatible read alias for earlier API/UI consumers.
- ▶ def _now_iso() -> str
Internal helper for the now iso step.

app/evaluation/runner.py — 104 lines

Evaluation runner.

- ▶ async def _eval_one(ex: GoldenExample, judge: LLMJudge, produce_answer: ProduceAnswer) -> ExampleResult

Internal helper for the eval one step.

```
▶ async def run_eval(*, workflow_name: str, examples: list[GoldenExample], judge: LLMJudge,
produce_answer: ProduceAnswer) -> Scorecard
```

Run the whole golden set through produce_answer + judge, aggregate.

`app/evaluation/workflow_golden.py` — 320 lines

Golden-case evaluation for a real end-to-end workflow (not a raw generation).

```
▶ class WorkflowGoldenCase
    Pydantic model defining the WorkflowGoldenCase shape.
▶ class FieldCheck
    Pydantic model defining the FieldCheck shape.
▶ class WorkflowCaseResult
    Pydantic model defining the WorkflowCaseResult shape.
▶ class ModelComparisonResult
    Pydantic model defining the ModelComparisonResult shape.
▶ class ModelOverrideGateway
    Forces every LLM call a run makes onto one candidate model, regardless
    def with_context(self, **kwargs: Any) -> 'ModelOverrideGateway'
        Compute the with context.
    async def complete(self, *, model: str | None=None, **kwargs: Any)
        Complete the result.
    async def complete_structured(self, *, model: str | None=None, **kwargs: Any)
        Complete the structured.
    async def chat_with_tools(self, *, model: str | None=None, **kwargs: Any)
        Compute the chat with tools.
▶ def load_workflow_golden_set(path: str | Path) -> list[WorkflowGoldenCase]
    Load the workflow golden set.
▶ def _actual_value(result: dict[str, Any], field: str) -> Any
    Internal helper for the actual value step.
▶ async def run_workflow_golden_case(case: WorkflowGoldenCase, *, model: str, services: dict[str,
Any], run_id: str) -> WorkflowCaseResult
    Run the workflow golden case.
▶ async def run_golden_set_with_model(cases: list[WorkflowGoldenCase], *, model: str, services:
dict[str, Any], run_id_prefix: str) -> ModelComparisonResult
    Run the golden set with model.
▶ def recommend_model(comparisons: list[ModelComparisonResult]) -> dict[str, Any] | None
    Highest pass rate wins; ties broken by lower cost, then lower latency.
```

evidence (6 modules)

app/evidence/__init__.py — 29 lines

Proposal-grade evidence gathering and citation verification.

app/evidence/claim_verification.py — 113 lines

Best-effort secondary verification for evidence content with no URL.

- ▶ `class ClaimVerificationResult`
Pydantic model defining the ClaimVerificationResult shape.
- ▶ `def _format_results(results: list[dict[str, Any]]) -> str`
Format the results.
- ▶ `async def verify_claim(content: str, *, source_name: str, llm: Any, web_search: Any) -> ClaimVerificationResult`
Verify the claim.

app/evidence/database_models.py — 225 lines

Typed provenance contracts for structured and internal proposal evidence.

- ▶ `class StructuredDatasetQuery`
One bounded, reproducible public-database retrieval contract.
`def _safe_identifier(cls, value: str) -> str`
Internal helper for the safe identifier step.
`def _safe_period(cls, value: str | None) -> str | None`
Internal helper for the safe period step.
`def _safe_filters(cls, value: dict[str, list[str]]) -> dict[str, list[str]]`
Internal helper for the safe filters step.
- ▶ `class CountReconciliation`
Pydantic model defining the CountReconciliation shape.
- ▶ `class StructuredDataEvidenceRecord`
One value decoded from a versioned official-database response.
- ▶ `class DatasetRetrievalAudit`
Pydantic model defining the DatasetRetrievalAudit shape.
- ▶ `class InternalEvidenceRecord`
Exact-passage-matched fact from an internal or partner source.
- ▶ `class InternalEvidenceReviewItem`
Pydantic model defining the InternalEvidenceReviewItem shape.

app/evidence/identifiers.py — 362 lines

Canonical scholarly identifier extraction and work identity.

- ▶ `class ResolvedWork`
Canonical identity for one scholarly work, as far as it is known.
`def as_dict(self) -> dict[str, str]`
Only the populated identifier fields, for storage on a candidate.
`def has_strong_identifier(self) -> bool`
The has strong identifier.
`def is_peer_reviewed(self) -> bool`
True ONLY when resolved metadata positively supports it.
- ▶ `def normalise_doi_value(value: Any) -> str | None`
Lowercased bare DOI from anything that might contain one.
- ▶ `def _arxiv_from(text: str) -> str | None`

Internal helper for the arxiv from step.

- ▶ `def extract_identifiers(url: str | None=None, *, metadata: dict[str, Any] | None=None, text: str | None=None) -> ResolvedWork`
Best-effort offline identifier extraction.
- ▶ `def normalise_title(title: str | None) -> str`
Punctuation/whitespace-insensitive title key.
- ▶ `def work_identity(work: ResolvedWork | None, *, title: str | None=None, fallback: str | None=None) -> str`
The single cross-lane dedup key for one work.
- ▶ `def classify_authority(url: str | None, work: ResolvedWork) -> tuple[str, str]`
“(source_label, authority)” from what is actually known.

app/evidence/models.py — 479 lines

Typed contracts for the EU proposal evidence factory.

- ▶ `class EvidencePolicy`
Hard evidence rules applied before drafting may use a citation.
- ▶ `class SearchAuditRecord`
Pydantic model defining the SearchAuditRecord shape.
- ▶ `class CandidateSource`
A discovered record. It is explicitly not verified evidence.
- ▶ `class FullTextDocument`
Immutable, fetched source version available for passage retrieval.
- ▶ `class RetrievedPassage`
Pydantic model defining the RetrievedPassage shape.
- ▶ `class ClaimEvidenceLink`
Pydantic model defining the ClaimEvidenceLink shape.
- ▶ `class CitationRegistryEntry`
Pydantic model defining the CitationRegistryEntry shape.
- ▶ `class VerifiedClaim`
Pydantic model defining the VerifiedClaim shape.
- ▶ `class RejectedCandidate`
Pydantic model defining the RejectedCandidate shape.
- ▶ `class EvidenceGap`
Pydantic model defining the EvidenceGap shape.
- ▶ `class EvidenceQAReport`
Pydantic model defining the EvidenceQAReport shape.
- ▶ `class ProposalEvidencePackage`
Pydantic model defining the ProposalEvidencePackage shape.
- ▶ `def coerce_typed_list_field(value: Any, model: type[_ModelT], field_name: str) -> str | list[_ModelT]`
Accept a node's records as a list, JSON text, or empty text.

app/evidence/retrieval.py — 652 lines

Deterministic helpers for discovery, full-text handling, and passages.

- ▶ `def utc_now() -> str`
Compute the utc now.
- ▶ `def stable_id(prefix: str, *parts: Any, length: int=16) -> str`
Compute the stable id.
- ▶ `def normalise_doi(value: Any) -> str | None`
Compute the normalise doi.
- ▶ `def parse_authors(value: Any) -> list[str]`
Parse the authors.
- ▶ `def parse_year(value: Any) -> int | None`

```

    Parse the year.
    ▶ def first_value(data: dict[str, Any], *keys: str) -> Any
      Compute the first value.
    ▶ def parse_mcp_payload(raw: Any) -> Any
      Parse the mcp payload.
    ▶ def papers_from_payload(raw: Any) -> list[dict[str, Any]]
      Compute the papers from payload.
    ▶ def source_errors_from_payload(raw: Any) -> dict[str, str]
      Per-source failures from a multi-source search payload (e.g.
    ▶ def candidate_from_paper(paper: dict[str, Any], *, claim_id: str, query: str, purpose: str,
      source_hint: str | None=None, discovery_lane: str | None=None) -> CandidateSource
      Compute the candidate from paper.
    ▶ def _normalise_title_for_identity(title: str) -> str
      Collapse cosmetic title differences that otherwise defeat dedup.
    ▶ def candidate_work_identity(candidate: CandidateSource) -> str
      Cross-lane work identity for one candidate.
    ▶ def deduplicate_candidates(candidates: list[CandidateSource]) -> list[CandidateSource]
      Deduplicate the same work for one claim, across queries AND lanes.
    ▶ def formatted_citation(candidate: CandidateSource) -> str
      Compute the formatted citation.
    ▶ def download_path_from_payload(raw: Any) -> str | None
      Download the path from payload.
    ▶ def validate_downloaded_pdf(raw_path: str | None, *, allowed_root: Path, max_bytes: int) -> Path
      Validate the downloaded pdf.
    ▶ def _tokens(text: str) -> list[str]
      Internal helper for the tokens step.
    ▶ def _page_windows(text: str, size: int=1800, overlap: int=250) -> list[str]
      Internal helper for the page windows step.
    ▶ def retrieve_passages(claim: str, pages: list[dict[str, Any]], *, document_id: str, limit: int=8)
-> list[RetrievedPassage]
      Page-aware lexical retrieval before semantic stance verification.

```

ingestion (14 modules)

app/ingestion/__init__.py — 0 lines

(no summary)

app/ingestion/chunker.py — 403 lines

Structure-aware recursive chunker.

- ▶ `class Chunk`
One unit of text ready for embedding.
`def embedding_content(self) -> str`
The text that should actually be embedded for this chunk.
- ▶ `class ChunkConfig`
Tunable chunking parameters.
- ▶ `def _get_tokenizer(name: str) -> tiktoken.Encoding`
Return the tokenizer.
- ▶ `def _split_sections(text: str) -> list[str]`
Split on top-level section headers. Returns list of section bodies.
- ▶ `def _split_paragraphs(text: str) -> list[str]`
Split on blank lines. Returns non-empty paragraph strings.
- ▶ `def _split_sentences(text: str) -> list[str]`
Split on sentence boundaries. Best-effort, no NLP library.
- ▶ `def _hard_split(tokens: list[int], target: int, max_size: int, encoding: tiktoken.Encoding) -> list[str]`
Last-resort: split a token list at fixed boundaries.
- ▶ `def _pack_pieces(pieces: list[str], encoding: tiktoken.Encoding, config: ChunkConfig) -> list[str]`
Greedy packer.
- ▶ `def _split_oversized(text: str, encoding: tiktoken.Encoding, config: ChunkConfig) -> list[str]`
Recursive descent: paragraphs → sentences → hard cut.
- ▶ `def _apply_overlap(chunks: list[str], encoding: tiktoken.Encoding, overlap_tokens: int) -> list[str]`
Prepend the last `overlap\_tokens` of chunk N-1 onto the start of chunk N.
- ▶ `def chunk_text(text: str, config: ChunkConfig | None=None) -> list[str]`
Chunk a raw text string. Returns list of chunk strings (no metadata).
- ▶ `def chunk_document(doc: ExtractedDocument, config: ChunkConfig | None=None, chunk_id_prefix: str | None=None) -> list[Chunk]`
Chunk a full ExtractedDocument into Chunk objects with metadata.
- ▶ `def main() -> None`
Quick CLI: ``python -m app.ingestion.chunker <path>``.

app/ingestion/cli.py — 92 lines

CLI entry point for ingestion. Invoked via ``make ingest FILE=path``.

- ▶ `def _parse_metadata(raw: list[str]) -> dict[str, str]`
Parse repeated `--meta key=value` flags into a dict.
- ▶ `async def _amain(args: argparse.Namespace) -> None`
Internal helper for the `amain` step.
- ▶ `def main() -> None`
Compute the main.

app/ingestion/collections.py — 115 lines

Per-collection configuration: the controlled vocabulary that makes the

- ▶ `class CollectionConfig`
Declares one use case's document taxonomy.
`def validate_doc_type(self, dt: str) -> None`
Raise if `dt` is not in this collection's declared vocabulary.
`def validate_doc_types(self, dts: list[str]) -> None`
Validate a list (retrieval filters carry a list of doc_types).
- ▶ `class CollectionRegistry`
Loads/saves `CollectionConfig` via the `MongoClient`'s `collections`
`async def get(self, collection_id: str) -> CollectionConfig`
Return the result.
`async def upsert(self, cfg: CollectionConfig) -> None`
Upsert the result.
`async def list_all(self) -> list[CollectionConfig]`
List the all.

app/ingestion/contracts.py — 122 lines

Protocols and the stage registry for the ingestion pipeline.

- ▶ `class DocumentParser`
Provides the `DocumentParser` behaviour.
`async def parse(self, path: Path, *, config: dict[str, Any]) -> ExtractedDocument`
- ▶ `class ChunkingStrategy`
Provides the `ChunkingStrategy` behaviour.
`async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]`
- ▶ `class ChunkEnricher`
Provides the `ChunkEnricher` behaviour.
`async def enrich(self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]) -> list[Chunk]`
- ▶ `class EmbeddingProvider`
Provides the `EmbeddingProvider` behaviour.
`async def embed(self, texts: list[str]) -> list[list[float]]`
- ▶ `class SearchIndex`
Provides the `SearchIndex` behaviour.
`async def index(self, objects: list[dict[str, Any]], vectors: list[list[float]]) -> int`
Write ``objects`` (with their parallel ``vectors``) and return the
- ▶ `class UnknownStrategyError`
Exception raised for the `UnknownStrategyError` case.
- ▶ `class StageRegistry`
Name -> strategy lookup for every ingestion stage kind.
`def parser(self, name: str, instance: DocumentParser) -> None`
Compute the parser.
`def chunker(self, name: str, instance: ChunkingStrategy) -> None`
Compute the chunker.
`def enricher(self, name: str, instance: ChunkEnricher) -> None`
Compute the enricher.
`def get_parser(self, name: str) -> DocumentParser`
Return the parser.
`def get_chunker(self, name: str) -> ChunkingStrategy`
Return the chunker.

app/ingestion/coordinator.py — 123 lines

Starts background ingestion work and recovers unfinished jobs.

- ▶ `class IngestionCoordinator`
Provides the IngestionCoordinator behaviour.
 - `def submit(self, job: IngestionJob) -> None`
Schedule ``job`` to run on the event loop. Returns immediately.
 - `async def recover(self) -> int`
Resubmit every job left non-terminal by an interrupted process.
 - `async def _materialize_sources(self, job: IngestionJob) -> list[Path]`
Download this job's already-uploaded sources to a local temp dir.
 - `async def _run_safely(self, job: IngestionJob) -> None`
Run the safely.

app/ingestion/embedder.py — 151 lines

Embed chunks into vectors using OpenAI.

- ▶ `class EmbedderConfig`
Tunable embedder parameters.
- ▶ `class Embedder`
Async OpenAI embedding client.
 - `async def embed(self, texts: list[str]) -> list[list[float]]`
Embed a list of texts. Returns vectors in the same order.
 - `async def _embed_batch_with_retry(self, batch: list[str]) -> list[list[float]]`
Single batch with tenacity-managed retry.
- ▶ `def get_embedder() -> Embedder`
Lazily-constructed module-level embedder.

app/ingestion/embedding_catalog.py — 184 lines

Embedding models offered when building an Index.

- ▶ `class EmbeddingModelChoice`
Provides the EmbeddingModelChoice behaviour.
- ▶ `def embedding_model_catalog() -> list[dict[str, object]]`
Compute the embedding model catalog.
- ▶ `def _looks_multilingual(sample: str) -> bool`
Internal helper for the looks multilingual step.
- ▶ `def select_embedding_model(*, doc_types: list[str] | None=None, document_count: int=0, total_bytes: int=0, sample_text: str='') -> tuple[EmbeddingModelChoice, str]`
Deterministically pick an embedding model for a corpus.

app/ingestion/extractor.py — 621 lines

PDF, Office, plain text, and Markdown extractors.

- ▶ `class ExtractedUnit`
One logical unit of an extracted document.
- ▶ `class ExtractedDocument`
A whole document, broken into ordered units, with metadata.
 - `def full_text(self) -> str`
All units joined with double newlines. Useful for whole-doc embeddings.
- ▶ `class Extractor`
Anything callable with `.extract(path)` returning `ExtractedDocument`.
 - `def extract(self, path: Path) -> ExtractedDocument`
- ▶ `class PdfExtractor`

Extract text from a PDF, one ExtractedUnit per page.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class XlsxExtractor

Extract data from an Excel workbook, one ExtractedUnit per worksheet.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class DocxExtractor

Extract text from a Word document.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class PptxExtractor

Extract text from a PowerPoint deck, one ExtractedUnit per slide.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class PlainTextExtractor

Extract any plain-text file. Used for .txt and (with language metadata) code.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class MarkdownExtractor

Extract text from a Markdown file.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class DelimitedTextExtractor

Extract rows from a CSV/TSV file, one ExtractedUnit per row block.

```
def extract(self, path: Path) -> ExtractedDocument
    Extract the result.
```

► class UnsupportedFormatError

Raised when no extractor is registered for a given file extension.

► def _is_number(value: str) -> bool

Return whether number.

► def get_extractor(path: Path) -> Extractor

Return the right extractor for the file's extension.

► def supported_extensions() -> list[str]

Sorted list of supported file extensions. Useful for UI / API surfaces.

► def main() -> None

Quick CLI: `python -m app.ingestion.extractor path/to/file.<ext>`.

app/ingestion/indexes.py — 56 lines

Narrow Weaviate index writer used by ingestion.

► class WeaviateSearchIndex

Satisfies :class:`app.ingestion.contracts.SearchIndex` for one physical

```
async def _ensure_schema(self) -> None
    Ensure the schema.
```

```
async def index(self, objects: list[dict[str, Any]], vectors: list[list[float]]) -> int
    Compute the index.
```

app/ingestion/jobs.py — 477 lines

Persistent, stage-visible Knowledge Studio ingestion jobs.

► class IngestionCancelled

Exception raised for the IngestionCancelled case.

► class IngestionJobRunner

Runs without a second queue platform; progress lives in Mongo.


```

def _embedder_for(self, embedding_profile: Any) -> EmbeddingProvider
    Embedder matching an Embedding Profile, cached per model+dimensions.
async def _raise_if_cancelled(self, job: IngestionJob) -> None
    Raise the if cancelled.
async def run(self, job: IngestionJob, paths: list[Path], *, metadata: dict[str, Any] |
None=None) -> IngestionJob
    Run the result.

```

app/ingestion/pipeline.py — 215 lines

End-to-end document ingestion pipeline.

- ▶ **class IngestionResult**
What `ingest\_file` returns. Mirrors the manifest stored in Mongo.
- ▶ **async def ingest_file(path: Path, metadata: dict[str, Any] | None=None, chunk_config: ChunkConfig | None=None, object_store: ObjectStore | None=None, embedder: Embedder | None=None, weaviate: WeaviateClient | None=None, mongo: MongoClient | None=None, collection_config: CollectionConfig | None=None) -> IngestionResult**
Ingest one document end-to-end. Idempotent.

app/ingestion/strategies.py — 535 lines

Built-in parser, chunking and enrichment strategies.

- ▶ **class StandardDocumentParser**
Provides the StandardDocumentParser behaviour.
async def parse(self, path: Path, *, config: dict[str, Any]) -> ExtractedDocument
Parse the result.
- ▶ **class LayoutAwareDocumentParser**
Use reliable native page/slide/sheet/heading structure when available.
- ▶ **class StructureAwareDocumentParser**
Provides the StructureAwareDocumentParser behaviour.
async def parse(self, path: Path, *, config: dict[str, Any]) -> ExtractedDocument
Parse the result.
- ▶ **class OcrFallbackDocumentParser**
Fall back to reading the page as an image when there is no text layer.
async def parse(self, path: Path, *, config: dict[str, Any]) -> ExtractedDocument
Parse the result.
- ▶ **class VisionAugmentedDocumentParser**
Add vision-model transcriptions of figures, charts and image tables.
async def parse(self, path: Path, *, config: dict[str, Any]) -> ExtractedDocument
Parse the result.
- ▶ **class RecursiveChunkingStrategy**
Provides the RecursiveChunkingStrategy behaviour.
async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]
Chunk the result.
- ▶ **class FixedTokenChunkingStrategy**
Provides the FixedTokenChunkingStrategy behaviour.
async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]
Chunk the result.
- ▶ **class StructureAwareChunkingStrategy**
Provides the StructureAwareChunkingStrategy behaviour.
async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]
Chunk the result.

- ▶ `class ParentChildChunkingStrategy`
Provides the `ParentChildChunkingStrategy` behaviour.
`async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]`
Chunk the result.
- ▶ `class ContextualChunkingStrategy`
Provides the `ContextualChunkingStrategy` behaviour.
`async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]`
Chunk the result.
- ▶ `class SentenceWindowChunkingStrategy`
Provides the `SentenceWindowChunkingStrategy` behaviour.
`async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]`
Chunk the result.
- ▶ `class SemanticChunkingStrategy`
Lightweight semantic-boundary strategy using paragraph topic overlap.
`async def chunk(self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str) -> list[Chunk]`
Chunk the result.
- ▶ `class MetadataContextEnricher`
Provides the `MetadataContextEnricher` behaviour.
`async def enrich(self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]) -> list[Chunk]`
Compute the enrich.
- ▶ `def _chunk_config(config: dict[str, Any]) -> ChunkConfig`
Chunk the config.
- ▶ `def _terms(text: str) -> set[str]`
Internal helper for the terms step.
- ▶ `def build_stage_registry() -> StageRegistry`
Build the stage registry.

app/ingestion/vision.py — 292 lines

Vision-model description of PDF pages carrying figures, charts and tables.

- ▶ `class PageDescription`
Provides the `PageDescription` behaviour.
- ▶ `class PdfPageVisionDescriber`
Render PDF pages and describe their visual content with a vision model.
`def provider(self) -> str`
The provider.
`def model(self) -> str`
The model.
`def available(self) -> bool`
Compute the available.
`def unavailable_reason(self) -> str`
Compute the unavailable reason.
`async def describe_pages(self, path: Path, page_indexes: list[int], *, prompt: str | None=None) -> dict[int, PageDescription]`
Describe the given pages. Pages that fail are simply omitted.
`async def _describe(self, index: int, png: bytes, prompt: str) -> PageDescription`
Internal helper for the describe step.
`def _build_client(self) -> Any`
Build the client.
- ▶ `def _can_render() -> bool`
Return whether render.

- ▶ `def _render_pages(path: Path, indexes: list[int], scale: float) -> dict[int, bytes]`
Render the given 0-based page indexes to PNG bytes.
- ▶ `def pages_with_visual_content(path: Path, *, min_images: int=1) -> list[int]`
0-based indexes of PDF pages that carry figures, images or table rules.

integrations (3 modules)

app/integrations/__init__.py — 0 lines

(no summary)

app/integrations/external_action.py — 284 lines

ExternalActionService — the one place an outbound REST/webhook call goes through.

- ▶ `class ExternalActionError`
A call failed in a way worth reporting to a workflow author.
- ▶ `class ExternalActionService`
Provides the ExternalActionService behaviour.
`async def call(self, *, run_id: str, node_id: str, safety_class: str, method: str, url: str, headers: dict[str, str], body: Any, timeout_seconds: float, approval_satisfied: bool) -> dict[str, Any]`
Compute the call.
`def _replay_or_refuse(self, existing: dict[str, Any], key: str) -> dict[str, Any]`
Internal helper for the replay or refuse step.
- ▶ `def _parse_body(response: httpx.Response) -> Any`
Parse the body.
- ▶ `def get_external_action_service() -> ExternalActionService`
Return the external action service.

app/integrations/operations.py — 163 lines

Durable record of attempted external side effects.

- ▶ `class OperationInFlight`
An identical side effect is already in flight, or ended ambiguously.
- ▶ `class AmbiguousOperationFailure`
An external call failed without a definitive outcome.
- ▶ `class ExternalOperationLedger`
Provides the ExternalOperationLedger behaviour.
`async def find(self, key: str) -> dict[str, Any] | None`
Find the result.
`async def reserve(self, key: str, record: dict[str, Any]) -> dict[str, Any] | None`
Claim the key. Returns the existing record if one is already there.
`async def complete(self, key: str, outcome: dict[str, Any]) -> None`
Complete the result.
`async def mark_ambiguous(self, key: str, error: str) -> None`
Mark the ambiguous.
`async def release(self, key: str, error: str) -> None`
Drop the reservation after a *definitive* failure.
- ▶ `def _now() -> datetime`
Internal helper for the now step.
- ▶ `def operation_key(*, scope: str, target: str, payload: Any) -> str`
Stable fingerprint of "this exact side effect".

integrations.email (9 modules)

app/integrations/email/__init__.py — 158 lines

Email integration — one capability, provider adapters underneath.

- ▶ `def load_connections(raw: str | None=None) -> dict[str, EmailConnection]`
Parse configured connections and resolve their credentials.
- ▶ `def _resolve_credentials(raw: dict[str, Any]) -> dict[str, Any]`
Turn `\*\_env` indirections into their values.
- ▶ `def build_email_service(*, db: Any=None, connections: dict[str, EmailConnection] | None=None, include_memory_adapter: bool=True) -> EmailService`
Assemble the service with every adapter registered.

app/integrations/email/base.py — 249 lines

Provider-neutral email contract.

- ▶ `class EmailAddress`
Pydantic model defining the EmailAddress shape.
`def render(self) -> str`
Render the result.
- ▶ `class EmailAttachmentRef`
Attachment metadata only. Bytes stay in object storage and are fetched by
- ▶ `class EmailMessage`
One message, in the shape every adapter maps onto.
- ▶ `class EmailSearchCriteria`
Search terms, expressed once and translated per provider.
- ▶ `class EmailDraft`
An outbound message, before it is sent or saved as a draft.
- ▶ `class EmailResult`
What an operation returns. One shape for all five operations, so the
- ▶ `class EmailConnection`
A named, credential-bearing connection. Workflow YAML references only
- ▶ `class EmailAdapterError`
A provider call failed in a way the caller can describe to a user.
- ▶ `class EmailAmbiguousFailure`
A side-effect call failed without a definitive outcome — a timeout, a
- ▶ `class EmailAdapter`
What every provider implementation must offer.
`async def search(self, connection: EmailConnection, criteria: EmailSearchCriteria) -> list[EmailMessage]`
Search the result.
`async def read(self, connection: EmailConnection, message_id: str) -> EmailMessage`
Read the result.
`async def create_draft(self, connection: EmailConnection, draft: EmailDraft) -> str`
Returns the provider's draft id.
`async def send(self, connection: EmailConnection, draft: EmailDraft) -> str`
Returns the provider's sent-message id.
`async def reply(self, connection: EmailConnection, draft: EmailDraft) -> str`
Reply defaults to a threaded send.

app/integrations/email/connections_store.py — 115 lines

Durable, non-secret record of an OAuth-established email connection.

- ▶ `async def save_connection_record(db: Any, *, connection_id: str, provider: str, display_name: str, address: str) -> None`
Save the connection record.
- ▶ `async def delete_connection_record(db: Any, connection_id: str) -> None`
Delete the connection record.
- ▶ `async def set_allow_send(db: Any, connection_id: str, allow_send: bool) -> bool`
Returns True if a record was actually updated (the connection exists).
- ▶ `async def load_dynamic_connections(db: Any) -> dict[str, EmailConnection]`
Every OAuth-established connection, with its tokens attached.

app/integrations/email/gmail.py — 436 lines

Gmail adapter — Gmail API v1 over the shared httpx client.

- ▶ `class GmailAdapter`
Provides the GmailAdapter behaviour.
 - `def _headers(self, connection: EmailConnection) -> dict[str, str]`
Internal helper for the headers step.
 - `async def _request(self, connection: EmailConnection, method: str, path: str, *, side_effect: bool=False, **kwargs: Any) -> dict[str, Any]`
Internal helper for the request step.
 - `async def search(self, connection: EmailConnection, criteria: EmailSearchCriteria) -> list[EmailMessage]`
Search the result.
 - `async def read(self, connection: EmailConnection, message_id: str) -> EmailMessage`
Read the result.
 - `async def create_draft(self, connection: EmailConnection, draft: EmailDraft) -> str`
Create the draft.
 - `async def send(self, connection: EmailConnection, draft: EmailDraft) -> str`
Send the result.
- ▶ `def _gmail_query(criteria: EmailSearchCriteria) -> str`
Translate neutral criteria into Gmail's `q` grammar.
- ▶ `def _encode(draft: EmailDraft, connection: EmailConnection) -> str`
Build a base64url RFC-2822 message, which is what Gmail accepts.
- ▶ `def _to_message(raw: dict[str, Any]) -> EmailMessage`
Internal helper for the to message step.
- ▶ `def _extract_bodies(payload: dict[str, Any]) -> tuple[str, str | None]`
Walk the MIME parts tree for text and HTML bodies.
- ▶ `def _extract_attachments(payload: dict[str, Any]) -> list[EmailAttachmentRef]`
Extract the attachments.
- ▶ `def _decode_b64(data: str) -> str`
Decode the b64.
- ▶ `def _parse_address(value: str | None) -> EmailAddress | None`
Parse the address.
- ▶ `def _parse_addresses(value: str | None) -> list[EmailAddress]`
Parse the addresses.
- ▶ `def _parse_internal_date(value: Any) -> datetime | None`
Parse the internal date.

app/integrations/email/memory.py — 168 lines

In-memory email adapter.

- ▶ `class InMemoryEmailAdapter`
Provides the InMemoryEmailAdapter behaviour.
 - `def add(self, message: EmailMessage) -> EmailMessage`
Add the result.

```

    async def search(self, connection: EmailConnection, criteria: EmailSearchCriteria) ->
list[EmailMessage]
    Search the result.
    async def read(self, connection: EmailConnection, message_id: str) -> EmailMessage
    Read the result.
    async def create_draft(self, connection: EmailConnection, draft: EmailDraft) -> str
    Create the draft.
    async def send(self, connection: EmailConnection, draft: EmailDraft) -> str
    Send the result.

```

app/integrations/email/msgraph.py — 432 lines

Microsoft Graph adapter — Outlook / Microsoft 365 mail.

```

▶ class MicrosoftGraphAdapter
    Provides the MicrosoftGraphAdapter behaviour.
    def _headers(self, connection: EmailConnection) -> dict[str, str]
    Internal helper for the headers step.
    def _mailbox(self, connection: EmailConnection) -> str
    Graph addresses either the signed-in user or a specific mailbox.
    async def _request(self, connection: EmailConnection, method: str, path: str, *, side_effect:
bool=False, **kwargs: Any) -> dict[str, Any]
    Internal helper for the request step.
    async def search(self, connection: EmailConnection, criteria: EmailSearchCriteria) ->
list[EmailMessage]
    Search the result.
    async def read(self, connection: EmailConnection, message_id: str) -> EmailMessage
    Read the result.
    async def create_draft(self, connection: EmailConnection, draft: EmailDraft) -> str
    Create the draft.
    async def send(self, connection: EmailConnection, draft: EmailDraft) -> str
    Send the result.
    async def reply(self, connection: EmailConnection, draft: EmailDraft) -> str
    Use Graph's own reply endpoint so it quotes and threads correctly.
▶ def _graph_filters(criteria: EmailSearchCriteria) -> list[str]
    Internal helper for the graph filters step.
▶ def _recipient(address: EmailAddress) -> dict[str, Any]
    Internal helper for the recipient step.
▶ def _draft_payload(draft: EmailDraft) -> dict[str, Any]
    Draft the payload.
▶ def _address(raw: dict[str, Any] | None) -> EmailAddress | None
    Internal helper for the address step.
▶ def _addresses(raw: list[dict[str, Any]] | None) -> list[EmailAddress]
    Internal helper for the addresses step.
▶ def _to_message(raw: dict[str, Any]) -> EmailMessage
    Internal helper for the to message step.
▶ def _parse_timestamp(value: Any) -> datetime | None
    Parse the timestamp.

```

app/integrations/email/oauth.py — 322 lines

OAuth authorization-code flow for Microsoft Graph and Gmail.

```

▶ class OAuthConfigurationError
    This provider has no client id/secret configured for this deployment.
▶ class OAuthExchangeError
    The provider rejected the code/refresh-token exchange.

```

- ▶ `class TokenSet`
Provides the TokenSet behaviour.
- ▶ `class Profile`
Provides the Profile behaviour.
- ▶ `def _redirect_uri(provider: Provider) -> str`
Internal helper for the redirect uri step.
- ▶ `def generate_pkce_pair() -> tuple[str, str]`
(code_verifier, code_challenge) for Google's PKCE requirement.
- ▶ `def authorize_url(provider: Provider, *, state: str, code_challenge: str | None=None) -> str`
Authorize the url.
- ▶ `async def exchange_code(provider: Provider, *, code: str, code_verifier: str | None=None, client: httpx.AsyncClient | None=None) -> TokenSet`
Compute the exchange code.
- ▶ `async def refresh_access_token(provider: Provider, *, refresh_token: str, client: httpx.AsyncClient | None=None) -> TokenSet`
Refresh the access token.
- ▶ `def _parse_token_response(provider: Provider, response: httpx.Response) -> TokenSet`
Parse the token response.
- ▶ `async def fetch_profile(provider: Provider, *, access_token: str, client: httpx.AsyncClient | None=None) -> Profile`
Fetch the profile.

`app/integrations/email/service.py` — 448 lines

EmailService — connection resolution, idempotency, and the operation switch.

- ▶ `class EmailConnectionError`
The referenced connection does not exist, or is not allowed to do this.
- ▶ `class EmailOperationInFlight`
An identical side effect is already in flight or ended ambiguously.
- ▶ `class EmailOperationLedger`
Email's view of the shared external-operation ledger.
- ▶ `class EmailService`
One capability, many providers, one place that owns the safety rules.
`def connection(self, connection_id: str) -> EmailConnection`
Compute the connection.
`def add_connection(self, connection: EmailConnection) -> None`
Adds or replaces a connection at runtime — how a newly completed
`def remove_connection(self, connection_id: str) -> None`
Remove the connection.
`async def _refreshed(self, connection: EmailConnection) -> EmailConnection`
Refresh this connection's access token if it's near/past expiry.
`def adapter(self, connection: EmailConnection) -> EmailAdapter`
Compute the adapter.
`def describe_connections(self) -> list[dict[str, Any]]`
Connection list for the Builder's picker. Credentials are never
`async def execute(self, *, connection_id: str, operation: EmailOperation, criteria: EmailSearchCriteria | None=None, message_id: str | None=None, draft: EmailDraft | None=None, idempotency_scope: str | None=None) -> EmailResult`
Execute the result.
`async def _perform_write(self, *, adapter: EmailAdapter, connection: EmailConnection, operation: EmailOperation, draft: EmailDraft, idempotency_scope: str | None=None) -> EmailResult`
Internal helper for the perform write step.
- ▶ `def _now() -> datetime`
Internal helper for the now step.
- ▶ `def idempotency_key(*, connection_id: str, operation: str, draft: EmailDraft, scope: str | None=None) -> str`

Stable fingerprint of "this exact email side effect".

`app/integrations/email/token_vault.py` — 181 lines

Encrypted storage for OAuth-issued email tokens.

- ▶ `class TokenVaultUnavailableError`
The token vault's storage (Mongo) could not be reached.
- ▶ `class TokenVault`
Mongo-backed envelope-encrypted store for one connection's OAuth tokens.
 - `def _tokens(self)`
The tokens.
 - `async def store(self, connection_id: str, *, access_token: str, refresh_token: str | None, expires_in_seconds: int) -> None`
Compute the store.
 - `async def load(self, connection_id: str) -> dict[str, Any] | None`
Returns `{access\_token, refresh\_token, expires\_at}` (expires_at a
 - `async def forget(self, connection_id: str) -> None`
Compute the forget.
- ▶ `def _load_kek() -> bytes`
Derive the vault's Key Encryption Key from settings, failing closed.
- ▶ `def _seal(payload: dict[str, Any], *, connection_id: str) -> dict[str, bytes]`
Internal helper for the seal step.
- ▶ `def _unseal(doc: dict[str, Any], *, connection_id: str) -> dict[str, Any]`
Internal helper for the unseal step.

integrations.files (8 modules)

app/integrations/files/__init__.py — 138 lines

Cloud file integrations — one capability, provider adapters underneath.

- ▶ `def load_connections(raw: str | None=None) -> dict[str, IntegrationConnection]`
Parse configured connections and resolve their credentials.
- ▶ `def _resolve_credentials(raw: dict[str, Any]) -> dict[str, Any]`
Turn ``\*\_env`` indirections into their values.
- ▶ `def build_integration_service(*, db: Any=None, connections: dict[str, IntegrationConnection] | None=None) -> IntegrationService`
Assemble the service with every provider adapter registered.

app/integrations/files/base.py — 197 lines

Provider-neutral cloud-file contract.

- ▶ `class CloudFileMeta`
Metadata only — never bytes. What a folder listing or search returns.
- ▶ `class DownloadedFile`
Bytes, held only long enough to hand off to object storage.
- ▶ `class Page`
One page of a folder listing or search result.
- ▶ `class IntegrationConnection`
A named, credential-bearing connection. Workflow YAML references only
- ▶ `class IntegrationAdapterError`
A provider call failed in a way the caller can describe to a user.
- ▶ `class IntegrationAuthError`
The access token was rejected — revoked or expired, reauth needed.
- ▶ `class IntegrationNotFoundError`
The referenced file or folder does not exist (or is not accessible).
- ▶ `class IntegrationProvider`
What every cloud-storage adapter must offer.
 - `async def list_folder(self, connection: IntegrationConnection, *, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
List the folder.
 - `async def search_files(self, connection: IntegrationConnection, *, query: str, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
Search the files.
 - `async def get_file_meta(self, connection: IntegrationConnection, *, file_id: str) -> CloudFileMeta`
Return the file meta.
 - `async def download_file(self, connection: IntegrationConnection, *, file_id: str) -> DownloadedFile`
Download the file.

app/integrations/files/connections_store.py — 117 lines

Durable, non-secret record of an OAuth-established Integration connection.

- ▶ `async def save_connection_record(db: Any, *, connection_id: str, provider: str, display_name: str, address: str) -> None`
Save the connection record.
- ▶ `async def delete_connection_record(db: Any, connection_id: str) -> None`
Delete the connection record.

- ▶ `async def set_needs_reauth(db: Any, connection_id: str, needs_reauth: bool) -> None`
Persist the reauth flag so the Builder can show it without a live
- ▶ `async def load_dynamic_connections(db: Any) -> dict[str, IntegrationConnection]`
Every OAuth-established connection, with its tokens attached.

app/integrations/files/google_drive.py — 314 lines

Google Drive adapter — Drive API v3 over the shared httpx client.

- ▶ `class GoogleDriveProvider`
Provides the GoogleDriveProvider behaviour.
 - `def _headers(self, connection: IntegrationConnection) -> dict[str, str]`
Internal helper for the headers step.
 - `async def _request(self, connection: IntegrationConnection, method: str, url: str, **kwargs: Any) -> httpx.Response`
Internal helper for the request step.
 - `async def list_folder(self, connection: IntegrationConnection, *, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
List the folder.
 - `async def search_files(self, connection: IntegrationConnection, *, query: str, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
Search the files.
 - `async def get_file_meta(self, connection: IntegrationConnection, *, file_id: str) -> CloudFileMeta`
Return the file meta.
 - `async def download_file(self, connection: IntegrationConnection, *, file_id: str) -> DownloadedFile`
Download the file.
- ▶ `def _to_meta(raw: dict[str, Any], *, parent_id: str | None=None) -> CloudFileMeta`
Internal helper for the to meta step.
- ▶ `def _to_page(raw: dict[str, Any], *, parent_id: str | None) -> Page`
Internal helper for the to page step.
- ▶ `def _parse_timestamp(value: Any) -> datetime | None`
Parse the timestamp.

app/integrations/files/oauth.py — 320 lines

OAuth authorization-code flow for Google Drive and OneDrive.

- ▶ `class OAuthConfigurationError`
This provider has no client id/secret configured for this deployment.
- ▶ `class OAuthExchangeError`
The provider rejected the code/refresh-token exchange.
- ▶ `class TokenSet`
Provides the TokenSet behaviour.
- ▶ `class Profile`
Provides the Profile behaviour.
- ▶ `def _redirect_uri(provider: Provider) -> str`
Internal helper for the redirect uri step.
- ▶ `def generate_pkce_pair() -> tuple[str, str]`
(code_verifier, code_challenge) for Google's PKCE requirement.
- ▶ `def authorize_url(provider: Provider, *, state: str, code_challenge: str | None=None) -> str`
Authorize the url.
- ▶ `async def exchange_code(provider: Provider, *, code: str, code_verifier: str | None=None, client: httpx.AsyncClient | None=None) -> TokenSet`
Compute the exchange code.

- ▶ `async def refresh_access_token(provider: Provider, *, refresh_token: str, client: httpx.AsyncClient | None=None) -> TokenSet`
Refresh the access token.
- ▶ `def _parse_token_response(provider: Provider, response: httpx.Response) -> TokenSet`
Parse the token response.
- ▶ `async def fetch_profile(provider: Provider, *, access_token: str, client: httpx.AsyncClient | None=None) -> Profile`
Fetch the profile.

app/integrations/files/onedrive.py — 285 lines

OneDrive adapter — Microsoft Graph over the shared httpx client.

- ▶ `class OneDriveProvider`
Provides the OneDriveProvider behaviour.
 - `def _headers(self, connection: IntegrationConnection) -> dict[str, str]`
Internal helper for the headers step.
 - `async def _request(self, connection: IntegrationConnection, method: str, url: str, *, follow_redirects: bool=False, **kwargs: Any) -> httpx.Response`
Internal helper for the request step.
 - `async def list_folder(self, connection: IntegrationConnection, *, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
List the folder.
 - `async def search_files(self, connection: IntegrationConnection, *, query: str, folder_id: str | None, page_size: int, page_token: str | None) -> Page`
Search the files.
 - `async def get_file_meta(self, connection: IntegrationConnection, *, file_id: str) -> CloudFileMeta`
Return the file meta.
 - `async def download_file(self, connection: IntegrationConnection, *, file_id: str) -> DownloadedFile`
Download the file.
- ▶ `def _to_meta(raw: dict[str, Any]) -> CloudFileMeta`
Internal helper for the to meta step.
- ▶ `def _to_page(raw: dict[str, Any]) -> Page`
Internal helper for the to page step.
- ▶ `def _parse_timestamp(value: Any) -> datetime | None`
Parse the timestamp.

app/integrations/files/service.py — 373 lines

IntegrationService — connection resolution, refresh, and the operation switch.

- ▶ `class IntegrationConnectionError`
The referenced connection does not exist, or has no adapter registered.
- ▶ `class IntegrationResult`
Pydantic model defining the IntegrationResult shape.
- ▶ `class IntegrationService`
One capability, many providers, one place that owns token refresh.
 - `def connection(self, connection_id: str) -> IntegrationConnection`
Compute the connection.
 - `def add_connection(self, connection: IntegrationConnection) -> None`
Adds or replaces a connection at runtime — how a newly completed
 - `def remove_connection(self, connection_id: str) -> None`
Remove the connection.
 - `async def _refreshed(self, connection: IntegrationConnection) -> IntegrationConnection`
Refresh this connection's access token if it's near/past expiry.

```

def provider_for(self, connection: IntegrationConnection) -> IntegrationProvider
    Compute the provider for.
def describe_connections(self) -> list[dict[str, Any]]
    Connection list for the Builder's picker. Credentials are never
    async def execute(self, *, connection_id: str, operation: IntegrationOperation, folder_id: str
| list[str] | None=None, file_id: str | list[str] | None=None, query: str='', page_size: int=25,
page_token: str | None=None) -> IntegrationResult
    Execute the result.
▶ def _now() -> datetime
    Internal helper for the now step.
▶ def _as_id_list(value: str | list[str] | None) -> list[str]
    Normalize a config field that may hold one id, several, or none.

```

app/integrations/files/token_vault.py — 187 lines

Encrypted storage for OAuth-issued Google Drive / OneDrive tokens.

```

▶ class TokenVaultUnavailableError
    The token vault's storage (Mongo) could not be reached.
▶ class TokenVault
    Mongo-backed envelope-encrypted store for one connection's OAuth tokens.
    def _tokens(self)
        The tokens.
    async def store(self, connection_id: str, *, access_token: str, refresh_token: str | None,
expires_in_seconds: int) -> None
        Compute the store.
    async def load(self, connection_id: str) -> dict[str, Any] | None
        Returns `{access_token, refresh_token, expires_at}` (expires_at a
    async def forget(self, connection_id: str) -> None
        Compute the forget.
▶ def _load_kek() -> bytes
    Derive the vault's Key Encryption Key from settings, failing closed.
▶ def _seal(payload: dict[str, Any], *, connection_id: str) -> dict[str, bytes]
    Internal helper for the seal step.
▶ def _unseal(doc: dict[str, Any], *, connection_id: str) -> dict[str, Any]
    Internal helper for the unseal step.

```

knowledge (6 modules)

app/knowledge/__init__.py — 1 lines

Knowledge Studio control plane: stable resources, profiles and indexes.

app/knowledge/doc_types.py — 76 lines

The document types a Collection can declare.

- ▶ `class DocTypeChoice`
Provides the DocTypeChoice behaviour.
`def precision_sensitive(self) -> bool`
The precision sensitive.
- ▶ `def doc_type_catalog() -> list[dict[str, object]]`
Compute the doc type catalog.

app/knowledge/ids.py — 131 lines

Opaque, sortable Knowledge Studio resource identifiers.

- ▶ `def _encode(value: int, length: int) -> str`
Encode the result.
- ▶ `def prefix_for(kind: str) -> str`
Compute the prefix for.
- ▶ `def new_resource_id(kind: str) -> str`
Mint a fresh opaque identifier for "kind".
- ▶ `def scoped_legacy_id(kind: str, owner_scope_id: str, legacy_key: str) -> str`
Derive a stable identifier for a pre-existing record.
- ▶ `def kind_of(resource_id: str) -> str | None`
Return the resource kind for "resource_id", or "None" if unknown.
- ▶ `def is_kind(resource_id: str, kind: str) -> bool`
Return whether kind.

app/knowledge/models.py — 529 lines

Versioned Knowledge Studio resource contracts.

- ▶ `class ResourceStatus`
Enumeration of ResourceStatus values.
- ▶ `class ProfileType`
Enumeration of ProfileType values.
- ▶ `class IngestionJobStatus`
Enumeration of IngestionJobStatus values.
- ▶ `class ScopedResource`
Pydantic model defining the ScopedResource shape.
- ▶ `class CollectionResource`
Provides the CollectionResource behaviour.
`def display_name(self) -> str`
The display name.
- ▶ `class DocumentResource`
Provides the DocumentResource behaviour.
- ▶ `class SourceVersionResource`
Provides the SourceVersionResource behaviour.
- ▶ `class ParserProfileConfig`
Pydantic model defining the ParserProfileConfig shape.

- ▶ `class ChunkingProfileConfig`
Pydantic model defining the `ChunkingProfileConfig` shape.
`def validate_token_windows(self) -> 'ChunkingProfileConfig'`
Validate the token windows.
- ▶ `class EmbeddingProfileConfig`
Pydantic model defining the `EmbeddingProfileConfig` shape.
- ▶ `class RetrievalProfileConfig`
Pydantic model defining the `RetrievalProfileConfig` shape.
`def validate_counts(self) -> 'RetrievalProfileConfig'`
Validate the counts.
- ▶ `class RetrievalRoute`
Pydantic model defining the `RetrievalRoute` shape.
- ▶ `class RetrievalRoutingProfileConfig`
Pydantic model defining the `RetrievalRoutingProfileConfig` shape.
`def validate_routes(self) -> 'RetrievalRoutingProfileConfig'`
Validate the routes.
- ▶ `class GenerationProfileConfig`
Pydantic model defining the `GenerationProfileConfig` shape.
- ▶ `class ProfileVersion`
Provides the `ProfileVersion` behaviour.
- ▶ `class IndexVersion`
Provides the `IndexVersion` behaviour.
- ▶ `class IngestionSourceInput`
Pydantic model defining the `IngestionSourceInput` shape.
- ▶ `class IngestionJob`
Provides the `IngestionJob` behaviour.
- ▶ `class RAGAgentDefinition`
Provides the `RAGAgentDefinition` behaviour.
- ▶ `class RetrievalTrace`
Provides the `RetrievalTrace` behaviour.
- ▶ `def utcnow() -> datetime`
Compute the `utcnow`.

`app/knowledge/repository.py` — 683 lines

Owner-scoped Mongo persistence for Knowledge Studio resources.

- ▶ `class ResourceNotFoundError`
A scoped resource lookup found nothing for this owner.
- ▶ `class ResourceConflictError`
A resource exists but is not in a state the caller may act on.
- ▶ `class KnowledgeRepository`
One instance shared across the app via `services["knowledge_repository"]`.
`def _collections(self)`
The collections.
`def _documents(self)`
The documents.
`def _source_versions(self)`
The source versions.
`def _profiles(self)`
The profiles.
`def _indexes(self)`
The indexes.
`def _index_documents(self)`
The index documents.
`def _ingestion_jobs(self)`
The ingestion jobs.

```

def _rag_agents(self)
    The rag agents.
def _traces(self)
    The traces.
async def ensure_indexes(self) -> None
    Ensure the indexes.
async def create_collection(self, resource: CollectionResource) -> CollectionResource
    Create the collection.
async def save_collection(self, resource: CollectionResource) -> CollectionResource
    Save the collection.
async def get_collection(self, owner_scope_id: str, collection_id: str) -> CollectionResource
    Return the collection.
async def list_collections(self, owner_scope_id: str) -> list[CollectionResource]
    List the collections.
async def save_document(self, resource: DocumentResource) -> DocumentResource
    Save the document.
async def get_document(self, owner_scope_id: str, document_id: str) -> DocumentResource
    Return the document.
async def find_document_by_hash(self, owner_scope_id: str, collection_id: str, content_hash:
str) -> DocumentResource | None
    Find the document by hash.
async def list_documents(self, owner_scope_id: str, collection_id: str) ->
list[DocumentResource]
    List the documents.
async def save_source_version(self, resource: SourceVersionResource) -> SourceVersionResource
    Save the source version.
async def get_source_version(self, owner_scope_id: str, source_version_id: str) ->
SourceVersionResource
    Return the source version.
async def next_profile_version(self, owner_scope_id: str, profile_id: str) -> int
    Compute the next profile version.
async def save_profile(self, profile: ProfileVersion) -> ProfileVersion
    Save the profile.
async def get_profile(self, owner_scope_id: str, profile_id: str, version: int | None=None,
expected_type: ProfileType | None=None) -> ProfileVersion
    Return the profile.
async def list_profiles(self, owner_scope_id: str, profile_type: ProfileType | None=None) ->
list[ProfileVersion]
    List the profiles.
async def save_index(self, index: IndexVersion) -> IndexVersion
    Save the index.
async def get_index(self, owner_scope_id: str, index_id: str) -> IndexVersion
    Return the index.
async def list_indexes(self, owner_scope_id: str, collection_id: str) -> list[IndexVersion]
    List the indexes.
async def save_index_document(self, *, owner_scope_id: str, index_id: str, document_id: str,
source_version_id: str, ingestion_job_id: str, chunk_ids: list[str]) -> None
    Save the index document.
async def get_index_document(self, owner_scope_id: str, index_id: str, document_id: str) ->
dict[str, Any] | None
    Return the index document.
async def index_document_stats(self, owner_scope_id: str, index_id: str) -> tuple[int, int]
    Return ``(document_count, chunk_count)`` from authoritative mappings.
async def save_ingestion_job(self, job: IngestionJob) -> IngestionJob
    Save the ingestion job.
async def get_ingestion_job(self, owner_scope_id: str, ingestion_job_id: str) -> IngestionJob

```



```

    Return the ingestion job.
    async def list_ingestion_jobs(self, owner_scope_id: str, collection_id: str | None=None) ->
list[IngestionJob]
    List the ingestion jobs.
    async def list_recoverable_jobs(self) -> list[IngestionJob]
    Non-terminal jobs across every owner, for startup recovery only.
    async def save_rag_agent(self, agent: RAGAgentDefinition) -> RAGAgentDefinition
    Save the rag agent.
    async def get_rag_agent(self, owner_scope_id: str, rag_agent_id: str) -> RAGAgentDefinition
    Return the rag agent.
    async def list_rag_agents(self, owner_scope_id: str, search: str | None=None) ->
list[RAGAgentDefinition]
    List the rag agents.
    async def save_trace(self, trace: RetrievalTrace) -> RetrievalTrace
    Save the trace.
    async def get_trace(self, owner_scope_id: str, retrieval_request_id: str) -> RetrievalTrace
    Return the trace.
    async def list_traces(self, owner_scope_id: str, limit: int=100) -> list[RetrievalTrace]
    List the traces.
    async def record_manifest_index_history(self, *, storage_key: str, manifest: dict[str, Any],
index_history: dict[str, Any]) -> None
    Upsert the legacy manifest while appending this index's history.
    ▶ def _scope(owner_scope_id: str, **extra: Any) -> dict[str, Any]
    Internal helper for the scope step.
    ▶ def _strip(document: dict[str, Any]) -> dict[str, Any]
    Internal helper for the strip step.

```

app/knowledge/service.py — 376 lines

Authorization-aware operations over Knowledge Studio resources.

```

    ▶ class KnowledgeService
    Provides the KnowledgeService behaviour.
    async def create_collection(self, *, owner_scope_id: str, name: str, description: str='',
metadata_schema: dict[str, Any] | None=None, doc_types: list[str] | None=None) ->
CollectionResource
    Create the collection.
    async def create_profile_version(self, *, owner_scope_id: str, profile_type: ProfileType, name:
str, strategy: str, config: dict[str, Any], profile_id: str | None=None, description: str='',
based_on_preset: str | None=None) -> ProfileVersion
    Create the profile version.
    def validate_profile_config(profile_type: ProfileType, config: dict[str, Any]) -> dict[str,
Any]
    Validate the profile config.
    async def create_index(self, *, owner_scope_id: str, collection_id: str, parser_profile_id:
str, parser_profile_version: int, chunking_profile_id: str, chunking_profile_version: int,
embedding_profile_id: str, embedding_profile_version: int) -> IndexVersion
    Create the index.
    async def activate_index(self, *, owner_scope_id: str, collection_id: str, index_id: str) ->
CollectionResource
    Compute the activate index.
    async def create_rag_agent(self, *, owner_scope_id: str, name: str, collection_id: str,
retrieval_profile_id: str, generation_profile_id: str, routing_profile_id: str | None=None,
description: str='') -> RAGAgentDefinition
    Create the rag agent.
    async def ensure_default_profiles(self, owner_scope_id: str) -> dict[str, ProfileVersion]
    Ensure the default profiles.

```

- ▶ `def validate_metadata_schema(schema: dict[str, Any]) -> dict[str, Any]`
Validate the metadata schema.
- ▶ `def workspace_for_scope(owner_scope_id: str) -> str`
Compute the workspace for scope.

llm (18 modules)

app/llm/__init__.py — 20 lines

LLM gateway package. Public API:

app/llm/anthropic_batch.py — 179 lines

Anthropic Message Batches API -- standalone, opt-in batch service.

- ▶ **class AnthropicBatchService**
Submit, poll, and read back an Anthropic Message Batch.
 - `async def submit(self, items: list[BatchRequestItem]) -> str`
Submit a batch and return its batch_id. Does not wait for results.
 - `async def status(self, batch_id: str) -> BatchStatus`
Compute the status.
 - `async def cancel(self, batch_id: str) -> str`
Request cancellation. Already-completed rows still return results.
 - `async def results(self, batch_id: str, *, response_models: dict[str, Type[BaseModel]] | None=None) -> list[BatchResultItem]`
Read back results for an ended batch.
- ▶ `def _tool_name_for(response_model: Type[BaseModel]) -> str`
Internal helper for the tool name for step.
- ▶ `def _build_params(item: BatchRequestItem) -> dict`
Build one MessageCreateParamsNonStreaming-shaped dict for a batch row.

app/llm/anthropic_gw.py — 388 lines

Anthropic Claude gateway.

- ▶ **class AnthropicGateway**
Live Anthropic Claude gateway. One AsyncAnthropic client per instance.
 - `async def probe_model_access(self, model: str) -> str`
Verify workspace access through model metadata, with no generation.
 - `async def _create(self, *, on_token: Callable[[str], Awaitable[None]] | None=None, **kwargs)`
Issue a request via streaming and return the assembled Message.
 - `async def complete(self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None, on_token: Callable[[str], Awaitable[None]] | None=None) -> LLMResponse`
Complete the result.
 - `async def complete_structured(self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str | None=None) -> StructuredResult`
Complete the structured.
 - `async def chat_with_tools(self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096) -> LLMTToolUseResponse`
Compute the chat with tools.
- ▶ `def _supports_temperature(model: str) -> bool`
Return True if the model accepts a non-default temperature.
- ▶ `def _cacheable_system(system: str) -> str | list[dict]`
Turn a plain system string into a cache-marked content-block list.
- ▶ `def _mark_last_block_cacheable(messages: list[dict]) -> None`
Add a cache breakpoint to the last content block of the last message.
- ▶ `def _cache_usage_fields(usage) -> dict[str, int]`
Extract cache_creation/cache_read counts off a Message.usage object.

app/llm/base.py — 143 lines

Abstract LLM gateway contract.

- ▶ class LLMResponse
Plain text response from an LLM call.
- ▶ class ToolCall
One tool invocation the model wants to make.
- ▶ class LLMTToolUseResponse
Response from a multi-turn chat-with-tools call.
- ▶ class LLMGateway
Abstract base for all LLM providers.
async def complete(self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None) -> LLMResponse
Plain text completion.
async def complete_structured(self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None) -> T
Structured completion. Returns an instance of response_model.
async def chat_with_tools(self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096) -> LLMTToolUseResponse
Multi-turn chat with tool calling. Provider-neutral.

app/llm/batch_types.py — 66 lines

Provider-neutral shapes for the Anthropic/OpenAI Batch API services.

- ▶ class BatchRequestItem
One request in a batch submission.
- ▶ class BatchResultItem
One result row, keyed by the custom_id it was submitted with.
- ▶ class BatchStatus
Point-in-time status of a submitted batch.

app/llm/catalog.py — 197 lines

Approved model catalog shared by the runtime, API, and Workflow Builder.

- ▶ class ModelDefinition
Provides the ModelDefinition behaviour.
def permits_data_class(self, data_class: str | None) -> bool
Compute the permits data class.
def as_dict(self) -> dict[str, Any]
Compute the as dict.
- ▶ def model_definition(model: str) -> ModelDefinition
Compute the model definition.
- ▶ def provider_for_model(model: str) -> str
Compute the provider for model.
- ▶ def is_local_model(model: str) -> bool
Return whether local model.
- ▶ def local_service_name(model: str) -> str | None
Compute the local service name.

app/llm/errors.py — 34 lines

Provider-neutral LLM errors used by the resilience layer.

- ▶ class StructuredOutputError
The provider responded, but its output violated the required schema.

- ▶ `class LLMInputLimitError`
The resolved prompt exceeds a configured token/cost safety boundary.
- ▶ `class LLMPProviderUnavailableError`
No configured provider can serve the requested model/fallback chain.
- ▶ `class BatchTimeoutError`
A submitted batch did not reach a terminal state within the deadline.
- ▶ `class LLMPolicyDeniedError`
The data-classification/OPA policy check (`app/security/llm_policy.py`) denied this

`app/llm/local_openai_gw.py` — 444 lines

OpenAI-compatible gateway for privately hosted Kimi K3 and GLM-5.

- ▶ `class LocalModelProfile`
Provides the `LocalModelProfile` behaviour.
- ▶ `class LocalOpenAIGateway`
Gateway used by one configured local/private model endpoint.
 - `def _generation_options(self) -> dict[str, Any]`
Internal helper for the generation options step.
 - `def _fixed_sampling_provider(self) -> bool`
Internal helper for the fixed sampling provider step.
 - `def _max_tokens_field(self) -> str`
Internal helper for the max tokens field step.
 - `async def complete(self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None, on_token: object | None=None) -> LLMResponse`
Complete the result.
 - `async def complete_structured(self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str | None=None) -> StructuredResult`
Complete the structured.
 - `async def chat_with_tools(self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096) -> LLMTToolUseResponse`
Compute the chat with tools.
 - `async def probe_details(self) -> dict[str, Any]`
Probe the details.
 - `async def probe(self) -> bool`
Probe the result.
- ▶ `class KimiK3LocalGateway`
Distinct class keeps Kimi and GLM instances separate in the registry.
- ▶ `class GLM5LocalGateway`
Distinct class keeps Kimi and GLM instances separate in the registry.
- ▶ `def normalize_openai_base_url(value: str) -> str`
Validate and normalize a deployment-owned OpenAI-compatible URL.
- ▶ `def _reasoning_content(message: Any) -> str | None`
Internal helper for the reasoning content step.
- ▶ `def _usage(response: Any) -> tuple[int, int]`
Internal helper for the usage step.

`app/llm/model_catalog.py` — 163 lines

Provider-neutral LLM catalog used by routing, validation, cost, and UI.

- ▶ `class ModelProfile`
Provides the `ModelProfile` behaviour.
- ▶ `def estimate_model_cost(model: str, input_tokens: int, output_tokens: int) -> float`
Estimate request cost using the platform's configured price catalog.

app/llm/model_router.py — 654 lines

Deterministic, zero-token model selection for automatic workflow routing.

- ▶ class ModelRoutingError
Raised when an automatic selection cannot satisfy its constraints.
- ▶ class ModelSelection
Provides the ModelSelection behaviour.
def to_event(self, *, actual_model: str, call_id: int, fallback: bool=False, cache_hit: bool=False) -> dict[str, Any]
Compute the to event.
- ▶ class ModelRouter
Choose the most suitable configured model without spending tokens.
def select(self, *, method_name: str, kwargs: Mapping[str, Any], input_tokens: int, allowed_models: Sequence[str], is_available: Callable[[str], bool], node_type: str | None=None, policy: Mapping[str, Any] | None=None) -> ModelSelection
Select the result.
def describe_manual(self, *, requested_model: str, selected_model: str, method_name: str, kwargs: Mapping[str, Any], input_tokens: int, node_type: str | None=None, mode: Literal['manual', 'cost_protection']='manual') -> ModelSelection
Compute the describe manual.
- ▶ def infer_task_kind(method_name: str, kwargs: Mapping[str, Any], *, node_type: str | None=None) -> TaskKind
Compute the infer task kind.
- ▶ def infer_complexity(method_name: str, kwargs: Mapping[str, Any], *, input_tokens: int, node_type: str | None=None) -> Complexity
Compute the infer complexity.
- ▶ def _request_text(kwargs: Mapping[str, Any], *, node_type: str | None) -> str
Internal helper for the request text step.
- ▶ def _accuracy_priority(value: Any) -> AccuracyPriority
Internal helper for the accuracy priority step.
- ▶ def _hosted_bonus(profile: ModelProfile, priority: AccuracyPriority) -> float
Break same-score ties toward hosted providers under max accuracy.
- ▶ def _target_tier(complexity: Complexity, priority: AccuracyPriority) -> int
Internal helper for the target tier step.
- ▶ def _score_profile(profile: ModelProfile, *, task: TaskKind, complexity: Complexity, priority: AccuracyPriority, estimated_cost: float, max_cost: float, prefer_low_latency: bool, observed_quality: float | None) -> float
Score the profile.
- ▶ def _selection_reason(profile: ModelProfile, *, task: TaskKind, complexity: Complexity, priority: AccuracyPriority, source: Literal['policy', 'evaluation'], estimated_cost: float, ceiling: float | None) -> str
Internal helper for the selection reason step.

app/llm/openai_batch.py — 219 lines

OpenAI Batch API -- standalone, opt-in batch service.

- ▶ class OpenAIBatchService
Submit, poll, and read back an OpenAI Batch.
async def submit(self, items: list[BatchRequestItem]) -> str
Upload the JSONL request file, create the batch, return its id.
async def status(self, batch_id: str) -> BatchStatus
Compute the status.
async def cancel(self, batch_id: str) -> str
Request cancellation. In-flight requests may still complete.
async def results(self, batch_id: str, *, response_models: dict[str, Type[BaseModel]] | None=None) -> list[BatchResultItem]

Read back results for an ended batch.

- ▶ `def _build_body(item: BatchRequestItem) -> dict`
Build one /v1/chat/completions request body for a batch JSONL row.
- ▶ `def _parse_jsonl(text: str) -> list[dict]`
Parse the jsonl.
- ▶ `def _result_from_output_row(row: dict, response_models: dict[str, Type[BaseModel]]) -> BatchResultItem`
Internal helper for the result from output row step.

`app/llm/openai_gw.py` — 449 lines

OpenAI implementation of LLMGateway.

- ▶ `class StructuredResult`
Carries a parsed Pydantic model plus token usage, so the registry
- ▶ `class OpenAIGateway`
OpenAI-backed gateway. One AsyncOpenAI client per instance.
 - `async def probe_model_access(self, model: str) -> str`
Verify project access through model metadata, with no generation.
 - `async def complete(self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None, on_token: Callable[[str], Awaitable[None]] | None=None) -> LLMResponse`
Complete the result.
 - `async def complete_structured(self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str | None=None) -> T`
Complete the structured.
 - `async def chat_with_tools(self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096) -> LLMTToolUseResponse`
Compute the chat with tools.
- ▶ `def _supports_custom_temperature(model: str) -> bool`
Return True if the model accepts a non-default temperature.
- ▶ `def _completion_tokens_for(model: str, requested: int) -> int`
Apply provider-specific minimums to max_completion_tokens.
- ▶ `def _cached_tokens_from_usage(usage: Any) -> int`
Read OpenAI's automatic-prompt-cache hit count off a usage payload.
- ▶ `def _prompt_cache_key(model: str, cacheable_prefix: str) -> str`
Stable prompt_cache_key so requests sharing a static prefix (same
- ▶ `def _prompt_cache_retention_kwargs(model: str) -> dict`
Extended (24h) cache retention for pre-GPT-5.6 models, at the same
- ▶ `def _system_content(text: str, model: str | None) -> str | list[dict]`
Render system-prompt text as a bare string, or (gpt-5.6+ only) as a
- ▶ `def _system_messages(system: str, *, model: str | None=None) -> list[dict]`
Build the leading system message, omitting it when empty.
- ▶ `def _chat_tool_reasoning_effort(model: str) -> str | None`
Return the safe Chat Completions effort for function-tool calls.

`app/llm/openai_registry.py` — 369 lines

Authoritative OpenAI model registry for every platform capability.

- ▶ `class OpenAIModelDefinition`
Provides the OpenAIModelDefinition behaviour.
 - `def as_dict(self) -> dict[str, Any]`
Compute the as dict.
- ▶ `def models_for_kind(kind: OpenAIModelKind) -> tuple[str, ...]`
Compute the models for kind.
- ▶ `def openai_model(model: str) -> OpenAIModelDefinition`

Compute the openai model.

- ▶ `def require_openai_model_kind(model: str, kind: OpenAIModelKind) -> str`
Compute the require openai model kind.

`app/llm/openrouter_catalog.py` — 228 lines

Live OpenRouter model catalog, sourced directly from OpenRouter's own ``GET /v1/models``.

- ▶ `class OpenRouterModelInfo`
Provides the `OpenRouterModelInfo` behaviour.
`def as_dict(self) -> dict[str, Any]`
Compute the as dict.
- ▶ `class OpenRouterCatalogCache`
Process-wide TTL cache. A dedicated instance (not module-level globals) so tests can
`def _is_stale(self) -> bool`
Return whether stale.
`async def get_models(self, *, client: httpx.AsyncClient | None=None, force_refresh: bool=False)`
-> `tuple[OpenRouterModelInfo, ...]`
Return the models.
`async def search(self, query: str | None=None, *, limit: int=50, client: httpx.AsyncClient | None=None) -> list[OpenRouterModelInfo]`
Search the result.
- ▶ `def is_openrouter_model_id(value: str) -> bool`
Structural check only (no live lookup) — used by preflight, which must stay
- ▶ `def _price_per_million(raw: Any) -> float | None`
OpenRouter reports pricing as a per-token USD string (e.g. "0.00000015").
- ▶ `def _to_model_info(entry: dict[str, Any]) -> OpenRouterModelInfo | None`
Internal helper for the to model info step.
- ▶ `async def _fetch_raw_catalog(client: httpx.AsyncClient | None) -> list[dict[str, Any]]`
Fetch the raw catalog.
- ▶ `def get_default_cache() -> OpenRouterCatalogCache`
Return the default cache.

`app/llm/openrouter_gw.py` — 375 lines

OpenRouter implementation of LLMGateway — direct calls to OpenRouter's real API.

- ▶ `class StructuredResult`
Mirrors `app/llm/openai_gw.py`'s `StructuredResult` — carries usage/cost alongside the
- ▶ `class OpenRouterGateway`
OpenRouter-backed gateway. One `httpx.AsyncClient` per instance.
`async def probe_model_access(self, model: str) -> str`
Cheap existence check via the catalog — OpenRouter has no per-model
`async def _post(self, body: dict[str, Any]) -> dict[str, Any]`
Internal helper for the post step.
`async def complete(self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None) -> LLMResponse`
Complete the result.
`async def complete_structured(self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str | None=None) -> StructuredResult`
Complete the structured.
`async def chat_with_tools(self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096) -> LLMTToolUseResponse`
Compute the chat with tools.
- ▶ `def strip_openrouter_prefix(model: str) -> str`
Compute the strip openrouter prefix.

- `def unfence_json(content: str) -> str`
Return the JSON body of a Markdown-fenced payload, or `content` unchanged.
- `def _parse_structured(content: str, response_model: Type[T]) -> T`
Parse a structured response, retrying once on the unfenced body.
- `def _usage_fields(usage: dict[str, Any] | None) -> dict[str, Any]`
Internal helper for the usage fields step.
- `def _safe_json_loads(value: str) -> dict[str, Any]`
Internal helper for the safe json loads step.

`app/llm/openrouter_ranking.py` — 102 lines

Auto-router config for OpenRouter — see docs/architecture/LLM_MODEL_ROUTING.md.

- `def _cost_tier_for_priority(accuracy_priority: str) -> str`
Internal helper for the cost tier for priority step.
- `def build_auto_router_plugin(*, accuracy_priority: str='balanced', max_estimated_cost_usd: float | None=None, quality_scores: Mapping[str, float] | None=None, faithfulness_scores: Mapping[str, float] | None=None) -> dict[str, Any]`
Builds one `plugins[]` entry for OpenRouter's auto-router from Eurskem's
- `def is_openrouter_router_model(value: str) -> bool`
True for OpenRouter's own router meta-models (openrouter/auto, openrouter/free, ...).

`app/llm/registry.py` — 1732 lines

Resolve a model name to the right gateway instance.

- `class RetryPolicy`
One explicit retry policy shared by all LLM providers.
- `class ModelAccessResult`
Result of a model-metadata probe that uses no generation tokens.
- `class RegistryLLMGateway`
LLMGateway adapter that dispatches every call through the registry.
`def with_context(self, run_id: str, session_id: str, node_id: str, ledger=None, *, event_bus=None, node_type: str | None=None, allowed_models: list[str] | None=None, routing_policy: dict[str, Any] | None=None, semantic_cache=None, use_cache: bool=False, entity_tokenizer=None, collection_id: str='default', processing_mode: str | None=None, workflow_name: str | None=None) -> 'RegistryLLMGateway'`
Return a context-bound copy for per-node cost tracking and routing.
`def ensure_routed(self, *, node_type: str | None=None) -> 'RegistryLLMGateway'`
Return a gateway guaranteed to resolve AUTO_MODEL via ModelRouter.
`def with_collection_id(self, collection_id: str) -> 'RegistryLLMGateway'`
Return a clone with only collection_id overridden.
`async def _tokenize_text(self, text: str | None) -> str | None`
Tokenize the text.
`async def _tokenize_value(self, value: Any) -> Any`
Tokenize the value.
`async def _tokenize_messages(self, messages: list[dict]) -> list[dict]`
Tokenize the messages.
`async def _detokenize_value(self, value: Any) -> Any`
Detokenize the value.
`def _cached_model_access(self, model: str) -> ModelAccessResult | None`
Internal helper for the cached model access step.
`def _available_for_auto(self, model: str) -> bool`
Combine configured-provider checks with verified project access.
`async def probe_model_access(self, models: list[str] | tuple[str, ...] | set[str]) -> dict[str, ModelAccessResult]`
Probe concrete models without creating a completion.

```

def selection_history(self) -> list[dict[str, Any]]
    Per-call model-selection events recorded on this bound gateway.
def _next_call_id(self) -> int
    Internal helper for the next call id step.
def _select_model(self, *, method_name: str, requested: str, kwargs: dict[str, Any])
    Deterministic, zero-token model choice for one call.
async def _publish(self, evt: RunEvent) -> None
    Publish the result.
def _record_cost(self, intended: str, resolved: str, resp, *, task_type: str='unknown',
latency_ms: float | None=None, fallback_reason: str | None=None, stage: str | None=None) -> None
    Write a LedgerEntry if we have context + a ledger.
def _models_for_call(self, intended: str, candidate_models: tuple[str, ...] | list[str] |
None=None) -> list[str]
    Return the permitted, ordered runtime fallback chain.
def _auto_candidates(self) -> list[str]
    Concrete models "auto" may resolve to, best first.
def _delay_for(self, attempt: int, exc: BaseException) -> float
    Internal helper for the delay for step.
async def _call_resilient(self, method_name: str, *, intended: str, kwargs: dict[str, Any],
candidate_models: tuple[str, ...] | list[str] | None=None) -> tuple[Any, str, float, str | None]
    Returns (response, resolved_model, latency_ms, fallback_reason).
async def complete(self, *, model: str, stage: str | None=None, **kwargs)
    Public entry point — tokenizes system/user, delegates to the
async def _complete_impl(self, *, model: str, stage: str | None=None, **kwargs)
    Complete the impl.
async def complete_structured(self, *, model: str, stage: str | None=None, **kwargs)
    Public entry point — tokenizes system/user, delegates, then
async def _complete_structured_impl(self, *, model: str, stage: str | None=None, **kwargs)
    Complete the structured impl.
async def chat_with_tools(self, *, model: str, **kwargs)
    Public entry point — tokenizes system + every message's content/
async def _chat_with_tools_impl(self, *, model: str, **kwargs)
    Internal helper for the chat with tools impl step.
▶ def _promote_tier_peers(requested: list[str]) -> list[str]
    Move each model's documented cross-provider peer to directly follow it.
▶ def _status_code(exc: BaseException) -> int | None
    Internal helper for the status code step.
▶ def _anthropic_error_type(exc: BaseException) -> str | None
    Anthropic streams some errors (e.g. overload) as an in-band SSE event
▶ def _is_retryable_error(exc: BaseException) -> bool
    Return whether another generation may succeed.
▶ def _provider_error_code(exc: BaseException) -> str | None
    Extract a provider error code without depending on one SDK's shape.
▶ def _is_model_unavailable_error(exc: BaseException) -> bool
    Return whether this candidate cannot serve the requested model.
▶ def _retry_after_seconds(exc: BaseException) -> float | None
    Retry the after seconds.
▶ def resolve_model(intended: str) -> str
    Map an intended model name to the model name we'll actually call.
▶ def get_gateway(model_name: str) -> tuple[LLMGateway, str]
    Return (gateway, resolved_model_name).
▶ def _gateway_class_for(model_name: str) -> type[LLMGateway]
    Internal helper for the gateway class for step.
▶ def _construct(gw_cls: type[LLMGateway]) -> LLMGateway
    Internal helper for the construct step.
▶ def _kimi_profile() -> LocalModelProfile

```

- Internal helper for the kimi profile step.
- ▶ `def _glm_profile() -> LocalModelProfile`
Internal helper for the glm profile step.
- ▶ `def _provider_name_for(model: str) -> str`
Internal helper for the provider name for step.
- ▶ `def _safe_task_kind(method_name: str, kwargs: dict[str, Any], node_type: str | None) -> str`
infer_task_kind(), but cost telemetry must never break a call.
- ▶ `def local_model_enabled(model: str) -> bool`
Compute the local model enabled.
- ▶ `def configured_local_model_probes() -> dict[str, Callable[[], Awaitable[bool]]]`
Return health probes for enabled local providers without exposing URLs.
- ▶ `async def probe_local_model(model: str) -> dict[str, Any]`
Probe the local model.
- ▶ `def _record_usage(intended: str, resolved: str, resp) -> None`
Record token metrics. Never raises — instrumentation must not break calls.
- ▶ `def _estimate_input_tokens(kwargs: dict[str, Any]) -> int`
Cheap pre-call token estimate for deterministic routing.
- ▶ `def _is_model_available(model: str) -> bool`
Return whether auto-routing may safely select “model”.
- ▶ `def get_llm_gateway() -> RegistryLLMGateway`
Module-level singleton used by main.py's lifespan.

`app/llm/semantic_cache.py` — 310 lines

Tenant-scoped Redis semantic cache for deterministic text completions.

- ▶ `class CacheLookup`
Provides the CacheLookup behaviour.
- ▶ `class SemanticLLMCache`
Small-scope semantic cache backed by Redis.
 - `async def probe(self) -> bool`
Probe the result.
 - `async def get(self, *, session_id: str, model: str, system: str, user: str, temperature: float, max_tokens: int) -> CacheLookup`
Return the result.
 - `async def put(self, *, session_id: str, model: str, system: str, user: str, temperature: float, max_tokens: int, response: dict[str, Any], query_embedding: list[float] | None) -> None`
Store the result.
 - `async def _prune(self, scope: str) -> None`
Prune the result.
 - `def _scope(*, session_id: str, model: str, system: str, temperature: float, max_tokens: int) -> str`
Internal helper for the scope step.
 - `def _entry_key(scope: str, user: str) -> str`
Internal helper for the entry key step.
 - `def _index_key(scope: str) -> str`
Internal helper for the index key step.
- ▶ `def _cosine(left: list[float], right: list[float]) -> float`
Internal helper for the cosine step.

mcp (8 modules)

app/mcp/client.py — 424 lines

MCP client wrapper — multi-server.

- ▶ class MCPClient
 - Multi-server MCP wrapper. call_tool(name, arguments, server=...) routes
 - def configured_servers(self) -> tuple[str, ...]
 - The configured servers.
 - def running_servers(self) -> tuple[str, ...]
 - The running servers.
 - async def start(self, servers: list[str] | None=None) -> None
 - Launch the requested servers (default: all declared). A server that
 - async def stop(self) -> None
 - Stop the result.
 - def _require(self, server: str) -> ClientSession
 - Internal helper for the require step.
 - def has_server(self, server: str) -> bool
 - Return whether server.
 - async def list_tools(self, server: str=DEFAULT_SERVER) -> list
 - List the tools.
 - async def probe(self, server: str=DEFAULT_SERVER) -> bool
 - Perform a live MCP request instead of trusting a cached session.
 - async def call_tool(self, name: str, arguments: dict, server: str=DEFAULT_SERVER) -> Any
 - Call a tool on `server` (default: primary). Returns the first
 - async def call_tool_raw(self, name: str, arguments: dict, *, server: str=DEFAULT_SERVER, timeout_seconds: float | None=None) -> Any
 - Call a tool and return the full CallToolResult.
- ▶ def _paper_search_mcp_env(app_settings: Settings) -> dict[str, str]
 - Research credentials for the paper-search-mcp subprocess.
- ▶ def _dynamics_env(app_settings: Settings) -> dict[str, str]
 - Environment for the Dynamics 365 MCP subprocess.
- ▶ def _finance_scm_spec(app_settings: Settings) -> StdioServerParameters
 - Launch spec for the Dynamics 365 Finance & Operations MCP server.
- ▶ def _business_records_env(app_settings: Settings) -> dict[str, str]
 - Environment for the Business Records MCP subprocess.
- ▶ def build_server_specs(app_settings: Settings=settings) -> dict[str, StdioServerParameters]
 - Build portable MCP launch specs from application configuration.
- ▶ async def launch_mcp_session(servers: list[str] | None=None) -> MCPClient
 - Convenience used by app/main.py at FastAPI startup.

app/mcp/connections.py — 326 lines

Built-in MCP connections, and how the integration service is assembled.

- ▶ def _dynamics_tool_policies() -> dict[str, MCPToolPolicy]
 - State each Dynamics tool's class as deployment policy.
- ▶ def dynamics_connection(app_settings: Settings=settings) -> MCPServerConnection
 - Compute the dynamics connection.
- ▶ def read_only_dynamics_connection(app_settings: Settings=settings) -> MCPServerConnection
 - A least-privilege variant, for deployments that want CRM context only.
- ▶ def finance_scm_connection(app_settings: Settings=settings) -> MCPServerConnection
 - The Dynamics 365 Finance & Operations connector.
- ▶ def business_records_connection(app_settings: Settings=settings) -> MCPServerConnection

A real, persistent MySQL database — genuine Lookup/Create/Update tools.

- ▶ `def build_registry(app_settings: Settings=settings) -> MCPRegistry`
Assemble every configured connection.
- ▶ `def build_mcp_service(*, client: Any, db: Any=None, app_settings: Settings=settings) -> MCPIntegrationService`
Build the mcp service.

app/mcp/paper_search_server.py — 266 lines

Credential-aware launcher for the pinned paper-search-mcp server.

- ▶ `class _JsonNullSafeOpenAireResponse`
Wraps a `requests.Response` so `.json()` never nests `None` under
`def json(self, *args: Any, **kwargs: Any) -> Any`
Compute the json.
- ▶ `class _JsonDefaultsDataKey`
Wraps a `requests.Response` so `.json()` always has a `'data'` key.
`def json(self, *args: Any, **kwargs: Any) -> Any`
Compute the json.
- ▶ `def _prepare_source_checkout() -> None`
Internal helper for the prepare source checkout step.
- ▶ `def inject_openalex_api_key(searcher: Any, api_key: str) -> bool`
Inject the key into a searcher session without logging the secret.
- ▶ `def _normalize_openaire_legacy_json(payload: Any) -> Any`
Guard against `"response": null` / `"results": null` in the OpenAIRE
- ▶ `def patch_openaire_query_param(module_name: str='paper_search_mcp.academic_platforms.openaire')`
`-> None`
Rename the legacy fallback's `'query'` param to the supported `'keywords'`,
- ▶ `def _wait_for_semantic_scholar_rate_limit(min_interval_seconds: float) -> None`
Block until at least `'min_interval_seconds'` has passed since the last
- ▶ `def patch_semantic_scholar_empty_results(module_name:`
`str='paper_search_mcp.academic_platforms.semantic', min_interval_seconds: float=1.1) -> None`
Make zero-result Semantic Scholar responses parse instead of `KeyError`,
- ▶ `def main() -> None`
Compute the main.

app/mcp/policy.py — 295 lines

What may be called, by whom, and whether a person has to see it first.

- ▶ `class MCPPolicyError`
A tool call was refused by policy. Carries a message written for the
- ▶ `class MCPApprovalRequired`
A write was refused because no human review is guaranteed before it.
- ▶ `class PolicyDecision`
The outcome of the gate, kept as data so it can be logged, shown in the
`def raise_if_denied(self) -> None`
Raise the if denied.
- ▶ `def _words(tool_name: str) -> list[str]`
Split a tool name into words across the naming styles servers use:
- ▶ `def classify_by_name(tool_name: str) -> OperationClass`
Best-effort classification from the tool's name alone.
- ▶ `def classify_by_annotations(annotations: Any) -> OperationClass | None`
Read the server's own hints, if it declared any.
- ▶ `def classify_tool(tool_name: str, *, connection: MCPServerConnection | None=None, annotations:`
`Any=None) -> OperationClass`
The platform's classification of one tool.

- ▶ `def is_write(operation: OperationClass) -> bool`
Anything not provably a read is treated as a write.
- ▶ `def evaluate(*, connection: MCPServerConnection, tool_name: str, annotations: Any=None, user_role: str | None=None, approval_satisfied: bool=False) -> PolicyDecision`
Run the gate for one tool call.

app/mcp/registry.py — 405 lines

MCP servers as first-class connections.

- ▶ `class MCPToolPolicy`
Per-tool overrides, set by the deployment rather than by the server.
- ▶ `class MCPServerConnection`
One configured MCP server.
 - `def id_is_safe(cls, value: str) -> str`
Compute the id is safe.
 - `def stdio_needs_a_command(self) -> 'MCPServerConnection'`
Compute the stdio needs a command.
 - `def secrets_are_references(self) -> 'MCPServerConnection'`
Reject a literal secret pasted where a reference belongs.
 - `def label(self) -> str`
The label.
 - `def resolve_environment(self) -> dict[str, str]`
Build the subprocess environment, reading secrets at launch time.
 - `def describe(self) -> dict[str, Any]`
Builder-facing description. Never includes a resolved secret —
 - `def permits_tool(self, tool_name: str) -> bool`
Compute the permits tool.
 - `def policy_for(self, tool_name: str) -> MCPToolPolicy`
Compute the policy for.
- ▶ `class MCPServerRegistry`
The set of configured servers, keyed by id.
 - `def ids(self) -> tuple[str, ...]`
The ids.
 - `def add(self, connection: MCPServerConnection) -> None`
Add the result.
 - `def get(self, server_id: str) -> MCPServerConnection | None`
Return the result.
 - `def require(self, server_id: str) -> MCPServerConnection`
Compute the require.
 - `def all(self) -> list[MCPServerConnection]`
Compute the all.
 - `def record_health(self, server_id: str, *, healthy: bool, tool_count: int=0, error: str | None=None) -> None`
Record the health.
 - `def health(self, server_id: str) -> dict[str, Any]`
Compute the health.
 - `def describe_all(self) -> list[dict[str, Any]]`
Compute the describe all.
- ▶ `def load_servers(raw: str | None=None) -> dict[str, MCPServerConnection]`
Parse configured servers from JSON.

app/mcp/results.py — 312 lines

Turn an MCP tool result into a typed workflow output.

- ▶ `class MCPToolResult`

The normalised result every MCP tool call produces.

- ▶ `def _depth(value: Any, current: int=0) -> int`
Internal helper for the depth step.
- ▶ `def _as_object(value: Any) -> dict[str, Any] | None`
Normalise a parsed payload into an object, or reject it.
- ▶ `def parse_json_text(text: str, *, max_bytes: int) -> tuple[dict[str, Any] | None, str]`
Parse a text payload that looks like JSON, safely.
- ▶ `def normalise_result(response: Any, *, max_bytes: int=256000, tool_name: str='') -> MCPToolResult`
Normalise an MCP `CallToolResult` into a workflow output.
- ▶ `def _collect_text(content: Any) -> str`
Internal helper for the collect text step.
- ▶ `def _trim(text: str, max_bytes: int) -> str`
Internal helper for the trim step.
- ▶ `def field_paths_from_schema(schema: dict[str, Any] | None, prefix: str='', depth: int=0) -> list[dict[str, Any]]`
Flatten a JSON Schema into typed dotted paths.
- ▶ `def _json_type(schema: dict[str, Any]) -> str`
Internal helper for the json type step.

`app/mcp/server.py` — 269 lines

MCP server: exposes hybrid retrieval as protocol-level tools.

- ▶ `def _services() -> dict[str, Any]`
Internal helper for the services step.
- ▶ `async def list_tools() -> list[types.Tool]`
List the tools.
- ▶ `async def call_tool(name: str, arguments: dict) -> list[types.TextContent]`
Compute the call tool.
- ▶ `async def main() -> None`
Compute the main.

`app/mcp/service.py` — 619 lines

The MCP integration service: discovery, policy, execution, audit.

- ▶ `class MCPToolError`
A tool call failed in a way worth reporting to a workflow author.
`def as_payload(self) -> dict[str, Any]`
Compute the as payload.
- ▶ `class ToolDescriptor`
A discovered tool, described for the Builder.
- ▶ `class MCPIntegrationService`
Provides the MCPIntegrationService behaviour.
`def describe_servers(self) -> list[dict[str, Any]]`
Connections for the Builder's server picker. No credentials — only
`async def discover_tools(self, server_id: str, *, refresh: bool=False) -> list[ToolDescriptor]`
Ask the server what it can do.
`def _describe_tool(self, connection: MCPServerConnection, tool: Any) -> ToolDescriptor`
Internal helper for the describe tool step.
`async def find_tool(self, server_id: str, tool_name: str) -> ToolDescriptor | None`
Find the tool.
`async def health_check(self, server_id: str) -> dict[str, Any]`
Compute the health check.
`async def call(self, *, server_id: str, tool_name: str, arguments: dict[str, Any], run_id: str='', node_id: str='', session_id: str='', user_role: str | None=None, approval_satisfied: bool=False, audit_db: Any=None, timeout_override: float | None=None) -> dict[str, Any]`

```

    Run one tool through the full gate.
    def _require_connection(self, server_id: str, tool_name: str) -> MCPServerConnection
        Internal helper for the require connection step.
    def _replay_or_refuse(self, existing: dict[str, Any], key: str, connection:
MCPServerConnection, tool_name: str, decision: PolicyDecision) -> dict[str, Any]
        Internal helper for the replay or refuse step.
    async def _audit(self, audit_db: Any, *, run_id: str, node_id: str, session_id: str, server_id:
str, tool_name: str, operation: str, status: str, started: float, detail: dict[str, Any]) -> None
        Record the invocation (§27).
    ▶ def _tool_error_payload(result: MCPToolResult) -> dict[str, Any] | None
        Recognise a structured error a server returned in place of a result.
    ▶ def _humanise(name: str) -> str
        Internal helper for the humanise step.

```


mcp.business_records (7 modules)

app/mcp/business_records/__init__.py — 17 lines

Real, live MySQL-backed business-records MCP server.

app/mcp/business_records/db.py — 92 lines

Connection helper for the business-records MySQL database.

- ▶ `def connect(app_settings: Settings=settings) -> MySQLConnectionAbstract`
Compute the connect.
- ▶ `def connect_readonly(app_settings: Settings=settings) -> MySQLConnectionAbstract`
A genuinely lower-privileged connection for `query_readonly`
- ▶ `def execute(conn: MySQLConnectionAbstract, query: str, params: tuple[Any, ...]=()) -> list[dict[str, Any]]`
Run a parameterized SELECT and return every row as a dict.
- ▶ `def execute_write(conn: MySQLConnectionAbstract, query: str, params: tuple[Any, ...]=()) -> int`
Run a parameterized INSERT/UPDATE, commit, and return the affected/

app/mcp/business_records/handlers.py — 474 lines

Real, parameterized-SQL handlers for the business-records MCP server.

- ▶ `def _collection(rows: list[dict[str, Any]], key: str, limit: int) -> dict[str, Any]`
Internal helper for the collection step.
- ▶ `def _limit(arguments: dict[str, Any], default: int, maximum: int) -> int`
Internal helper for the limit step.
- ▶ `async def customer_search(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Compute the customer search.
- ▶ `async def order_search(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Compute the order search.
- ▶ `async def inventory_check(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Compute the inventory check.
- ▶ `async def product_search(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Compute the product search.
- ▶ `async def query_readonly(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Deliberately ignores ``conn`` — the shared module-level connection
- ▶ `async def create_case(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Create the case.
- ▶ `async def create_opportunity(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Create the opportunity.
- ▶ `async def create_order(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Create the order.
- ▶ `def _order_view(row: dict[str, Any]) -> dict[str, Any]`
Normalizes a raw ``crm_salesorders`` row to `create_order`'s field names —
- ▶ `async def update_order(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`

Update the order.

- ▶ `def _case_view(row: dict[str, Any]) -> dict[str, Any]`
Normalizes a raw `crm\_service\_cases` row to create `case`'s field names
- ▶ `async def update_case(conn: MySQLConnectionAbstract, arguments: dict[str, Any]) -> dict[str, Any]`
Update the case.
- ▶ `def _next_suffix(conn: MySQLConnectionAbstract, table: str, column: str, prefix: str) -> int`
A small counter derived from how many rows already start with `prefix`

app/mcp/business_records/seed.py — 473 lines

Load both fixture files into the real MySQL database.

- ▶ `def _load(path: Path) -> dict[str, list[dict[str, Any]]]`
Load the result.
- ▶ `def _dt(value: str | None) -> str | None`
ISO 8601 ("...T...Z") -> MySQL DATETIME literal ("... ...").
- ▶ `def _upsert(conn, table: str, row: dict[str, Any]) -> None`
Upsert the result.
- ▶ `def ensure_readonly_user(app_settings: Any=settings) -> None`
Create (idempotently) the genuinely lower-privileged credential
- ▶ `def apply_schema(conn) -> None`
Apply the schema.
- ▶ `def seed_d365_finance(conn) -> dict[str, int]`
Seed the d365 finance.
- ▶ `def seed_dynamics_crm(conn) -> dict[str, int]`
Seed the dynamics crm.
- ▶ `def seed_all(app_settings=settings) -> dict[str, int]`
Seed the all.

app/mcp/business_records/server.py — 189 lines

Business Records MCP server — a real, persistent MySQL-backed connector.

- ▶ `def backend() -> MySQLConnectionAbstract`
Compute the backend.
- ▶ `async def list_tools() -> list[types.Tool]`
List the tools.
- ▶ `async def call_tool(name: str, arguments: dict[str, Any] | None) -> Any`
Compute the call tool.
- ▶ `def _error_result(error: dict[str, Any]) -> types.CallToolResult`
Internal helper for the error result step.
- ▶ `async def main() -> None`
Compute the main.

app/mcp/business_records/sql_guard.py — 162 lines

Defense-in-depth for the query_readonly MCP tool.

- ▶ `class SQLGuardError`
Exception raised for the SQLGuardError case.
- ▶ `def classify_sql_verb(sql: str) -> str`
"read" or "write", from the statement's first keyword alone.
- ▶ `async def run_readonly_query(sql: str, params: dict[str, Any], *, connect: Any, max_rows: int, timeout_seconds: float) -> dict[str, Any]`
Run one read-only query with every layer above applied.
- ▶ `async def _kill_query(connect: Any, connection_id: int) -> None`
A second, short-lived connection to cancel the first's running

Tool definitions for the business-records MySQL server.

- ▶ `def _string(description: str, **extra: Any) -> dict[str, Any]`
Internal helper for the string step.
- ▶ `def _collection(key: str, item_schema: dict[str, Any]) -> dict[str, Any]`
Every collection-returning tool shares this envelope.
- ▶ `def _customer_item() -> dict[str, Any]`
Internal helper for the customer item step.
- ▶ `def _order_item() -> dict[str, Any]`
Internal helper for the order item step.
- ▶ `def _inventory_item() -> dict[str, Any]`
Internal helper for the inventory item step.
- ▶ `def _product_item() -> dict[str, Any]`
Internal helper for the product item step.
- ▶ `def _case_record() -> dict[str, Any]`
Internal helper for the case record step.
- ▶ `def _opportunity_record() -> dict[str, Any]`
Internal helper for the opportunity record step.
- ▶ `def _order_record() -> dict[str, Any]`
Internal helper for the order record step.

mcp.d365_finance (4 modules)

app/mcp/d365_finance/__init__.py — 15 lines

Fixture-backed mock for the d365-finance-scm-mcp business-tool layer.

app/mcp/d365_finance/handlers.py — 705 lines

Fixture-backed handlers for the d365-finance-scm-mcp business-tool layer.

- ▶ `def _collection(rows: list[dict[str, Any]], key: str, limit: int) -> dict[str, Any]`
Internal helper for the collection step.
- ▶ `def _limit(arguments: dict[str, Any], default: int, maximum: int) -> int`
Internal helper for the limit step.
- ▶ `def _any_of_present(*filters: str | None) -> str | None`
Internal helper for the any of present step.
- ▶ `def _priority_filter(*fields: tuple[str, str | None]) -> str | None`
Search by the first-given reference only.
- ▶ `def _customer(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the customer step.
- ▶ `async def find_customer(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Search customers by name — a customer writing 'Meridian' should find
- ▶ `async def find_account_ownership(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Resolve account ownership to people who can actually be assigned today.
- ▶ `async def find_credit_status(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the credit status.
- ▶ `def _quote(row: dict[str, Any], *, account_id: str, purchase_order_number: str) -> dict[str, Any]`
Internal helper for the quote step.
- ▶ `async def find_quote(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find a quotation by quotation number or purchase-order number.
- ▶ `def _order_filter(arguments: dict[str, Any]) -> str | None`
Customers quote their own purchase-order number at least as often as
- ▶ `def _sales_order(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the sales order step.
- ▶ `async def find_sales_order(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the sales order.
- ▶ `async def find_order_fulfilment_status(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Ask why an order is where it is — the customer's wording is identical
- ▶ `def _availability(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the availability step.
- ▶ `async def _resolve_pump_models(backend: DynamicsBackend, arguments: dict[str, Any]) -> list[str]`
product_name/product_family describe a catalogue product, not an
- ▶ `async def find_inventory_availability(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the inventory availability.
- ▶ `async def find_installed_unit(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Resolve installed equipment by serial, model, manufacturer or site.
- ▶ `def _shipment(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the shipment step.
- ▶ `async def find_shipment(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the shipment.

- `def _invoice(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the invoice step.
- `async def find_invoice(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the invoice.
- `def _contract(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the contract step.
- `async def find_contract(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the contract.
- `def _product(row: dict[str, Any]) -> dict[str, Any]`
Internal helper for the product step.
- `async def find_products(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the products.

app/mcp/d365_finance/server.py — 173 lines

Fixture-backed mock for the Dynamics 365 Finance & Operations MCP server.

- `def backend() -> DynamicsBackend`
Compute the backend.
- `async def list_tools() -> list[types.Tool]`
List the tools.
- `async def call_tool(name: str, arguments: dict[str, Any] | None) -> Any`
Compute the call tool.
- `def _error_result(error: dict[str, Any]) -> types.CallToolResult`
Internal helper for the error result step.
- `async def main() -> None`
Compute the main.

app/mcp/d365_finance/tools.py — 319 lines

Atomic read tools for the fixture-backed D365 Finance & SCM connector.

- `def _string(description: str, **extra: Any) -> dict[str, Any]`
Internal helper for the string step.
- `def _collection(key: str) -> dict[str, Any]`
Internal helper for the collection step.

mcp.dynamics (6 modules)

app/mcp/dynamics/__init__.py — 0 lines

(no summary)

app/mcp/dynamics/client.py — 802 lines

Dataverse Web API client and its fixture-backed twin.

```
▶ class DynamicsError
    A CRM call failed in a way worth describing to a workflow author.
    def as_payload(self) -> dict[str, Any]
        Compute the as payload.
▶ class DynamicsAuthError
    Exception raised for the DynamicsAuthError case.
▶ class DynamicsBackend
    What the MCP server needs from a CRM backend.
    async def whoami(self) -> dict[str, Any]
        Compute the whoami.
    async def query(self, entity_set: str, *, select: list[str], filter_expression: str |
None=None, order_by: str | None=None, top: int | None=None, expand: str | None=None) ->
list[dict[str, Any]]
        Query the result.
    async def get(self, entity_set: str, record_id: str, *, select: list[str]) -> dict[str, Any] |
None
        Return the result.
    async def create(self, entity_set: str, payload: dict[str, Any]) -> str
        Create the result.
    async def update(self, entity_set: str, record_id: str, payload: dict[str, Any]) -> None
        Update the result.
    async def close(self) -> None
        Close the result.
▶ class DataverseClient
    Live Dynamics 365 over the Dataverse Web API.
    def _http(self) -> httpx.AsyncClient
        Internal helper for the http step.
    async def close(self) -> None
        Close the result.
    async def _access_token(self) -> str
        Client-credentials token for the organisation's `default` scope.
    async def _headers(self, *, write: bool=False) -> dict[str, str]
        Internal helper for the headers step.
    def _url(self, path: str, params: dict[str, str] | None=None) -> str
        Internal helper for the url step.
    async def _request(self, method: str, path: str, *, params: dict[str, str] | None=None,
json_body: dict[str, Any] | None=None, write: bool=False) -> httpx.Response
        Internal helper for the request step.
    async def whoami(self) -> dict[str, Any]
        Compute the whoami.
    async def query(self, entity_set: str, *, select: list[str], filter_expression: str |
None=None, order_by: str | None=None, top: int | None=None, expand: str | None=None) ->
list[dict[str, Any]]
        Query the result.
```

```

    async def get(self, entity_set: str, record_id: str, *, select: list[str]) -> dict[str, Any] |
None
    Return the result.
    async def create(self, entity_set: str, payload: dict[str, Any]) -> str
    Create the result.
    async def update(self, entity_set: str, record_id: str, payload: dict[str, Any]) -> None
    Update the result.
▶ class FixtureBackend
    The same contracts, served from a fixture file.
    def from_file(cls, path: str | Path) -> 'FixtureBackend'
    Compute the from file.
    def _next_id(self, prefix: str) -> str
    Internal helper for the next id step.
    async def whoami(self) -> dict[str, Any]
    Compute the whoami.
    async def query(self, entity_set: str, *, select: list[str], filter_expression: str |
None=None, order_by: str | None=None, top: int | None=None, expand: str | None=None) ->
list[dict[str, Any]]
    Query the result.
    async def get(self, entity_set: str, record_id: str, *, select: list[str]) -> dict[str, Any] |
None
    Return the result.
    async def create(self, entity_set: str, payload: dict[str, Any]) -> str
    Create the result.
    async def update(self, entity_set: str, record_id: str, payload: dict[str, Any]) -> None
    Update the result.
▶ def _safe_error_text(response: httpx.Response) -> str
    Extract Dataverse's error message without echoing an entire payload.
▶ def _project(row: dict[str, Any], select: list[str]) -> dict[str, Any]
    Mimic '$select': a fixture row returns only the requested columns.
▶ def _fixture_matches(row: dict[str, Any], expression: str) -> bool
    Interpret the small OData subset this package generates.
▶ def _split_top_level(expression: str, joiner: str) -> list[str]
    Split on a joiner that is not inside parentheses or a string literal.

```

app/mcp/dynamics/handlers.py — 819 lines

Tool implementations: business question in, typed record out.

```

▶ def _account(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the account step.
▶ def _contact(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the contact step.
▶ def _opportunity(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the opportunity step.
▶ def _order(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the order step.
▶ def _product(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the product step.
▶ def _activity(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the activity step.
▶ def _shipment(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the shipment step.
▶ def _service_case(row: dict[str, Any]) -> dict[str, Any]
    Internal helper for the service case step.
▶ def _collection(rows: list[dict[str, Any]], key: str, limit: int) -> dict[str, Any]
    Wrap results in the shared envelope.

```

- ▶ `def _limit(arguments: dict[str, Any], default: int, maximum: int) -> int`
Internal helper for the limit step.
- ▶ `async def get_current_user(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the current user.
- ▶ `async def find_account(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the account.
- ▶ `async def get_account(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the account.
- ▶ `async def find_contact(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the contact.
- ▶ `async def get_contacts_for_account(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the contacts for account.
- ▶ `async def get_open_opportunities(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the open opportunities.
- ▶ `async def find_previous_orders(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the previous orders.
- ▶ `async def find_product(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the product.
- ▶ `async def find_shipment(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the shipment.
- ▶ `async def find_service_case(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Find the service case.
- ▶ `async def get_service_cases_for_account(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the service cases for account.
- ▶ `async def get_recent_activities(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Return the recent activities.
- ▶ `def _text(arguments: dict[str, Any], key: str, limit: int) -> str | None`
Read an optional write field, bounded.
- ▶ `async def create_lead(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Create the lead.
- ▶ `async def create_followup_activity(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Create the followup activity.
- ▶ `async def update_account_contact_details(backend: DynamicsBackend, arguments: dict[str, Any]) -> dict[str, Any]`
Update the account contact details.

`app/mcp/dynamics/odata.py` — 220 lines

Safe construction of Dataverse Web API queries.

- ▶ `class ODataValueError`
A caller-supplied value cannot be used safely in a query.
- ▶ `def guid(value: Any, *, field: str='id') -> str`
Validate an identifier as a GUID and return its canonical form.
- ▶ `def escape_literal(value: Any, *, field: str='value') -> str`
Escape a string for use inside an OData string literal.
- ▶ `def string_filter(field: str, value: str) -> str`
`field eq 'value'` with the value escaped.
- ▶ `def contains_filter(field: str, value: str) -> str`
`contains(field,'value')` with the value escaped.

- ▶ `def lookup_filter(field: str, record_id: str) -> str`
`_lookup_value eq <guid>` — the case the reference implementation gets
- ▶ `def any_of(*filters: str) -> str`
OR the given filters, parenthesised so precedence cannot surprise.
- ▶ `def all_of(*filters: str) -> str`
Compute the all of.
- ▶ `def entity_path(entity_set: str, record_id: str | None=None) -> str`
Build an entity path, validating the key predicate.
- ▶ `def build_query(*, select: list[str] | None=None, filter_expression: str | None=None, order_by: str | None=None, top: int | None=None, expand: str | None=None) -> dict[str, str]`
Assemble OData system query options.
- ▶ `def _column(name: str) -> str`
Internal helper for the column step.
- ▶ `def _order_by(clause: str) -> str`
Internal helper for the order by step.
- ▶ `def encode_params(params: dict[str, str]) -> str`
Percent-encode query values while leaving OData's own syntax readable.

app/mcp/dynamics/server.py — 264 lines

The Dynamics 365 MCP server.

- ▶ `def build_backend() -> DynamicsBackend`
Choose a backend from the environment.
- ▶ `def backend() -> DynamicsBackend`
Compute the backend.
- ▶ `def _annotations(definition: dict[str, Any]) -> types.ToolAnnotations`
Declare the tool's nature honestly.
- ▶ `async def list_tools() -> list[types.Tool]`
List the tools.
- ▶ `async def call_tool(name: str, arguments: dict[str, Any] | None) -> Any`
Execute one tool.
- ▶ `def _error_result(error: dict[str, Any]) -> types.CallToolResult`
Internal helper for the error result step.
- ▶ `async def main() -> None`
Compute the main.

app/mcp/dynamics/tools.py — 948 lines

The Dynamics 365 CRM tool vocabulary.

- ▶ `def _string(description: str, **extra: Any) -> dict[str, Any]`
Internal helper for the string step.
- ▶ `def _account_summary() -> dict[str, Any]`
Internal helper for the account summary step.
- ▶ `def _contact_summary() -> dict[str, Any]`
Internal helper for the contact summary step.
- ▶ `def _opportunity_summary() -> dict[str, Any]`
Internal helper for the opportunity summary step.
- ▶ `def _product_summary() -> dict[str, Any]`
Internal helper for the product summary step.
- ▶ `def _order_summary() -> dict[str, Any]`
Internal helper for the order summary step.
- ▶ `def _activity_summary() -> dict[str, Any]`
Internal helper for the activity summary step.
- ▶ `def _owner_summary() -> dict[str, Any]`
Internal helper for the owner summary step.

- ▶ `def _quote_summary() -> dict[str, Any]`
Internal helper for the quote summary step.
- ▶ `def _shipment_summary() -> dict[str, Any]`
Internal helper for the shipment summary step.
- ▶ `def _service_case_summary() -> dict[str, Any]`
Internal helper for the service case summary step.
- ▶ `def _collection(item_schema: dict[str, Any], key: str) -> dict[str, Any]`
Every collection-returning tool shares this envelope.

nodes (61 modules)

app/nodes/__init__.py — 52 lines

Built-in workflow-node discovery.

- ▶ `def discover_nodes(*, force: bool=False) -> dict[str, str]`
Import all built-in node modules and return module import failures.
- ▶ `def node_discovery_errors() -> dict[str, str]`
Return a copy so callers cannot mutate discovery state.

app/nodes/_stubs.py — 85 lines

Stub nodes used only to test the runtime in isolation from LLMs/RAG.

- ▶ `class _Empty`
Pydantic model defining the Empty shape.
- ▶ `class _LiteralConfig`
Pydantic model defining the LiteralConfig shape.
- ▶ `class _LiteralOutput`
Pydantic model defining the LiteralOutput shape.
- ▶ `class LiteralNode`
Emits whatever literal is in its config. Useful for smoke tests.
`async def run(self, state, resolved_config)`
Run the result.
- ▶ `class _EchoConfig`
Pydantic model defining the EchoConfig shape.
- ▶ `class _EchoOutput`
Pydantic model defining the EchoOutput shape.
- ▶ `class EchoNode`
Renders a template string against state. Useful for testing templating.
`async def run(self, state, resolved_config)`
Run the result.

app/nodes/about_synthesis.py — 224 lines

Fills in `about` metadata for node types that don't declare their own.

- ▶ `def _repo_root() -> Path`
Internal helper for the repo root step.
- ▶ `def _edge_targets(edge: dict[str, Any]) -> list[str]`
Internal helper for the edge targets step.
- ▶ `def _adjacency() -> dict[str, dict[str, Any]]`
`type_name -> {"upstream": [...], "downstream": [...], "example": str|None},`
- ▶ `def _schema_fields_summary(schema: Type[Any] | None) -> str`
Internal helper for the schema fields summary step.
- ▶ `def _important_config_fields(config_schema: Type[Any] | None) -> list[str]`
Internal helper for the important config fields step.
- ▶ `def synthesize_about(klass: 'Type[NodeType]') -> dict[str, Any]`
Best-effort `about` dict for a node type that didn't author its own.

app/nodes/ai_task.py — 855 lines

AITaskAgent — one configurable AI capability.

- ▶ `class LanguagePolicy`

```

Multilingual behaviour, configured on the AI Task rather than modelled as
def directive(self) -> str
    Compute the directive.
▶ class AITaskExample
    A few-shot example. Output is free-form so the same field works for both
▶ class AITaskConfig
    Pydantic model defining the AITaskConfig shape.
    def structured_tasks_declare_a_schema(self) -> 'AITaskConfig'
        Compute the structured tasks declare a schema.
▶ class AITaskInput
    Pydantic model defining the AITaskInput shape.
▶ class AITaskOutput
    The typed contract every downstream node sees.
▶ class AITaskAgent
    Workflow node type implementing the AITaskAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
        Authorise `result.<path>` references for the author's own schema.
    def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
        Compute the preflight static output values.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
    def _system_prompt(self, cfg: AITaskConfig, fields: list[FieldSpec]) -> str
        Internal helper for the system prompt step.
    def _user_prompt(self, cfg: AITaskConfig, fields: list[FieldSpec]) -> str
        Internal helper for the user prompt step.
    def _examples_block(cfg: AITaskConfig) -> str
        Internal helper for the examples block step.
    async def _run_text(self, cfg: AITaskConfig, llm: Any, system: str, user: str) -> dict[str,
Any]
        Run the text.
    async def _run_structured(self, cfg: AITaskConfig, llm: Any, system: str, user: str, fields:
list[FieldSpec]) -> dict[str, Any]
        Run the structured.
    def _success(self, cfg: AITaskConfig, parsed: dict[str, Any], attempt: int) -> dict[str, Any]
        Internal helper for the success step.
    def _failure(self, cfg: AITaskConfig, status: str, message: str, *, attempts: int) -> dict[str,
Any]
        Internal helper for the failure step.
▶ def _confidence_fields(fields: list[FieldSpec]) -> list[FieldSpec]
    Internal helper for the confidence fields step.
▶ def effective_fields(config: dict[str, Any]) -> list[FieldSpec]
    The schema the model is actually held to, generated fields included.
▶ def _detected_language(parsed: dict[str, Any]) -> str | None
    Surface a language field from the author's schema as a runtime field.
▶ def _looks_like_refusal(error: Exception) -> bool
    Distinguish a model declining the task from a malformed response.

```

app/nodes/approval.py — 33 lines

Shared "did a human already approve this?" check.

```

▶ def human_approved(state: dict[str, Any], approved_by: str | None=None) -> bool
    Did a human review approve something earlier on this run's path?

```

app/nodes/base.py — 116 lines

Base contract for every node type in the platform.

- ▶ class NodeType
Abstract base defining the NodeType contract.
 async def run(self, state: 'WorkflowState', resolved_config: dict[str, Any]) -> dict[str, Any]
 Execute the node.
 def required_services(cls, config: dict[str, Any]) -> set[str]
 Service names this node needs at runtime, given its resolved
 def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
 Valid dotted template-reference suffixes for this node's output,
 def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
 Output fields whose value preflight can already prove, without

app/nodes/bounded_deep_research_agent.py — 506 lines

Run several skill-guided bounded research jobs within hard budgets.

- ▶ class BoundedDeepResearchInput
Pydantic model defining the BoundedDeepResearchInput shape.
- ▶ class BoundedDeepResearchConfig
Pydantic model defining the BoundedDeepResearchConfig shape.
 def _coerce_briefs(cls, value: Any) -> Any
 Internal helper for the coerce briefs step.
- ▶ class BoundedDeepResearchOutput
Pydantic model defining the BoundedDeepResearchOutput shape.
- ▶ class BoundedDeepResearchAgent
Workflow node type implementing the BoundedDeepResearchAgent capability.
 def required_services(cls, config: dict[str, Any]) -> set[str]
 Compute the required services.
 async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
 Run the result.
 def _record_costs(self, state: dict[str, Any], dossiers: list[ResearchDossier]) -> None
 Record the costs.
- ▶ def _candidate_records(dossiers: list[ResearchDossier], *, max_citations_per_brief: int, max_candidates_per_claim: int) -> list[CandidateSource]
 Internal helper for the candidate records step.
- ▶ def _search_audit(briefs: list[ResearchBrief], dossiers: list[ResearchDossier], failures: list[dict[str, str]]) -> list[SearchAuditRecord]
 Search the audit.

app/nodes/call_coverage.py — 63 lines

Workflow node that computes and writes the deterministic call matrix.

- ▶ class CallCoverageInput
Pydantic model defining the CallCoverageInput shape.
- ▶ class CallCoverageConfig
Pydantic model defining the CallCoverageConfig shape.
- ▶ class CallCoverageMatrixAgent
Workflow node type implementing the CallCoverageMatrixAgent capability.
 async def run(self, state: dict, resolved_config: dict) -> dict
 Run the result.

app/nodes/categories.py — 214 lines

Category taxonomy for the node palette (Builder) and Cockpit's node

- `def category_for(type_name: str) -> str`
Compute the category for.
- `def icon_for(type_name: str) -> str`
Compute the icon for.
- `def family_for(type_name: str) -> str`
Compute the family for.
- `def execution_kind_for(type_name: str, *, uses_llm: bool) -> str`
Compute the execution kind for.

app/nodes/citation_registry_builder.py — 298 lines

CitationRegistryBuilder: deterministic reshaping of acquired full-text

- `class CitationRegistryBuilderInput`
Pydantic model defining the CitationRegistryBuilderInput shape.
- `class CitationRegistryBuilderConfig`
Pydantic model defining the CitationRegistryBuilderConfig shape.
`def _coerce_documents(cls, value: Any) -> Any`
Internal helper for the coerce documents step.
- `class CitationEntry`
Pydantic model defining the CitationEntry shape.
- `class CitationRegistryBuilderOutput`
Pydantic model defining the CitationRegistryBuilderOutput shape.
- `class CitationRegistryBuilder`
Workflow node type implementing the CitationRegistryBuilder capability.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/claim_evidence_verifier.py — 231 lines

Verify linked proposal claims against exact, versioned source passages.

- `class ClaimEvidenceVerifierInput`
Pydantic model defining the ClaimEvidenceVerifierInput shape.
- `class ClaimEvidenceVerifierConfig`
Pydantic model defining the ClaimEvidenceVerifierConfig shape.
- `class ClaimEvidenceVerifierOutput`
Pydantic model defining the ClaimEvidenceVerifierOutput shape.
- `class ClaimEvidenceVerifier`
Workflow node type implementing the ClaimEvidenceVerifier capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def _source_text(self, source, cap: int) -> tuple[str, str]`
Internal helper for the source text step.
`async def run(self, state: dict, resolved_config: dict[str, Any]) -> dict`
Run the result.

app/nodes/concept_alternatives.py — 112 lines

Workflow node for three grounded Horizon concept alternatives.

- `class ConceptAlternativesInput`
Pydantic model defining the ConceptAlternativesInput shape.
- `class ConceptAlternativesConfig`
Pydantic model defining the ConceptAlternativesConfig shape.
- `class ConceptAlternativesOutput`
Provides the ConceptAlternativesOutput behaviour.
- `class ConceptAlternativesAgent`

Workflow node type implementing the ConceptAlternativesAgent capability.

```
def required_services(cls, config: dict[str, Any]) -> set[str]
    Compute the required services.
async def run(self, state: dict, resolved_config: dict) -> dict
    Run the result.
```

app/nodes/concept_freeze.py — 103 lines

Deterministic freeze step for Human Gate 3 ("select and freeze the concept").

- ▶ `class ConceptFreezeInput`
Pydantic model defining the ConceptFreezeInput shape.
- ▶ `class ConceptFreezeConfig`
Pydantic model defining the ConceptFreezeConfig shape.
`def _coerce_alternatives(cls, value: Any) -> Any`
Internal helper for the coerce alternatives step.
- ▶ `class ConceptFreezeOutput`
Pydantic model defining the ConceptFreezeOutput shape.
- ▶ `class ConceptFreezeAgent`
Workflow node type implementing the ConceptFreezeAgent capability.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/consistency_checker.py — 196 lines

ConsistencyChecker — a DETERMINISTIC node (no LLM) that gates the render.

- ▶ `class ConsistencyCheckerOutput`
Validated output for ConsistencyChecker.run() (output_schema ClassVar).
- ▶ `class ConsistencyCheckerInput`
Pydantic model defining the ConsistencyCheckerInput shape.
- ▶ `class ConsistencyCheckerConfig`
Node config — base.__init__ does config_schema(**raw_config).
- ▶ `class ConsistencyChecker`
Deterministic gate. Reads the proposal_graph from state, returns a
`async def run(self, state: dict, config: dict) -> dict`
Run the result.
- ▶ `def _check(graph: ProposalGraph) -> list[dict[str, Any]]`
Check the result.

app/nodes/data_transform.py — 441 lines

DataTransformAgent — deterministic data shaping. No model call.

- ▶ `class TransformOperation`
One deterministic assignment into the output object.
`def operation_has_its_inputs(self) -> 'TransformOperation'`
Compute the operation has its inputs.
- ▶ `class DataTransformConfig`
Pydantic model defining the DataTransformConfig shape.
`def targets_are_unique(self) -> 'DataTransformConfig'`
Compute the targets are unique.
- ▶ `class DataTransformInput`
Pydantic model defining the DataTransformInput shape.
- ▶ `class DataTransformOutput`
Pydantic model defining the DataTransformOutput shape.
- ▶ `class DataTransformAgent`
Workflow node type implementing the DataTransformAgent capability.

```

def required_services(cls, config: dict[str, Any]) -> set[str]
    Compute the required services.
def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
    Compute the preflight output fields.
def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
    Compute the preflight static output values.
async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
    Run the result.
▶ def _is_blank(value: Any) -> bool
    Return whether blank.
▶ def _apply(op: TransformOperation, context: dict[str, Any]) -> Any
    Apply the result.
▶ def _as_text(value: Any) -> str
    Internal helper for the as text step.
▶ def _as_number(value: Any) -> float | None
    Internal helper for the as number step.
▶ def _as_boolean(value: Any) -> bool
    Internal helper for the as boolean step.

```

app/nodes/decision.py — 278 lines

DecisionAgent — deterministic business logic, no model call.

```

▶ class DecisionConfig
    Pydantic model defining the DecisionConfig shape.
    def output_field_names(self) -> set[str]
        Compute the output field names.
▶ class DecisionInput
    Pydantic model defining the DecisionInput shape.
▶ class DecisionOutput
    Facts the rules established, plus the reasoning behind them.
▶ class DecisionAgent
    Workflow node type implementing the DecisionAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
        Authorise `decisions.<field>` for every field the rules can set.
    def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
        Compute the preflight static output values.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.

```

app/nodes/docx_renderer.py — 408 lines

app/nodes/docx_renderer.py

```

▶ class DOCXRenderInput
    Pydantic model defining the DOCXRenderInput shape.
▶ class DOCXRenderConfig
    Pydantic model defining the DOCXRenderConfig shape.
▶ class DOCXRenderOutput
    Pydantic model defining the DOCXRenderOutput shape.
▶ class DOCXProposalRenderer
    Workflow node type implementing the DOCXProposalRenderer capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]

```



```

    Run the result.
    def _render_docx_bytes(self, sections: dict[str, str], template_name: str, client: str, title:
str) -> bytes
    Render the docx bytes.
    def _add_title_block(self, doc, title, client, tpl) -> None
    Add the title block.
    def _style_heading(self, paragraph, tpl, top_level: bool) -> None
    Internal helper for the style heading step.
    def _render_markdown(self, doc, md: str, tpl) -> None
    Render the markdown.
    def _render_table(self, doc, lines, start, tpl) -> int
    Render the table.
    def _add_inline(self, paragraph, text: str, bold: bool=False) -> None
    Add the inline.

```

app/nodes/dynamic_figure_agent.py — 337 lines

DynamicFigureAgent: generate an image for every inline image marker and

```

▶ class DynamicFigureAgentInput
    Pydantic model defining the DynamicFigureAgentInput shape.
▶ class DynamicFigureAgentConfig
    Pydantic model defining the DynamicFigureAgentConfig shape.
▶ class GeneratedFigure
    Pydantic model defining the GeneratedFigure shape.
▶ class DynamicFigureAgentOutput
    Pydantic model defining the DynamicFigureAgentOutput shape.
▶ class DynamicFigureAgent
    Workflow node type implementing the DynamicFigureAgent capability.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
    Run the result.
▶ def _sub_captioned(content: str) -> str
    Internal helper for the sub captioned step.
▶ def _escape(text: str) -> str
    Internal helper for the escape step.

```

app/nodes/email_integration.py — 432 lines

EmailAgent — one email capability with an operation selector.

```

▶ class EmailRecipient
    Pydantic model defining the EmailRecipient shape.
▶ class EmailNodeConfig
    Pydantic model defining the EmailNodeConfig shape.
    def operation_has_what_it_needs(self) -> 'EmailNodeConfig'
    Compute the operation has what it needs.
▶ class EmailNodeInput
    Pydantic model defining the EmailNodeInput shape.
▶ class EmailMessageSummary
    The message shape downstream nodes address.
▶ class EmailNodeOutput
    Pydantic model defining the EmailNodeOutput shape.
▶ class EmailAgent
    Workflow node type implementing the EmailAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
    Compute the required services.
    def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]

```

Authorise the message shape as dotted paths.

```
async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
```

Run the result.

- ▶ `def _criteria(cfg: EmailNodeConfig) -> EmailSearchCriteria`
Internal helper for the criteria step.
- ▶ `def _draft(cfg: EmailNodeConfig) -> EmailDraft`
Draft the result.
- ▶ `def _summarise(message: Any) -> EmailMessageSummary`
Internal helper for the summarise step.
- ▶ `def is_side_effect(operation: str) -> bool`
Whether this operation changes something outside the platform.

app/nodes/end.py — 221 lines

EndAgent: the canonical workflow exit point.

- ▶ `class EndOutputField`
Pydantic model defining the EndOutputField shape.
- ▶ `class EndConfig`
Pydantic model defining the EndConfig shape.
- ▶ `class EndInput`
Pydantic model defining the EndInput shape.
- ▶ `class EndOutput`
Pydantic model defining the EndOutput shape.
- ▶ `class EndAgent`
Workflow node type implementing the EndAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]`
Compute the preflight output fields.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _humanize(value: str) -> str`
Internal helper for the humanize step.

app/nodes/evidence_agent.py — 502 lines

Scholarly candidate discovery for proposal claims.

- ▶ `class ClaimSearchPlan`
Pydantic model defining the ClaimSearchPlan shape.
- ▶ `class ScholarlyCandidateDiscoveryInput`
Pydantic model defining the ScholarlyCandidateDiscoveryInput shape.
- ▶ `class ScholarlyCandidateDiscoveryConfig`
Pydantic model defining the ScholarlyCandidateDiscoveryConfig shape.
- ▶ `class ScholarlyCandidateDiscoveryOutput`
Pydantic model defining the ScholarlyCandidateDiscoveryOutput shape.
- ▶ `class _ScholarlyCandidateDiscovery`
Workflow node type implementing the ScholarlyCandidateDiscovery capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
`def _query_planning_guidance(self, cfg: ScholarlyCandidateDiscoveryConfig) -> str`
Search methodology text from the configured Scientific Agent Skill.
`async def _search_plan(self, llm: Any, claim: Claim, model: str | None, *, skill_guidance: str='') -> ClaimSearchPlan`

Search the plan.

- ▶ `class ScholarlyCandidateDiscoveryAgent`
Provides the ScholarlyCandidateDiscoveryAgent behaviour.
- ▶ `def _sanitize_query(text: str) -> str`
Strip boolean/advanced-search syntax from a generated query.

`app/nodes/excel_tool.py` — 89 lines

Excel tool node — pre-baked variant: extract tables from .xlsx in MinIO.

- ▶ `class ExcelInput`
Pydantic model defining the ExcelInput shape.
- ▶ `class ExcelConfig`
Pydantic model defining the ExcelConfig shape.
- ▶ `class ExcelOutput`
Pydantic model defining the ExcelOutput shape.
- ▶ `class ExcelTableExtractor`
Workflow node type implementing the ExcelTableExtractor capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

`app/nodes/external_action.py` — 275 lines

ExternalActionAgent — call a REST API or a webhook, kept apart from MCP.

- ▶ `class ExternalActionConfig`
Pydantic model defining the ExternalActionConfig shape.
- ▶ `class ExternalActionInput`
Pydantic model defining the ExternalActionInput shape.
- ▶ `class ExternalActionOutput`
Pydantic model defining the ExternalActionOutput shape.
- ▶ `class ExternalActionAgent`
Workflow node type implementing the ExternalActionAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
`def _failure(self, cfg: ExternalActionConfig, error: Exception) -> dict[str, Any]`
Internal helper for the failure step.

`app/nodes/figure_embedder.py` — 177 lines

Splice generated figures into compiled proposal text before rendering.

- ▶ `class FigureSpec`
Pydantic model defining the FigureSpec shape.
- ▶ `class FigureEmbedderInput`
Pydantic model defining the FigureEmbedderInput shape.
- ▶ `class FigureEmbedderConfig`
Pydantic model defining the FigureEmbedderConfig shape.
- ▶ `class FigureEmbedderOutput`
Pydantic model defining the FigureEmbedderOutput shape.
- ▶ `class FigureEmbedder`
Workflow node type implementing the FigureEmbedder capability.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

- ▶ `def _marker_pattern(marker: str) -> re.Pattern[str]`
Internal helper for the marker pattern step.

app/nodes/graph_normalizer.py — 341 lines

GraphNormalizer — turns source text into VALIDATED typed ProposalGraph objects.

- ▶ `class GraphNormalizerOutput`
What GraphNormalizer.run() writes back. The compiler validates the
- ▶ `class GraphNormalizerInput`
Pydantic model defining the GraphNormalizerInput shape.
- ▶ `class GraphNormalizerConfig`
Node config — `base.__init__` does `config_schema(**raw_config)`.
- ▶ `class GraphExtraction`
Provider-enforced extraction envelope.
- ▶ `class GraphNormalizer`
LLM extracts structure from source text; pydantic validates it; valid
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict, config: dict) -> dict`
Run the result.
`def _parse_json(raw: str) -> dict`
Parse the LLM's JSON and fail loudly when extraction is malformed.

app/nodes/horizon_docx_renderer.py — 265 lines

Horizon proposal HTML/Markdown to editable DOCX workflow node.

- ▶ `class HorizonDOCXProposalRendererInput`
Pydantic model defining the HorizonDOCXProposalRendererInput shape.
- ▶ `class HorizonDOCXProposalRendererConfig`
Pydantic model defining the HorizonDOCXProposalRendererConfig shape.
- ▶ `class HorizonDOCXProposalRendererOutput`
Pydantic model defining the HorizonDOCXProposalRendererOutput shape.
- ▶ `class HorizonDOCXProposalRenderer`
Workflow node type implementing the HorizonDOCXProposalRenderer capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _safe_component(value: str) -> str`
Internal helper for the safe component step.

app/nodes/horizon_evaluation.py — 88 lines

Workflow node for independent two-provider Horizon evaluation.

- ▶ `class HorizonEvaluationInput`
Pydantic model defining the HorizonEvaluationInput shape.
- ▶ `class HorizonEvaluationConfig`
Pydantic model defining the HorizonEvaluationConfig shape.
- ▶ `class HorizonEvaluationAgent`
Workflow node type implementing the HorizonEvaluationAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict, resolved_config: dict) -> dict`
Run the result.

app/nodes/html_proposal_renderer.py — 240 lines

Horizon proposal HTML/Markdown to PDF workflow node.

- ▶ `class HorizonHTMLProposalRendererInput`
Pydantic model defining the HorizonHTMLProposalRendererInput shape.
- ▶ `class HorizonHTMLProposalRendererConfig`
Pydantic model defining the HorizonHTMLProposalRendererConfig shape.
- ▶ `class HorizonHTMLProposalRendererOutput`
Pydantic model defining the HorizonHTMLProposalRendererOutput shape.
- ▶ `class HorizonHTMLProposalRenderer`
Workflow node type implementing the HorizonHTMLProposalRenderer capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _safe_component(value: str) -> str`
Internal helper for the safe component step.

app/nodes/human_in_loop.py — 633 lines

HumanInLoopAgent: pause and wait for user decision.

- ▶ `class ReviewPanel`
One labelled block the reviewer sees (§29).
- ▶ `class HITLConfig`
Pydantic model defining the HITLConfig shape.
- ▶ `class HITLInput`
Pydantic model defining the HITLInput shape.
- ▶ `class HITLReviewContent`
Canonical review document passed between the UI and workflow runtime.
- ▶ `class HITLOutput`
Pydantic model defining the HITLOutput shape.
- ▶ `class ReviewPanelValue`
A resolved review panel, ready to render.
- ▶ `class HITLInterruptPayload`
What the Cockpit sees when this node pauses.
- ▶ `class HumanInLoopAgent`
Workflow node type implementing the HumanInLoopAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `class _RichTextSanitizer`
Small allow-list sanitizer for browser-generated editor markup.
`def handle_starttag(self, tag: str, attrs: list[tuple[str, str | None]]) -> None`
Handle the starttag.
`def handle_startendtag(self, tag: str, attrs: list[tuple[str, str | None]]) -> None`
Handle the startendtag.
`def handle_endtag(self, tag: str) -> None`
Handle the endtag.
`def handle_data(self, data: str) -> None`
Handle the data.
- ▶ `def _resolve_path(path: str, state: dict) -> Any`
Resolve the path.
- ▶ `def _review_panels(cfg: HITLConfig, state: dict[str, Any]) -> list[ReviewPanelValue]`
Resolve each configured panel against state.

- ▶ `def _review_content(cfg: HITLConfig, state: dict[str, Any], context: dict[str, Any]) -> HITLReviewContent | None`
Internal helper for the review content step.
- ▶ `def _content_format(value: Any) -> Literal['text', 'json']`
Internal helper for the content format step.
- ▶ `def _content_as_text(value: Any) -> str`
Internal helper for the content as text step.
- ▶ `def _edited_value(content: HITLReviewContent) -> Any`
Restore structured editor content before patching workflow state.
- ▶ `def _patch_reviewed_content(state: dict[str, Any], path: str, value: Any) -> dict[str, Any]`
Replace only the configured editable path for downstream nodes.
- ▶ `def _set_nested(container: Any, parts: list[str], value: Any) -> None`
Set the nested.
- ▶ `def sanitize_rich_html(value: str | None) -> str | None`
Sanitize the rich html.

app/nodes/integration.py — 459 lines

IntegrationAgent — one cloud-storage capability with a provider selector.

- ▶ `class IntegrationNodeConfig`
Pydantic model defining the IntegrationNodeConfig shape.
`def operation_has_what_it_needs(self) -> 'IntegrationNodeConfig'`
Compute the operation has what it needs.
- ▶ `class IntegrationNodeInput`
Pydantic model defining the IntegrationNodeInput shape.
- ▶ `class CloudFileMetaOut`
The file shape downstream nodes address — flattened so a mapping
- ▶ `class IntegrationNodeOutput`
Pydantic model defining the IntegrationNodeOutput shape.
- ▶ `class IntegrationAgent`
Workflow node type implementing the IntegrationAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]`
Compute the preflight output fields.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _meta_out(meta: CloudFileMeta) -> CloudFileMetaOut`
Internal helper for the meta out step.
- ▶ `def _to_workflow_file_ref(downloaded: DownloadedFile) -> WorkflowFileRef`
Turn a downloaded cloud file into the platform's own file contract.
- ▶ `def _error_output(cfg: IntegrationNodeConfig, *, status: str, error_code: str, error: str, retryable: bool=False) -> dict[str, Any]`
Internal helper for the error output step.

app/nodes/internal_project_evidence_retriever.py — 580 lines

Retrieve exact, approval-gated facts from internal project sources.

- ▶ `class _InternalFactDraft`
Pydantic model defining the InternalFactDraft shape.
- ▶ `class _InternalExtraction`
Pydantic model defining the InternalExtraction shape.
- ▶ `class InternalProjectEvidenceRetrieverInput`
Pydantic model defining the InternalProjectEvidenceRetrieverInput shape.
- ▶ `class InternalProjectEvidenceRetrieverConfig`

```

Pydantic model defining the InternalProjectEvidenceRetrieverConfig shape.
▶ class InternalProjectEvidenceRetrieverOutput
    Pydantic model defining the InternalProjectEvidenceRetrieverOutput shape.
▶ class InternalProjectEvidenceRetrieverAgent
    Workflow node type implementing the InternalProjectEvidenceRetrieverAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
▶ def _research_questions(*, graph: Any, research_briefs: Any, limit: int) -> list[str]
    Internal helper for the research questions step.
▶ def _normalise_registry(value: Any) -> dict[str, dict[str, Any]]
    Internal helper for the normalise registry step.
▶ def _source_blocks(text: str) -> dict[str, str]
    Internal helper for the source blocks step.
▶ def _source_class(metadata: dict[str, Any]) -> str
    Internal helper for the source class step.
▶ def _explicitly_approved(metadata: dict[str, Any]) -> bool
    Internal helper for the explicitly approved step.
▶ def _resolve_source_name(requested: str, blocks: dict[str, str]) -> str | None
    Resolve the source name.

```

app/nodes/kimi_vision_agent.py — 156 lines

Understand one uploaded image with Kimi K3 vision.

```

▶ class KimiVisionInput
    Pydantic model defining the KimiVisionInput shape.
▶ class KimiVisionConfig
    Pydantic model defining the KimiVisionConfig shape.
▶ class KimiVisionOutput
    Pydantic model defining the KimiVisionOutput shape.
▶ class KimiVisionAgent
    Workflow node type implementing the KimiVisionAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.

```

app/nodes/knowledge_retrieval.py — 144 lines

One generic retrieval-only workflow capability.

```

▶ class KnowledgeRetrievalConfig
    Pydantic model defining the KnowledgeRetrievalConfig shape.
▶ class KnowledgeRetrievalInput
    Pydantic model defining the KnowledgeRetrievalInput shape.
▶ class KnowledgeRetrievalOutput
    Pydantic model defining the KnowledgeRetrievalOutput shape.
▶ class KnowledgeRetrieval
    Workflow node type implementing the KnowledgeRetrieval capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.

```

app/nodes/mcp_agent.py — 223 lines

MCPAgent: an LLM-driven agent loop calling MCP tools.

- ▶ `class MCPAgentConfig`
Pydantic model defining the MCPAgentConfig shape.
- ▶ `class MCPAgentInput`
Pydantic model defining the MCPAgentInput shape.
- ▶ `class ToolCallRecord`
Pydantic model defining the ToolCallRecord shape.
- ▶ `class MCPAgentOutput`
Pydantic model defining the MCPAgentOutput shape.
- ▶ `class MCPAgent`
Workflow node type implementing the MCPAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/mcp_tool.py — 510 lines

MCPToolAgent — call one capability on one MCP server.

- ▶ `class MCPToolConfig`
Pydantic model defining the MCPToolConfig shape.
- ▶ `class MCPToolInput`
Pydantic model defining the MCPToolInput shape.
- ▶ `class MCPToolOutput`
One output shape for every tool on every server.
- ▶ `class MCPToolAgent`
Workflow node type implementing the MCPToolAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]`
Authorise the fixed envelope.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
`async def _required_arguments(self, service: Any, cfg: MCPToolConfig) -> set[str]`
The argument names the server itself declares as required.
`def _skipped(self, cfg: MCPToolConfig, missing: list[str]) -> dict[str, Any]`
Internal helper for the skipped step.
`def _failure(self, cfg: MCPToolConfig, error: Exception) -> dict[str, Any]`
Internal helper for the failure step.
- ▶ `def _empty_arguments(arguments: dict[str, Any]) -> list[str]`
Argument names that were supplied but resolved to nothing.
- ▶ `def _summarise(data: dict[str, Any]) -> tuple[int, bool, dict[str, Any]]`
Lift 'count', 'found' and 'first' out of whatever shape the tool returned.

app/nodes/methodology_engineering.py — 320 lines

Produce one structured Method Card per frozen-concept objective.

- ▶ `class MethodologyEngineeringInput`
Pydantic model defining the MethodologyEngineeringInput shape.
- ▶ `class MethodologyEngineeringConfig`
Pydantic model defining the MethodologyEngineeringConfig shape.
- ▶ `class _MethodCardDraft`
Pydantic model defining the MethodCardDraft shape.

- ▶ `class MethodCard`
Pydantic model defining the MethodCard shape.
- ▶ `class MethodologyEngineeringOutput`
Pydantic model defining the MethodologyEngineeringOutput shape.
- ▶ `class MethodologyEngineeringAgent`
Workflow node type implementing the MethodologyEngineeringAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
`async def _draft_one_card(self, *, llm: Any, catalog: Any, objective_id: str, objective_text: str, model: str, max_skills: int, known_brief_ids: set[str], known_claim_ids: set[str], selected_concept: Any=None) -> MethodCard`
Draft the one card.

`app/nodes/minio_evidence_ingestion.py` — 367 lines

MinIOEvidenceIngestion: index already-acquired full-text sources into

- ▶ `class MinIOEvidenceIngestionInput`
Pydantic model defining the MinIOEvidenceIngestionInput shape.
- ▶ `class MinIOEvidenceIngestionConfig`
Pydantic model defining the MinIOEvidenceIngestionConfig shape.
`def _reject_unresolved(cls, value: Any) -> Any`
Internal helper for the reject unresolved step.
- ▶ `class MinIOEvidenceIngestionOutput`
Pydantic model defining the MinIOEvidenceIngestionOutput shape.
- ▶ `class MinIOEvidenceIngestion`
Workflow node type implementing the MinIOEvidenceIngestion capability.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _load_pages(raw: bytes) -> list[dict[str, Any]]`
Load the pages.
- ▶ `def _source_path(doc: dict[str, Any], number: int) -> str`
Internal helper for the source path step.
- ▶ `def _chunk_pages(pages: list[dict[str, Any]], *, chunk_chars: int, overlap: int, max_chunks: int) -> list[tuple[int, str]]`
Chunk the pages.
- ▶ `def _rfc3339_now() -> str`
Internal helper for the rfc3339 now step.

`app/nodes/openai_image_generation_agent.py` — 162 lines

Generate one OpenAI image and persist it in object storage.

- ▶ `class ImageGenerationInput`
Pydantic model defining the ImageGenerationInput shape.
- ▶ `class ImageGenerationConfig`
Pydantic model defining the ImageGenerationConfig shape.
- ▶ `class ImageGenerationOutput`
Pydantic model defining the ImageGenerationOutput shape.
- ▶ `class OpenAIImageGenerationAgent`
Workflow node type implementing the OpenAIImageGenerationAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/paperqa_evidence_synthesizer.py — 297 lines

Per-document, gap-aware literature synthesis over already-acquired sources.

- ▶ `class PaperQAEvidenceSynthesizerInput`
Pydantic model defining the PaperQAEvidenceSynthesizerInput shape.
- ▶ `class PaperQAEvidenceSynthesizerConfig`
Pydantic model defining the PaperQAEvidenceSynthesizerConfig shape.
 `def _coerce_documents(cls, value: Any) -> Any`
 Internal helper for the coerce documents step.
- ▶ `class DocumentCoverage`
Pydantic model defining the DocumentCoverage shape.
- ▶ `class ClaimSynthesis`
Pydantic model defining the ClaimSynthesis shape.
- ▶ `class PaperQAEvidenceSynthesizerOutput`
Pydantic model defining the PaperQAEvidenceSynthesizerOutput shape.
- ▶ `class PaperQAEvidenceSynthesizerAgent`
Workflow node type implementing the PaperQAEvidenceSynthesizerAgent capability.
 `def required_services(cls, config: dict[str, Any]) -> set[str]`
 Compute the required services.
 `async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.
 `async def _synthesize_claim(self, *, claim_id: str, question: str, documents: list[FullTextDocument], store: Any, settings: Any, paperqa: Any) -> ClaimSynthesis`
 Internal helper for the synthesize claim step.

app/nodes/pdf_tool.py — 191 lines

PDF tool node — two pre-baked variants:

- ▶ `class PDFExtractInput`
Pydantic model defining the PDFExtractInput shape.
- ▶ `class PDFExtractConfig`
Pydantic model defining the PDFExtractConfig shape.
- ▶ `class PDFExtractOutput`
Pydantic model defining the PDFExtractOutput shape.
- ▶ `class PDFTTextExtractor`
Workflow node type implementing the PDFTTextExtractor capability.
 `def required_services(cls, config: dict[str, Any]) -> set[str]`
 Compute the required services.
 `async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.
- ▶ `class PDFRenderInput`
Pydantic model defining the PDFRenderInput shape.
- ▶ `class PDFRenderConfig`
Pydantic model defining the PDFRenderConfig shape.
- ▶ `class PDFRenderOutput`
Pydantic model defining the PDFRenderOutput shape.
- ▶ `class PDFProposalRenderer`
Workflow node type implementing the PDFProposalRenderer capability.
 `def required_services(cls, config: dict[str, Any]) -> set[str]`
 Compute the required services.
 `async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.

app/nodes/powerpoint_tool.py — 107 lines

PowerPoint tool node — pre-baked variant: generate slide deck from sections.

- ▶ `class PPTInput`
Pydantic model defining the PPTInput shape.
- ▶ `class PPTConfig`
Pydantic model defining the PPTConfig shape.
- ▶ `class PPTOutput`
Pydantic model defining the PPTOutput shape.
- ▶ `class PowerPointProposalSlides`
Workflow node type implementing the PowerPointProposalSlides capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/prior_project_retriever.py — 360 lines

Official prior-project discovery for Horizon proposal positioning.

- ▶ `class PriorProjectRetrieverInput`
Pydantic model defining the PriorProjectRetrieverInput shape.
- ▶ `class PriorProjectRetrieverConfig`
Pydantic model defining the PriorProjectRetrieverConfig shape.
`def _coerce_briefs(cls, value: Any) -> Any`
Internal helper for the coerce briefs step.
- ▶ `class PriorProjectRetrieverOutput`
Pydantic model defining the PriorProjectRetrieverOutput shape.
- ▶ `class PriorProjectRetrieverAgent`
Workflow node type implementing the PriorProjectRetrieverAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _allowed_official_url(url: str, hosts: tuple[str, ...]) -> bool`
Internal helper for the allowed official url step.

app/nodes/proposal_evidence_factory.py — 1377 lines

Full-text, claim-level evidence verification for EU proposals.

- ▶ `class PairVerdict`
Pydantic model defining the PairVerdict shape.
- ▶ `class ProposalEvidenceFactoryInput`
Pydantic model defining the ProposalEvidenceFactoryInput shape.
- ▶ `class ProposalEvidenceFactoryConfig`
Pydantic model defining the ProposalEvidenceFactoryConfig shape.
`def _coerce_candidates(cls, value: Any) -> Any`
Internal helper for the coerce candidates step.
`def _coerce_documents(cls, value: Any) -> Any`
Internal helper for the coerce documents step.
`def _coerce_search_audit(cls, value: Any) -> Any`
Internal helper for the coerce search audit step.
`def _coerce_rejected_candidates(cls, value: Any) -> Any`
Internal helper for the coerce rejected candidates step.
- ▶ `class ProposalEvidenceFactoryOutput`
Provides the ProposalEvidenceFactoryOutput behaviour.
- ▶ `class ProposalEvidenceFactoryAgent`
Workflow node type implementing the ProposalEvidenceFactoryAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`

```

    Compute the required services.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
    async def _verify_document(self, *, llm: Any, store: Any, claim_text: str, document:
FullTextDocument, candidate: CandidateSource | None, model: str, passage_limit: int) ->
ClaimEvidenceLink | None
        Verify the document.
    def _document(document_id: str, documents: list[FullTextDocument]) -> FullTextDocument
        Internal helper for the document step.
    def _source_id_for_document(document: FullTextDocument) -> str
        Internal helper for the source id for document step.
    def _source_from_document(self, document: FullTextDocument, candidate: CandidateSource | None,
*, passage: str) -> EvidenceSource
        Internal helper for the source from document step.
    def _relation_from_link(link: ClaimEvidenceLink, document: FullTextDocument) ->
EvidenceRelation
        Internal helper for the relation from link step.
    def _gap_for_claim(claim_id: str, final_status: str, materiality: str, *, independent_count:
int, required_independent: int, contradiction_searched: bool, metadata_ok: bool, retraction_ok:
bool, policy: EvidencePolicy) -> EvidenceGap
        Internal helper for the gap for claim step.
    def _citation_registry(self, retained: dict[str, FullTextDocument], candidates: dict[str,
CandidateSource], claims: list[VerifiedClaim]) -> list[CitationRegistryEntry]
        Internal helper for the citation registry step.
    def _cited_markdown(claims: list[VerifiedClaim]) -> str
        Internal helper for the cited markdown step.
    def _bibtex(entry: CitationRegistryEntry) -> str
        Internal helper for the bibtex step.
    def _quantitative_registry(claims: list[VerifiedClaim], links: list[ClaimEvidenceLink]) ->
list[dict[str, Any]]
        Internal helper for the quantitative registry step.
    def _quality_report(*, policy: EvidencePolicy, verified_claims: list[VerifiedClaim],
citation_registry: list[CitationRegistryEntry], links: list[ClaimEvidenceLink], audit:
list[SearchAuditRecord], rejected: list[RejectedCandidate], quantitative_registry: list[dict[str,
Any]]) -> tuple[EvidenceQARReport, list[str]]
        Internal helper for the quality report step.
    ▶ def _materiality(claim_type: str) -> str
        Internal helper for the materiality step.
    ▶ def _authority(value: str) -> Authority
        Internal helper for the authority step.
    ▶ def _strength(score: int) -> str
        Internal helper for the strength step.
    ▶ def _evidence_score(*, document: FullTextDocument, stance: str, confidence: float, exact_locator:
bool, corroborated: bool) -> int
        Internal helper for the evidence score step.
    ▶ def _bibtex_key(entry: CitationRegistryEntry) -> str
        Internal helper for the bibtex key step.

```

app/nodes/proposal_submission_gate.py — 313 lines

Deterministic final release gate for proposal-engineering workflows.

- ▶ class ProposalSubmissionGateInput
 Pydantic model defining the ProposalSubmissionGateInput shape.
- ▶ class ProposalSubmissionGateConfig
 Pydantic model defining the ProposalSubmissionGateConfig shape.
- ▶ class ProposalSubmissionGateOutput

- Pydantic model defining the ProposalSubmissionGateOutput shape.
- ▶ `class ProposalSubmissionGate`
Combine hard proposal checks into one auditable release decision.
 `async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.
- ▶ `def _messages(value: str | list[Any]) -> list[str]`
Internal helper for the messages step.

app/nodes/proposal_truth_graph.py — 405 lines

Create the drafting-safe, integrity-stamped proposal truth graph.

- ▶ `class ProposalTruthGraphInput`
Pydantic model defining the ProposalTruthGraphInput shape.
- ▶ `class ProposalTruthGraphConfig`
Pydantic model defining the ProposalTruthGraphConfig shape.
 `def _coerce_verified_claims(cls, value: Any) -> Any`
 Internal helper for the coerce verified claims step.
 `def _coerce_evidence_gaps(cls, value: Any) -> Any`
 Internal helper for the coerce evidence gaps step.
 `def _coerce_structured_data_records(cls, value: Any) -> Any`
 Internal helper for the coerce structured data records step.
 `def _coerce_internal_evidence_records(cls, value: Any) -> Any`
 Internal helper for the coerce internal evidence records step.
- ▶ `class ProposalTruthGraphOutput`
Pydantic model defining the ProposalTruthGraphOutput shape.
- ▶ `class ProposalTruthGraphAgent`
Workflow node type implementing the ProposalTruthGraphAgent capability.
 `async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.
- ▶ `def _review_queue(claims: list[VerifiedClaim], gaps: list[EvidenceGap], graph: Any) -> list[dict[str, Any]]`
Internal helper for the review queue step.
- ▶ `def _string_list(value: Any) -> list[str]`
Internal helper for the string list step.

app/nodes/python_snippet.py — 312 lines

PythonSnippetAgent — write-and-run a Python snippet as a workflow step.

- ▶ `class PythonSnippetConfig`
Pydantic model defining the PythonSnippetConfig shape.
- ▶ `class PythonSnippetInput`
Pydantic model defining the PythonSnippetInput shape.
- ▶ `class PythonSnippetOutput`
Pydantic model defining the PythonSnippetOutput shape.
- ▶ `class PythonSnippetAgent`
Workflow node type implementing the PythonSnippetAgent capability.
 `def required_services(cls, config: dict[str, Any]) -> set[str]`
 Compute the required services.
 `def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]`
 Compute the preflight static output values.
 `async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
 Run the result.
 `def _failure(self, cfg: PythonSnippetConfig, code: str, message: str, *, stdout: str='', stderr: str='', duration_s: float=0.0) -> dict[str, Any]`
 Internal helper for the failure step.

- ▶ `def scan_snippet_for_warnings(code: str) -> list[str]`
AST-level authoring hints — never a security control. Returns plain
- ▶ `def assigns_output(code: str) -> bool`
Best-effort: does this snippet's top level ever touch `output` at

app/nodes/rag.py — 325 lines

RAGAgent: hybrid retrieval + grounded generation with citations.

- ▶ `class RAGConfig`
Pydantic model defining the RAGConfig shape.
`def reject_ambiguous_saved_agent_config(self) -> 'RAGConfig'`
Compute the reject ambiguous saved agent config.
- ▶ `class RAGInput`
Pydantic model defining the RAGInput shape.
- ▶ `class Citation`
Pydantic model defining the Citation shape.
- ▶ `class RAGOutput`
Pydantic model defining the RAGOutput shape.
- ▶ `class RAGAgent`
Workflow node type implementing the RAGAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/registry.py — 196 lines

Global registry of node types.

- ▶ `class _EmptySchema`
Placeholder for nodes without a declared schema.
- ▶ `class NodeRegistryError`
Exception raised for the NodeRegistryError case.
- ▶ `class NodeRegistry`
Provides the NodeRegistry behaviour.
`def register(cls, node_class: Type[NodeType]) -> Type[NodeType]`
Register a node class.
`def get(cls, type_name: str) -> Type[NodeType]`
Return the result.
`def manifest(cls) -> list[dict]`
Return the JSON schemas and presentation metadata used by the Builder.
- ▶ `def _manifest_entry(klass: Type[NodeType]) -> dict`
Internal helper for the manifest entry step.
- ▶ `def _uses_llm(klass: Type[NodeType]) -> bool`
Whether this node type needs a model, per its own preflight declaration.
- ▶ `def _with_model_catalog(schema: dict) -> dict`
Expose the model catalog as a Builder dropdown.

app/nodes/research_source_acquirer.py — 551 lines

Acquire Deep Research citations directly, without a paper-search MCP.

- ▶ `class ResearchSourceAcquirerInput`
Pydantic model defining the ResearchSourceAcquirerInput shape.
- ▶ `class ResearchSourceAcquirerConfig`
Pydantic model defining the ResearchSourceAcquirerConfig shape.
`def _coerce_candidates(cls, value: Any) -> Any`

Internal helper for the coerce candidates step.

- ▶ `class ResearchSourceAcquirerOutput`
Pydantic model defining the ResearchSourceAcquirerOutput shape.
- ▶ `class ResearchSourceAcquirer`
Workflow node type implementing the ResearchSourceAcquirer capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `async def _acquire_candidate(candidate: CandidateSource, *, client: httpx.AsyncClient, store: Any, run_id: str, policy: EvidencePolicy, max_redirects: int) -> FullTextDocument`
Acquire the candidate.
- ▶ `def _bounded_candidates(candidates: list[CandidateSource], *, per_claim: int, total: int) -> list[CandidateSource]`
Internal helper for the bounded candidates step.
- ▶ `async def _fetch_public_source(client: httpx.AsyncClient, url: str, *, max_bytes: int, max_redirects: int) -> tuple[str, str, bytes]`
Fetch the public source.
- ▶ `async def _require_public_url(url: str) -> None`
Internal helper for the require public url step.
- ▶ `def _html_pages(raw: bytes, *, chars_per_page: int=9000) -> list[dict[str, Any]]`
Internal helper for the html pages step.
- ▶ `async def _metadata_status(candidate: CandidateSource, *, final_url: str, client: httpx.AsyncClient) -> tuple[bool, str]`
Internal helper for the metadata status step.

`app/nodes/router.py` — 778 lines

RouterAgent: the workflow's branching primitive.

- ▶ `class RouteRule`
One legacy rule. The first matching rule wins. `default` matches anything.
- ▶ `class RouteCase`
One deterministic branch in `conditions` mode.
`def case_is_decidable(self) -> 'RouteCase'`
Compute the case is decidable.
- ▶ `class RouterConfig`
Pydantic model defining the RouterConfig shape.
`def mode_has_what_it_needs(self) -> 'RouterConfig'`
Compute the mode has what it needs.
`def multi_selection_not_supported_in_rule_mode(self) -> 'RouterConfig'`
Compute the multi selection not supported in rule mode.
`def route_names(self) -> list[str]`
Every route this router can emit — read by preflight to compare
- ▶ `class RouterOutput`
Pydantic model defining the RouterOutput shape.
- ▶ `class RouterInput`
Pydantic model defining the RouterInput shape.
- ▶ `class RouterAgent`
Workflow node type implementing the RouterAgent capability.
`def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
`async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
`def _route_by_field(self, cfg: RouterConfig, state: dict) -> dict[str, Any]`
Internal helper for the route by field step.
`def _route_by_field_multi(self, cfg: RouterConfig, state: dict) -> dict[str, Any]`

```

Multi-Route, field mode: `route_field` resolves to a *list* of
def _route_by_conditions(self, cfg: RouterConfig, state: dict) -> dict[str, Any]
    Internal helper for the route by conditions step.
def _route_by_conditions_multi(self, cfg: RouterConfig, state: dict) -> dict[str, Any]
    Multi-Route, conditions mode: every case is evaluated (never stop)
def _route_by_rule(self, cfg, state)
    Internal helper for the route by rule step.
def _eval_condition(self, condition: str | None, state: dict) -> bool
    Safe expression eval — supports only `path OP literal`.
def _parse_literal(s: str) -> Any
    Parse the literal.
async def _route_by_llm(self, cfg)
    Internal helper for the route by llm step.
async def _route_by_llm_multi(self, cfg: RouterConfig) -> dict[str, Any]
    Multi-Route, llm mode: ask the model to choose all routes that
▶ def _branch_key(value: Any) -> str
    Normalise a routed value into a branch key.
▶ def _matched_summaries(trace: GroupTrace) -> list[str]
    Internal helper for the matched summaries step.

```

app/nodes/scientific_research_planner.py — 361 lines

Proposal-aware research planning guided by curated scientific skills.

```

▶ class _ResearchBriefDraft
    Pydantic model defining the ResearchBriefDraft shape.
▶ class _ResearchPlanDraft
    Pydantic model defining the ResearchPlanDraft shape.
▶ class ScientificResearchPlannerInput
    Pydantic model defining the ScientificResearchPlannerInput shape.
▶ class ScientificResearchPlannerConfig
    Pydantic model defining the ScientificResearchPlannerConfig shape.
▶ class ScientificResearchPlannerOutput
    Pydantic model defining the ScientificResearchPlannerOutput shape.
▶ class ScientificResearchPlannerAgent
    Workflow node type implementing the ScientificResearchPlannerAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
▶ def _json_text(value: Any) -> str
    Internal helper for the json text step.

```

app/nodes/scientific_skill_agent.py — 143 lines

LLM research node guided by approved K-Dense Scientific Agent Skills.

```

▶ class ScientificSkillConfig
    Pydantic model defining the ScientificSkillConfig shape.
▶ class ScientificSkillInput
    Pydantic model defining the ScientificSkillInput shape.
▶ class ScientificSkillOutput
    Pydantic model defining the ScientificSkillOutput shape.
▶ class ScientificSkillAgent
    Workflow node type implementing the ScientificSkillAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.

```



```
async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
    Run the result.
```

app/nodes/sql_query.py — 259 lines

SQLQueryAgent — a read-only SQL lookup, for a query the classified MCP

- ▶ `class SQLQueryConfig`
Pydantic model defining the SQLQueryConfig shape.
- ▶ `class SQLQueryInput`
Pydantic model defining the SQLQueryInput shape.
- ▶ `class SQLQueryOutput`
Pydantic model defining the SQLQueryOutput shape.
- ▶ `class SQLQueryAgent`
Workflow node type implementing the SQLQueryAgent capability.
 - `def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
 - `def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]`
Compute the preflight output fields.
 - `async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
 - `def _failure(self, cfg: SQLQueryConfig, error: MCPToolError) -> dict[str, Any]`
Internal helper for the failure step.

app/nodes/start.py — 280 lines

StartAgent: the canonical workflow entry point.

- ▶ `class StartFileField`
A file-typed form field. Kept separate from InputFieldBinding/FieldSpec
 - `def resolved_source(self) -> str`
Compute the resolved source.
- ▶ `class StartConfig`
Pydantic model defining the StartConfig shape.
- ▶ `class StartInput`
Pydantic model defining the StartInput shape.
- ▶ `class StartOutput`
Pydantic model defining the StartOutput shape.
- ▶ `class StartAgent`
Workflow node type implementing the StartAgent capability.
 - `def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
 - `def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]`
Compute the preflight output fields.
 - `def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]`
Compute the preflight static output values.
 - `async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.
- ▶ `def _as_field_specs(fields: list[InputFieldBinding]) -> list[FieldSpec]`
Internal helper for the as field specs step.

app/nodes/structured_dataset_retriever.py — 613 lines

Reproducible retrieval of official structured proposal evidence.

- ▶ `class _StructuredDatasetPlan`
Pydantic model defining the StructuredDatasetPlan shape.
- ▶ `class StructuredDatasetRetrieverInput`

```

Pydantic model defining the StructuredDatasetRetrieverInput shape.
▶ class StructuredDatasetRetrieverConfig
    Pydantic model defining the StructuredDatasetRetrieverConfig shape.
    def _coerce_queries(cls, value: Any) -> Any
        Internal helper for the coerce queries step.
▶ class StructuredDatasetRetrieverOutput
    Pydantic model defining the StructuredDatasetRetrieverOutput shape.
▶ class StructuredDatasetRetrieverAgent
    Workflow node type implementing the StructuredDatasetRetrieverAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
    async def _plan_queries(self, cfg: StructuredDatasetRetrieverConfig, graph: Any) ->
_StructuredDatasetPlan
        Internal helper for the plan queries step.
▶ def _json_text(value: Any) -> str
    Internal helper for the json text step.
▶ def _deduplicate_queries(queries: list[StructuredDatasetQuery]) -> list[StructuredDatasetQuery]
    Internal helper for the deduplicate queries step.
▶ def _decode_jsonstat(*, query: StructuredDatasetQuery, payload: dict[str, Any], endpoint: str,
parameters: dict[str, Any], accessed_at: str, response_sha256: str, snapshot_object_key: str,
limit: int, source_version: str) -> tuple[list[StructuredDataEvidenceRecord], CountReconciliation]
    Decode the jsonstat.
▶ def _coordinates(flat_index: int, sizes: list[int]) -> list[int]
    Internal helper for the coordinates step.
▶ def _dimension_code(dimension: dict[str, Any], position: int) -> str
    Internal helper for the dimension code step.
▶ def _indexed_value(values: Any, index: int) -> Any
    Internal helper for the indexed value step.

```

app/nodes/subprocess.py — 414 lines

SubprocessAgent — call another saved workflow as a reusable business

```

▶ class SubprocessConfig
    Pydantic model defining the SubprocessConfig shape.
▶ class SubprocessInput
    Pydantic model defining the SubprocessInput shape.
▶ class SubprocessOutput
    Pydantic model defining the SubprocessOutput shape.
▶ class SubprocessAgent
    Workflow node type implementing the SubprocessAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
    async def _get_or_create_launch(self, cfg: SubprocessConfig, state: dict[str, Any]) ->
dict[str, Any]
        Return the or create launch.
    def _finalize(self, cfg: SubprocessConfig, decision: dict[str, Any]) -> dict[str, Any]
        Finalize the result.
▶ def _load_child_workflow(name: str)
    Load the child workflow.
▶ def _resolve_child_inputs(cfg: SubprocessConfig, child_spec: Any, state: dict[str, Any]) ->
dict[str, Any]
    Explicit mapping -> same-named parent workflow input -> same-named

```

app/nodes/text_assembler.py — 122 lines

TextAssemblerAgent: this platform's business-facing Join node.

- ▶ class TextAssemblerConfig
Pydantic model defining the TextAssemblerConfig shape.
- ▶ class TextAssemblerInput
Pydantic model defining the TextAssemblerInput shape.
- ▶ class TextAssemblerOutput
Pydantic model defining the TextAssemblerOutput shape.
- ▶ class TextAssemblerAgent
Workflow node type implementing the TextAssemblerAgent capability.
async def run(self, state: Any, resolved_config: dict[str, Any]) -> dict[str, Any]
Run the result.

app/nodes/transform.py — 692 lines

TransformAgent: pure LLM transform with structured output.

- ▶ class TransformInputField
One named input the Prompt Template editor's Inputs section declares.
- ▶ class TransformConfig
Pydantic model defining the TransformConfig shape.
def has_a_prompt_source(self) -> 'TransformConfig'
Return whether a prompt source.
def deterministic_mode_has_operations(self) -> 'TransformConfig'
Compute the deterministic mode has operations.
- ▶ class TransformInput
Pydantic model defining the TransformInput shape.
- ▶ class TransformOutput
Pydantic model defining the TransformOutput shape.
- ▶ class TransformAgent
Workflow node type implementing the TransformAgent capability.
def required_services(cls, config: dict[str, Any]) -> set[str]
Compute the required services.
def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
Compute the preflight output fields.
def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
Compute the preflight static output values.
async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
Run the result.
def _run_deterministic(self, cfg: TransformConfig, state: Any) -> dict[str, Any]
Run the deterministic.
async def _run_legacy(self, cfg: TransformConfig, llm: Any) -> dict[str, Any]
Run the legacy.
async def _run_new_style(self, cfg: TransformConfig, llm: Any) -> dict[str, Any]
Run the new style.
def _new_style_system_prompt(self, cfg: TransformConfig) -> str
Internal helper for the new style system prompt step.
def _new_style_user_prompt(self, cfg: TransformConfig) -> str
Internal helper for the new style user prompt step.
def _failure_result(self, error: Exception) -> dict[str, Any]
The non-raising counterpart to the two 'RuntimeError's above —
async def _complete_structured_with_retries(self, llm: Any, *, model: str, system: str, user: str, response_model: Type[BaseModel], temperature: float, max_tokens: int, max_retries: int, fail_on_error: bool=True) -> dict[str, Any]
Complete the structured with retries.

- ▶ `def is_new_style(cfg: TransformConfig) -> bool`
Whether this node was authored through the Inputs/Instructions/Outputs
- ▶ `def _render_input_value(value: Any) -> str`
Render one resolved input field's value for embedding in the prompt.
- ▶ `def _decode_json_container(value: Any) -> Any`
Decode a JSON object or array returned inside a string.
- ▶ `def _build_response_model(name: str, schema: dict[str, str]) -> Type[BaseModel]`
Build the response model.

app/nodes/web_search_agent.py — 146 lines

Typed workflow node for live public-web candidate discovery.

- ▶ `class WebSearchAgentInput`
Pydantic model defining the WebSearchAgentInput shape.
- ▶ `class WebSearchAgentConfig`
Pydantic model defining the WebSearchAgentConfig shape.
- ▶ `class WebSearchHit`
Pydantic model defining the WebSearchHit shape.
- ▶ `class WebSearchAgentOutput`
Pydantic model defining the WebSearchAgentOutput shape.
- ▶ `class WebSearchAgent`
Workflow node type implementing the WebSearchAgent capability.
 - `def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
 - `async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/workflow_file_loader.py — 224 lines

Node that turns uploaded workflow-file references into usable content.

- ▶ `class WorkflowFileLoaderInput`
Pydantic model defining the WorkflowFileLoaderInput shape.
- ▶ `class WorkflowFileLoaderConfig`
Pydantic model defining the WorkflowFileLoaderConfig shape.
- ▶ `class LoadedWorkflowFile`
Pydantic model defining the LoadedWorkflowFile shape.
- ▶ `class WorkflowFileLoaderOutput`
Pydantic model defining the WorkflowFileLoaderOutput shape.
- ▶ `class WorkflowFileLoader`
Read PDF/Office/Markdown/code inputs while retaining images as refs.
 - `def required_services(cls, config: dict[str, Any]) -> set[str]`
Compute the required services.
 - `async def run(self, state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Run the result.

app/nodes/workflow_input.py — 550 lines

WorkflowInputAgent — the entry primitive.

- ▶ `class InputFieldBinding`
One declared field and where its value comes from.
 - `def _default_incomplete_legacy_shape(cls, data: Any) -> Any`
A binding saved before ``item_type`/`fields`/`enum_values`` existed
 - `def resolved_source(self) -> str`
Compute the resolved source.
- ▶ `class WorkflowInputConfig`

```

Pydantic model defining the WorkflowInputConfig shape.
def as_field_specs(self) -> list[FieldSpec]
    The declared shape, as the same FieldSpec rows every other node uses.
▶ class WorkflowInputInput
    Pydantic model defining the WorkflowInputInput shape.
▶ class WorkflowInputOutput
    Pydantic model defining the WorkflowInputOutput shape.
▶ class WorkflowInputAgent
    Workflow node type implementing the WorkflowInputAgent capability.
    def required_services(cls, config: dict[str, Any]) -> set[str]
        Compute the required services.
    def preflight_output_fields(cls, config: dict[str, Any]) -> set[str]
        Compute the preflight output fields.
    def preflight_static_output_values(cls, config: dict[str, Any]) -> dict[str, Any]
        Compute the preflight static output values.
    async def run(self, state, resolved_config: dict[str, Any]) -> dict[str, Any]
        Run the result.
▶ def _friendly_number(value: float) -> str
    "1.0" reads as a technical artifact of a float field; a business-
▶ def resolve_field_bindings(bindings: list[InputFieldBinding], sample: dict[str, Any], state:
dict[str, Any]) -> tuple[dict[str, Any], list[str]]
    Resolve each declared field's source path against workflow state.
▶ def apply_conditional_fields(bindings: list[InputFieldBinding], data: dict[str, Any], missing:
list[str]) -> tuple[dict[str, Any], list[str]]
    Evaluate visible_when/required_when against this form's OWN
▶ def validate_field_constraints(bindings: list[InputFieldBinding], data: dict[str, Any]) -> None
    Business-friendly validation beyond shape (§20/21 — never a raw
▶ def reject_values_outside_declared_shape(specs: list[FieldSpec], data: dict[str, Any]) -> None
    Validate what actually arrived against the declared shape before any

```

observability (5 modules)

app/observability/__init__.py — 0 lines

(no summary)

app/observability/cost_ledger.py — 410 lines

Cost ledger module.

- ▶ class LedgerEntry
Provides the LedgerEntry behaviour.
- ▶ class CostLedger
Provides the CostLedger behaviour.
 - def calculate(model: str, input_tokens: int, output_tokens: int, *, cache_creation_input_tokens: int=0, cache_read_input_tokens: int=0) -> float
Compute the result.
 - def record(self, entry: LedgerEntry) -> None
Record the result.
 - def record_no_charge(self, *, run_id: str, session_id: str, node_id: str, stage: str, task_type: str='retrieval', note: str='') -> None
Log a pipeline stage that genuinely has no model charge.
 - def _group_by(entries: list[dict], key: str) -> list[dict]
Group the by.
 - def run_summary(self, run_id: str, session_id: str | None=None) -> dict
Run the summary.
 - def session_summary(self, session_id: str) -> dict
Compute the session summary.
 - def query(self, *, since: datetime | None=None, until: datetime | None=None, collection_id: str | None=None) -> list[dict]
Raw entries across the whole ledger (not scoped to one run/session), for
- ▶ def configure_pricing_db(db: Any) -> None
Called once from app/main.py's lifespan, right after Mongo is available.
- ▶ def _refresh_pricing_overrides_if_stale() -> None
Refresh the pricing overrides if stale.

app/observability/logging.py — 57 lines

Logging module.

- ▶ def configure_logging(environment: str='development') -> None
Configure the logging.
- ▶ def get_logger(name: str=__name__)
Return the logger.

app/observability/metrics.py — 238 lines

Prometheus metrics for the agentic workflow platform.

- ▶ def track_node(node_type: str) -> Iterator[None]
Time a node, record its outcome, and track in-flight count.
- ▶ def record_paused(node_type: str) -> None
HITL pause is not an error and not a success — its own outcome.
- ▶ def record_llm_usage(model: str, prompt_tokens: int, completion_tokens: int, ok: bool) -> None
Record token usage and call outcome. Called by the LLM gateway.

app/observability/tracing.py — 58 lines

Optional OpenTelemetry wiring.

- `def configure_tracing(app) -> None`
Configure the tracing.

proposal_graph (9 modules)

app/proposal_graph/__init__.py — 17 lines

Proposal workspace graph: domain state and workspace store.

app/proposal_graph/concepts.py — 450 lines

Generate three independently-produced concept alternatives, then score and

- ▶ `class ConceptAlternativeSet`
Pydantic model defining the ConceptAlternativeSet shape.
`def one_of_each_posture(self) -> 'ConceptAlternativeSet'`
Compute the one of each posture.
- ▶ `class _ConceptDraft`
Pydantic model defining the ConceptDraft shape.
- ▶ `class _ConceptJudgment`
Pydantic model defining the ConceptJudgment shape.
- ▶ `def _valid_ids(values: list[str], known: set[str]) -> list[str]`
Internal helper for the valid ids step.
- ▶ `def score_concept(concept: ConceptAlternative, graph: ProposalGraph) -> float`
Evidence-weighted score based only on IDs that exist in the graph.
- ▶ `def _composite_score(concept: ConceptAlternative, graph: ProposalGraph) -> float`
Internal helper for the composite score step.
- ▶ `async def _draft_one_concept(llm: Any, *, posture: ConceptPosture, graph_payload: dict[str, Any], concept_note: str, model: str) -> _ConceptDraft`
Draft the one concept.
- ▶ `async def judge_concept(llm: Any, *, concept: ConceptAlternative, model: str, skill_guidance: str='') -> _ConceptJudgment`
Independently score the 5 qualitative dimensions and attack the concept.
- ▶ `async def generate_concept_alternatives(llm: Any, *, graph: ProposalGraph, model: str='claude-opus-5', judge_model: str | None=None, concept_note: str='', skill_catalog: Any | None=None) -> ConceptAlternativeSet`
Create conservative, balanced, and ambitious options from graph facts.

app/proposal_graph/coverage.py — 212 lines

Deterministic Horizon call-coverage matrix.

- ▶ `class CallCoverageRow`
Pydantic model defining the CallCoverageRow shape.
- ▶ `class CallCoverageMatrix`
Pydantic model defining the CallCoverageMatrix shape.
- ▶ `def _known_object_ids(graph: ProposalGraph) -> set[str]`
Internal helper for the known object ids step.
- ▶ `def build_call_coverage_matrix(graph: ProposalGraph) -> CallCoverageMatrix`
Build the call coverage matrix.

app/proposal_graph/evidence_verification.py — 147 lines

Claim-evidence verification with an exact-passage safety check.

- ▶ `class ClaimSupportVerdict`
Pydantic model defining the ClaimSupportVerdict shape.
- ▶ `def _normalise_space(value: str) -> str`
Internal helper for the normalise space step.

- ▶ `def quote_exists_in_source(quote: str, source_text: str) -> bool`
Whitespace-tolerant, otherwise verbatim passage validation.
- ▶ `async def verify_claim_against_text(llm: Any, *, claim: str, source_text: str, model: str='claude-sonnet-4-5') -> ClaimSupportVerdict`
Return a typed verdict and fail closed when the quote is not traceable.
- ▶ `def relation_from_verdict(*, relation_id: str, claim_id: str, source_id: str, source_version_id: str | None, locator: str, verifier_model: str, verdict: ClaimSupportVerdict) -> EvidenceRelation`
Compute the relation from verdict.

app/proposal_graph/graph.py — 114 lines

ProposalGraph — the container that lives inside WorkflowState under one key.

- ▶ `class ProposalGraph`
All 15 object types, each as an id-keyed dict.
- ▶ `def _merge_object(left: BaseModel, right: BaseModel) -> BaseModel`
Field-by-field merge; right wins only where it has a non-null/non-empty
- ▶ `def merge_graph(left: Any, right: Any) -> dict`
Reducer for the `proposal\_graph` state key. Associative + lossless.
- ▶ `def empty_graph() -> ProposalGraph`
Compute the empty graph.

app/proposal_graph/horizon_evaluator.py — 314 lines

Independent two-provider Horizon Europe proposal evaluation panel.

- ▶ `class HorizonJudgeVerdict`
Pydantic model defining the HorizonJudgeVerdict shape.
- ▶ `class HorizonJudgeResult`
Provides the HorizonJudgeResult behaviour.
- ▶ `class HorizonCriterionPanel`
Pydantic model defining the HorizonCriterionPanel shape.
- ▶ `class HorizonEvaluationReport`
Pydantic model defining the HorizonEvaluationReport shape.
- ▶ `def _provider(model: str) -> str`
Internal helper for the provider step.
- ▶ `def validate_independent_models(evaluator_models: list[str], generator_model: str | None) -> None`
Validate the independent models.
- ▶ `def deterministic_horizon_blockers(graph: ProposalGraph) -> tuple[float, list[str]]`
Compute the deterministic horizon blockers.
- ▶ `async def evaluate_horizon_proposal(llm: Any, *, proposal_text: str, graph: ProposalGraph, generator_model: str | None, evaluator_models: list[str], criterion_threshold: float=3.0, total_threshold: float=10.0) -> HorizonEvaluationReport`
Compute the evaluate horizon proposal.

app/proposal_graph/models.py — 458 lines

Proposal Knowledge Graph — typed objects.

- ▶ `class Status`
Enumeration of Status values.
- ▶ `class Authority`
Provenance strength for evidence — used by the evidence layer later.
- ▶ `class EvidenceStance`
Enumeration of EvidenceStance values.
- ▶ `class ConceptPosture`
Enumeration of ConceptPosture values.
- ▶ `class CallRequirement`

- Pydantic model defining the CallRequirement shape.
- ▶ `class EvidenceSource`
Pydantic model defining the EvidenceSource shape.
- ▶ `class Claim`
Pydantic model defining the Claim shape.
- ▶ `class EvidenceRelation`
Auditable claim -> exact passage -> immutable source-version relation.
- ▶ `class Objective`
Pydantic model defining the Objective shape.
- ▶ `class Innovation`
Pydantic model defining the Innovation shape.
- ▶ `class Result`
Pydantic model defining the Result shape.
- ▶ `class Outcome`
Pydantic model defining the Outcome shape.
- ▶ `class Impact`
Pydantic model defining the Impact shape.
- ▶ `class Task`
Pydantic model defining the Task shape.
- ▶ `class WorkPackage`
Pydantic model defining the WorkPackage shape.
- ▶ `class Partner`
Pydantic model defining the Partner shape.
- ▶ `class KPI`
Pydantic model defining the KPI shape.
- ▶ `class Risk`
Pydantic model defining the Risk shape.
- ▶ `class ComplianceObject`
Pydantic model defining the ComplianceObject shape.
- ▶ `class OpenQuestion`
Pydantic model defining the OpenQuestion shape.
- ▶ `class ConceptAlternative`
Pydantic model defining the ConceptAlternative shape.

`app/proposal_graph/state.py` — 37 lines

Workflow-state adapter for the EU proposal use-case pack.

- ▶ `def proposal_graph_from_state(state: dict[str, Any]) -> ProposalGraph`
Compute the proposal graph from state.
- ▶ `def proposal_graph_state_update(graph: ProposalGraph) -> dict[str, Any]`
Compute the proposal graph state update.

`app/proposal_graph/workspace_store.py` — 522 lines

Immutable proposal snapshots, source versions, and approval decisions.

- ▶ `class SourceVersionRecord`
Pydantic model defining the SourceVersionRecord shape.
- ▶ `class ProposalSnapshot`
Pydantic model defining the ProposalSnapshot shape.
- ▶ `class ApprovalRecord`
Pydantic model defining the ApprovalRecord shape.
- ▶ `class ProposalWorkspaceStore`
Provides the ProposalWorkspaceStore behaviour.
`async def ensure_indexes(self) -> None`
Ensure the indexes.

```

    async def register_source_version(self, *, session_id: str, proposal_id: str, source_id: str,
content: str, title: str, created_by: str, identifier: str | None=None, authority:
str='unverified', metadata: dict[str, Any] | None=None) -> SourceVersionRecord
    Register the source version.
    async def list_source_versions(self, *, session_id: str, proposal_id: str, source_id: str |
None=None) -> list[dict[str, Any]]
    List the source versions.
    async def source_text(self, *, session_id: str, proposal_id: str, source_id: str, version_id:
str | None=None) -> tuple[SourceVersionRecord, str]
    Compute the source text.
    async def save_snapshot(self, *, session_id: str, proposal_id: str, graph: ProposalGraph,
created_by: str, reason: str) -> ProposalSnapshot
    Save the snapshot.
    async def request_approval(self, *, session_id: str, proposal_id: str, graph: ProposalGraph,
stage: str, requested_by: str, selected_concept_id: str | None=None) -> ApprovalRecord
    Compute the request approval.
    async def decide_approval(self, *, session_id: str, proposal_id: str, approval_id: str,
decision: Literal['approved', 'rejected', 'changes_requested'], decided_by: str, comment: str |
None) -> ApprovalRecord
    Compute the decide approval.
    async def list_approvals(self, *, session_id: str, proposal_id: str) -> list[dict[str, Any]]
    List the approvals.
    ▶ def _now() -> datetime
    Internal helper for the now step.
    ▶ def _canonical_hash(payload: Any) -> str
    Internal helper for the canonical hash step.

```

rag (3 modules)

app/rag/__init__.py — 1 lines

Saved RAG Agent resources and grounded generation.

app/rag/models.py — 68 lines

Saved RAG Agent query request/response contracts.

- ▶ class RAGCitation
Pydantic model defining the RAGCitation shape.
- ▶ class RAGQueryResponse
Pydantic model defining the RAGQueryResponse shape.

app/rag/service.py — 379 lines

Resolve a saved RAG Agent, retrieve securely, then generate an answer.

- ▶ class RAGService
Provides the RAGService behaviour.
 async def _select_route(self, config: RetrievalRoutingProfileConfig, query: str, llm: Any) -> tuple[RetrievalRoute, str]
 Select the route.
 async def query(self, *, owner_scope_id: str, rag_agent_id: str, query: str, runtime_filters: dict[str, Any] | MetadataFilterGroup | None=None, runtime_context: dict[str, Any] | None=None, llm: Any | None=None) -> RAGQueryResponse
 Query the result.
- ▶ def _build_sources(citations: list[RAGCitation], chunk_by_id: dict[str, Any]) -> list[dict[str, Any]]
 Group citations into one entry per source document (§37 dedup shape).
- ▶ def _build_relevant_context(chunks: list[Any]) -> list[dict[str, Any]]
 One entry per retrieved chunk actually used to ground the answer.

research (3 modules)

app/research/__init__.py — 5 lines

Controlled adapters for external scientific research assets.

app/research/deep_research.py — 661 lines

Bounded Deep Research contracts and provider adapter.

- ▶ class ResearchBrief
Pydantic model defining the ResearchBrief shape.
- ▶ class ResearchCitation
Pydantic model defining the ResearchCitation shape.
- ▶ class ResearchToolTrace
Pydantic model defining the ResearchToolTrace shape.
- ▶ class ResearchUsage
Pydantic model defining the ResearchUsage shape.
- ▶ class ResearchDossier
Pydantic model defining the ResearchDossier shape.
- ▶ class BoundedToolResearchService
Bounded, in-process replacement for the OpenAI Deep Research job API.
def available(self) -> bool
Compute the available.
async def research(self, *, brief: ResearchBrief, instructions: str, max_duration_seconds: float=1800.0, max_iterations: int=15, max_cost_per_call_usd: float=15.0) -> ResearchDossier
Compute the research.
async def _attribute_citations(self, brief: ResearchBrief, citations: list[ResearchCitation], max_cost_per_call_usd: float) -> tuple[int, int]
Map each gathered citation to ONE claim, in place.
- ▶ class _CitationAttribution
Pydantic model defining the CitationAttribution shape.
- ▶ class _CitationAttributionSet
Pydantic model defining the CitationAttributionSet shape.
- ▶ def _projected_call_cost(model: str, prompt_text: str, max_tokens: int) -> float
Worst-case pre-call cost: rough input-token estimate, full max_tokens output.
- ▶ def _brief_prompt(brief: ResearchBrief) -> str
Internal helper for the brief prompt step.
- ▶ def get_deep_research_service(*, llm: Any, web_search: Any) -> BoundedToolResearchService
Return the deep research service.

app/research/skills.py — 515 lines

Allowlisted adapter for K-Dense Scientific Agent Skills.

- ▶ class SkillDocument
Provides the SkillDocument behaviour.
def metadata(self) -> dict[str, str]
Compute the metadata.
- ▶ class SkillSelection
Provides the SkillSelection behaviour.
def names(self) -> list[str]
The names.
def versions(self) -> dict[str, str]
The versions.

```

▶ class ScientificSkillCatalog
    Read and select only configured, locally installed skill documents.
    def refresh(self) -> None
        Refresh the result.
    async def probe(self) -> bool
        Probe the result.
    def metadata(self) -> list[dict[str, str]]
        Compute the metadata.
    def load_errors(self) -> dict[str, str]
        The load errors.
    def loaded_skill_names(self) -> tuple[str, ...]
        Allowlisted skills whose SKILL.md actually parsed successfully.
    def select(self, *, objective: str, requested: Iterable[str]=(), auto_select: bool=True,
max_skills: int=3) -> SkillSelection
        Select the result.
    def prompt_bundle(self, selection: SkillSelection) -> str
        Compute the prompt bundle.
▶ def _parse_skill(text: str) -> SkillDocument
    Parse the skill.
▶ def _sanitize_instructions(text: str) -> str
    Keep research guidance while removing execution/install directions.
▶ def _skill_score(objective: str, skill: SkillDocument) -> int
    Internal helper for the skill score step.

```

retrieval (16 modules)

app/retrieval/__init__.py — 20 lines

Hybrid RAG retrieval pipeline.

app/retrieval/compressor.py — 109 lines

Stage 4 of the retrieval pipeline: contextual compression.

- ▶ `async def _compress_one(query: str, chunk: RetrievedChunk, llm: LLMGateway, model: str) -> RetrievedChunk`
Compress a single chunk. Mutates and returns the same chunk.
- ▶ `async def compress_chunks(query: str, chunks: list[RetrievedChunk], llm: LLMGateway, model: str='claude-haiku-4-5') -> tuple[list[RetrievedChunk], float]`
Compress all chunks in parallel.

app/retrieval/context.py — 81 lines

Parent/sentence-window/contextual expansion and final context assembly.

- ▶ `class GraphContextProvider`
Provides the GraphContextProvider behaviour.
`async def expand(self, chunks: list[RetrievedChunk], *, query: str) -> list[RetrievedChunk]`
- ▶ `async def expand_context(chunks: list[RetrievedChunk], *, strategy: str, client: Any, collection_name: str, filters: RetrievalFilters, index_id: str | None) -> list[RetrievedChunk]`
Apply the requested context-expansion strategy to already-kept chunks.
- ▶ `def assemble_context(chunks: list[RetrievedChunk]) -> tuple[str, int]`
Build the final "[N] (source: ...)" block passed to generation.

app/retrieval/contracts.py — 55 lines

Small interfaces for retrieval, fusion, reranking and context expansion.

- ▶ `class RetrievalStrategy`
Provides the RetrievalStrategy behaviour.
- ▶ `class FusionStrategy`
Provides the FusionStrategy behaviour.
- ▶ `class Reranker`
Provides the Reranker behaviour.
- ▶ `class ContextExpander`
Provides the ContextExpander behaviour.

app/retrieval/filters.py — 291 lines

Typed metadata filters and the non-removable authorization-filter clauses.

- ▶ `class MetadataFilterValidationError`
Exception raised for the MetadataFilterValidationError case.
- ▶ `def metadata_property_name(key: str) -> str`
Sanitize a user-supplied metadata key into a safe Weaviate property name.
- ▶ `def _field_type(field: str, schema: dict[str, Any]) -> str | None`
Internal helper for the field type step.
- ▶ `def _check_field(field: str, schema: dict[str, Any]) -> None`
Check the field.
- ▶ `def _check_value(field: str, value: Any, schema: dict[str, Any]) -> None`

Reject a value whose type contradicts the collection's declared schema.

- ▶ `def validate_metadata_document(metadata: dict[str, Any], schema: dict[str, Any]) -> None`
Validate an ingestion-time metadata document against a Collection schema.
- ▶ `def _validate_group(group: MetadataFilterGroup, schema: dict[str, Any]) -> None`
Validate the group.
- ▶ `def validate_metadata_filters(metadata: MetadataFilterGroup | None, schema: dict[str, Any]) -> None`
Validate a caller-supplied (or self-query-generated) filter tree.
- ▶ `def coerce_metadata_filter_group(value: dict[str, Any] | MetadataFilterGroup | None) -> MetadataFilterGroup | None`
Normalize workflow ``runtime\_filters``/API filter payloads to one shape.
- ▶ `def _predicate_to_filter(predicate: MetadataFilterPredicate) -> Filter`
Internal helper for the predicate to filter step.
- ▶ `def _group_to_filter(group: MetadataFilterGroup) -> Filter | None`
Group the to filter.
- ▶ `def build_secure_where_filter(filters: RetrievalFilters, *, index_id: str | None=None, exclude_parent_chunks: bool=False) -> Filter`
Compile the non-removable security clause, ANDed with user metadata.

app/retrieval/fusion.py — 63 lines

Reciprocal Rank Fusion for independently executed retrieval result sets.

- ▶ `def reciprocal_rank_fusion(result_sets: list[list[RetrievedChunk]], *, limit: int) -> list[RetrievedChunk]`
Fuse independently-ranked result sets (dense+sparse, or per-query)
- ▶ `def deduplicate(candidates: list[RetrievedChunk]) -> list[RetrievedChunk]`
Drop repeat ``chunk\_id``s, keeping the highest-scoring occurrence.

app/retrieval/hybrid_search.py — 102 lines

Stage 2 of the retrieval pipeline: hybrid search with metadata pre-filter.

- ▶ `def _build_where_filter(f: RetrievalFilters) -> Filter`
Translate filters into a Weaviate Filter expression.
- ▶ `async def hybrid_search(client: weaviate.WeaviateAsyncClient, embedder: Embedder, collection_name: str, query: str, filters: RetrievalFilters, top_k: int, alpha: float) -> tuple[list[RetrievedChunk], float]`
Hybrid BM25 + vector search with metadata pre-filter.

app/retrieval/models.py — 206 lines

Pydantic models for the retrieval pipeline.

- ▶ `class MetadataFilterPredicate`
Pydantic model defining the MetadataFilterPredicate shape.
- ▶ `class MetadataFilterGroup`
Pydantic model defining the MetadataFilterGroup shape.
- ▶ `class RetrievalFilters`
Metadata pre-filter applied BEFORE hybrid search runs in Weaviate.
`def user_filter_dump(self) -> dict[str, Any]`
Trace-safe view of caller-supplied filters (no security clauses).
- ▶ `class RetrievalQuery`
Single retrieval request. Defaults come from app.config.
- ▶ `class RetrievedChunk`
One chunk that survived the pipeline.
- ▶ `class RetrievalStage`
One named step in the staged retrieval pipeline, for the trace.

- ▶ `class RetrievalResult`
Result of one `retrieve()` call. This is what the RAG Agent node returns

`app/retrieval/presets.py` — 86 lines

Editable Retrieval Playground starting configurations.

`app/retrieval/query_transform.py` — 154 lines

Rewrite, multi-query, decomposition, HyDE and validated self-query.

- ▶ `async def _safe_complete(llm: Any, *, system: str, user: str) -> str | None`
Internal helper for the safe complete step.
- ▶ `def _split_lines(text: str, *, limit: int) -> list[str]`
Split the lines.
- ▶ `async def transform_query(request: RetrievalQuery, *, llm: Any, metadata_schema: dict[str, Any]) -> tuple[str, list[str], MetadataFilterGroup | None]`
Return "(semantic_query, transformed_queries, generated_filters)".

`app/retrieval/query_understanding.py` — 28 lines

Stage 1 of the retrieval pipeline: query understanding.

- ▶ `async def rewrite_query(query: str, llm: LLMGateway) -> str`
Return a search-optimized version of the query.

`app/retrieval/reranker.py` — 159 lines

Stage 3 of the retrieval pipeline: LLM-as-judge reranking.

- ▶ `class _RerankScore`
One score for one chunk. Returned as part of a JSON array.
- ▶ `class _RerankResponse`
Whole response: a list of scores, one per input chunk.
- ▶ `def _format_candidates(candidates: list[RetrievedChunk]) -> str`
Build the numbered candidate block for the user prompt.
- ▶ `async def rerank(query: str, candidates: list[RetrievedChunk], top_n: int, llm: LLMGateway, model: str='gpt-5') -> tuple[list[RetrievedChunk], float]`
Score all candidates in one LLM call, return top_n sorted by score.

`app/retrieval/retriever.py` — 144 lines

The retrieval pipeline orchestrator.

- ▶ `async def retrieve(q: RetrievalQuery, *, weaviate_client: weaviate.WeaviateAsyncClient, llm: LLMGateway, embedder: Embedder, collection_registry=None) -> RetrievalResult`
Run the four-stage retrieval pipeline.

`app/retrieval/service.py` — 666 lines

Canonical retrieval path for workflows, RAG, Playground, MCP and Eval.

- ▶ `class RetrievalAuthorizationError`
Exception raised for the `RetrievalAuthorizationError` case.
- ▶ `class RetrievalCompatibilityError`
Exception raised for the `RetrievalCompatibilityError` case.
- ▶ `class RetrievalService`
Provides the `RetrievalService` behaviour.

```

    async def retrieve(self, request: RetrievalQuery, *, owner_scope_id: str, llm: Any | None=None)
-> RetrievalResult
    Retrieve the result.
    async def _record_failed_trace(self, *, request: RetrievalQuery, request_id: str,
owner_scope_id: str, error: Exception, total_ms: float) -> None
    Record the failed trace.
    async def _retrieve_impl(self, request: RetrievalQuery, *, owner_scope_id: str, request_id:
str, llm: Any | None=None) -> RetrievalResult
    Retrieve the impl.
    async def compare(self, requests: list[RetrievalQuery], *, owner_scope_id: str, llm: Any |
None=None) -> list[RetrievalResult]
    Compare the result.
    ▶ def _safe_candidate(chunk: RetrievedChunk) -> dict[str, Any]
    Trace score/provenance, not duplicate raw source text per stage.

```

app/retrieval/strategies.py — 222 lines

Replaceable dense, sparse and hybrid Weaviate retrieval strategies.

```

    ▶ def _chunk_from_object(obj: Any, *, score_kind: str, rank: int) -> RetrievedChunk
    Chunk the from object.
    ▶ async def dense_search(*, client: Any, collection_name: str, embedder: Embedder, query: str,
filters: RetrievalFilters, index_id: str | None, top_k: int, exclude_parent_chunks: bool=False) ->
tuple[list[RetrievedChunk], float]
    Compute the dense search.
    ▶ async def sparse_search(*, client: Any, collection_name: str, query: str, filters:
RetrievalFilters, index_id: str | None, top_k: int, exclude_parent_chunks: bool=False) ->
tuple[list[RetrievedChunk], float]
    Compute the sparse search.
    ▶ async def native_hybrid_search(*, client: Any, collection_name: str, embedder: Embedder, query:
str, filters: RetrievalFilters, index_id: str | None, top_k: int, alpha: float,
exclude_parent_chunks: bool=False) -> tuple[list[RetrievedChunk], float]
    Compute the native hybrid search.
    ▶ async def fetch_chunks_by_id(*, client: Any, collection_name: str, chunk_ids: list[str], filters:
RetrievalFilters, index_id: str | None=None) -> list[RetrievedChunk]
    Fetch the chunks by id.

```

app/retrieval/weaviate_client.py — 326 lines

Weaviate client and schema definition.

```

    ▶ class WeaviateClient
    Thin wrapper around weaviate-client v4.
    def connect(self) -> weaviate.WeaviateClient
    Open the gRPC + REST connection. Call once at startup.
    def close(self) -> None
    Close the underlying gRPC channel. Call at app shutdown.
    def ensure_schema(self) -> None
    Create the DocumentChunk collection if it doesn't exist.
    def ensure_collection_schema(self, name: str) -> None
    Like :meth:`ensure_schema`, but for an arbitrary physical collection.
    def upsert_objects(self, collection_name: str, objects: list[dict[str, Any]], vectors:
list[list[float]]) -> int
    Like :meth:`upsert_chunks`, but targeting an arbitrary physical collection.
    def reset_schema(self) -> None
    DESTRUCTIVE: delete the collection and recreate it.
    def upsert_chunks(self, chunks: list[dict[str, Any]], vectors: list[list[float]]) -> int
    Insert or replace chunks. Idempotent on chunk_id.

```

```

def count_chunks(self) -> int
    Total chunks in the collection. Useful for tests and diagnostics.
def add_property(self, prop: Property) -> None
    Non-destructive schema evolution. Old objects get null for new field.
▶ def ensure_collection_schema_on(client: weaviate.WeaviateClient, name: str) -> None
    Create "name" with the standard chunk schema if it doesn't exist yet.
▶ def upsert_objects_on(client: weaviate.WeaviateClient, collection_name: str, objects:
list[dict[str, Any]], vectors: list[list[float]]) -> int
    Batch-insert "objects"/"vectors" into "collection_name".
▶ def get_weaviate_client() -> WeaviateClient
    Lazily-constructed module-level client. Use this from app code.

```

runtime (25 modules)

app/runtime/__init__.py — 1 lines

Phase 4 runtime: YAML to LangGraph compiler and executor.

app/runtime/autofix.py — 841 lines

Auto-fix preflight errors.

- ▶ `class DeterministicFixResult`
Provides the `DeterministicFixResult` behaviour.
- ▶ `class LlmRepairAttempt`
Provides the `LlmRepairAttempt` behaviour.
- ▶ `def _get_path(raw: Any, path: str) -> Any`
Return the path.
- ▶ `def _set_path(raw: Any, path: str, value: Any) -> None`
Set the path.
- ▶ `def _replace_template_reference(text: str, node_id: str, old_remainder: str, new_remainder: str) -> tuple[str, bool]`
Rewrite the field portion of a ``{{node_id.old_remainder}}`` (or
- ▶ `def _replace_template_node_id(text: str, old_node_id: str, new_node_id: str) -> tuple[str, bool]`
Rewrite the node-id portion of a template reference, keeping whatever
- ▶ `def _node_by_id(raw: dict, node_id: str) -> dict | None`
Internal helper for the node by id step.
- ▶ `def _fix_template_unknown_output_field(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix template unknown output field step.
- ▶ `def _fix_template_unknown_structured_field(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix template unknown structured field step.
- ▶ `def _fix_template_unknown_node(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix template unknown node step.
- ▶ `def _replace_template_named_root_segment(text: str, root: str, old_name: str, new_name: str) -> tuple[str, bool]`
Rewrite a ``{{root.old_name...}}`` reference (root is ``inputs`` or
- ▶ `def _fix_template_unknown_named_root(raw: dict, issue: PreflightIssue, *, root: str, noun: str) -> str | None`
Internal helper for the fix template unknown named root step.
- ▶ `def _fix_template_unknown_input(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix template unknown input step.
- ▶ `def _fix_template_unknown_variable(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix template unknown variable step.
- ▶ `def _fix_router_multiple_defaults(raw: dict, issue: PreflightIssue) -> str | None`
A legacy rule-mode Router with more than one ``default: true`` rule —
- ▶ `def _fix_router_duplicate_rule(raw: dict, issue: PreflightIssue) -> str | None`
Duplicate rule names in a legacy rule-mode Router — keep the first
- ▶ `def _fix_unknown_node_type(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix unknown node type step.
- ▶ `def _fix_model_not_in_catalog(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix model not in catalog step.
- ▶ `def _fix_unknown_node_config_field(raw: dict, issue: PreflightIssue) -> str | None`
Internal helper for the fix unknown node config field step.
- ▶ `def _accepts_plain_string(annotation: Any) -> bool`
Whether a pydantic field annotation permits an ordinary ``str`` — `true`
- ▶ `def _fix_dict_where_string_expected(raw: dict, issue: PreflightIssue) -> str | None`
A recurring, purely mechanical generation mistake: giving a field

- ▶ `def apply_deterministic_fixes(yaml_text: str, report: WorkflowPreflightReport) -> DeterministicFixResult`
Apply the deterministic fixes.
- ▶ `def _workflow_identity(yaml_text: str) -> tuple[str | None, set[str]]`
(name, node ids) for a parseable workflow dict, or (None, set()) if
- ▶ `def _preserves_identity(original_yaml: str, candidate_yaml: str) -> bool`
Guards against a repaired workflow that is well-formed but is not
- ▶ `def format_preflight_feedback(report: WorkflowPreflightReport) -> str`
Format the preflight feedback.
- ▶ `async def repair_with_llm(yaml_text: str, report: WorkflowPreflightReport, *, static_check: Callable[[str], Awaitable[WorkflowPreflightReport]], generate_yaml: Callable[[str, str | None, str | None], Awaitable[str]]) -> tuple[str, WorkflowPreflightReport, list[LlmRepairAttempt]]`
Feed remaining preflight errors back to the LLM up to

app/runtime/compiler.py — 738 lines

Compile a WorkflowSpec into a runnable LangGraph StateGraph.

- ▶ `def _make_runtime_fn(instance, bus: RunEventBus | None, services: dict)`
Wrap a NodeType instance so it conforms to LangGraph's
- ▶ `def _wire_edges(graph: StateGraph, edges: list[EdgeSpec], hitl_ids: set[str], multi_router_ids: set[str] | None=None) -> set[str]`
Add edges, return the set of source node ids (used to compute terminals).
- ▶ `def compile_workflow(spec: WorkflowSpec, checkpoint=..., services=None)`
Compute the compile workflow.

app/runtime/coordination.py — 106 lines

Small Redis coordination primitives shared by Uvicorn workers.

- ▶ `class RedisLease`
A renewable, compare-and-release distributed lease.
`async def acquire(self) -> bool`
Acquire the result.
`async def renew(self) -> bool`
Renew the result.
`async def release(self) -> None`
Release the result.
`async def keep_alive(self) -> None`
Return only when the lease is lost; cancellation stops the loop.

app/runtime/domain_state.py — 84 lines

Namespaced state for optional use-case packs.

- ▶ `class DomainStateRegistry`
Reducer registry keyed by a stable use-case namespace.
`def register(cls, namespace: str, reducer: DomainReducer) -> None`
Register the result.
`def reducer_for(cls, namespace: str) -> DomainReducer`
Compute the reducer for.
- ▶ `def _merge_mapping(left: Any, right: Any) -> Any`
Safe default for simple dict-shaped domain state.
- ▶ `def merge_domain_state(left: dict[str, Any] | None, right: dict[str, Any] | None) -> dict[str, Any]`
LangGraph reducer for state written by independent use-case packs.

app/runtime/events.py — 413 lines

Bounded, session-scoped pub/sub for workflow Server-Sent Events.

```
► class RunEvent
    Provides the RunEvent behaviour.
    def terminal(self) -> bool
        The terminal.
    def to_json(self) -> dict[str, Any]
        Compute the to json.
► class RunEventBus
    Provides the RunEventBus behaviour.
    def _key(run_id: str, session_id: str | None) -> tuple[str, str]
        Internal helper for the key step.
    async def publish(self, evt: RunEvent) -> None
        Publish the result.
    def _redis_names(run_id: str, session_id: str | None) -> tuple[str, str, str]
        Return opaque, tenant-scoped Redis keys without leaking identifiers.
    async def _publish_redis(self, evt: RunEvent) -> None
        Publish the redis.
    async def subscribe(self, run_id: str, session_id: str | None=None, *, after_event_id: int |
None=None) -> asyncio.Queue[RunEvent]
        Subscribe the result.
    async def _subscribe_redis(self, run_id: str, session_id: str | None, *, after_event_id: int |
None) -> asyncio.Queue[RunEvent]
        Subscribe the redis.
    def _decode_redis_event(fields: dict[Any, Any]) -> RunEvent
        Decode the redis event.
    def _offer(queue: asyncio.Queue[RunEvent], event: RunEvent) -> None
        Internal helper for the offer step.
    async def _pump_redis(self, pubsub: Any, queue: asyncio.Queue[RunEvent], last_seen: int) ->
None
        Internal helper for the pump redis step.
    async def unsubscribe(self, run_id: str, queue: asyncio.Queue[RunEvent], session_id: str |
None=None) -> None
        Unsubscribe the result.
    async def close(self) -> None
        Stop Redis subscription pumps before the shared client is closed.
```

app/runtime/executor.py — 268 lines

Compile a WorkflowSpec and run it.

```
► def _find_rejection(state: dict[str, Any]) -> dict[str, Any] | None
    Find the rejection.
► def _project_output(spec: WorkflowSpec, state: dict[str, Any], original_inputs: dict[str, Any])
-> dict[str, Any] | None
    Build the caller-facing output declared by the workflow contract.
► async def run_workflow(spec: WorkflowSpec, inputs: dict[str, Any], session_id: str | None=None,
collection_id: str='default', services: dict[str, Any] | None=None, run_id: str | None=None,
reused_node_results: dict[str, dict[str, Any]] | None=None, retry_source_run_id: str | None=None,
hitl_resume_decisions: dict[str, dict[str, Any]] | None=None, resume_replay: bool=False) ->
dict[str, Any]
    Run the workflow.
```

app/runtime/field_schema.py — 515 lines

Visual field schema — the authoring format behind the Builder's structured

- ▶ `class FieldSpec`
One row in the visual schema builder.
`def _migrate_legacy_item_enum_values(cls, data: Any) -> Any`
Fold list-of-enum values stored under the field-level 'enum_values'
`def name_is_an_identifier(cls, value: str) -> str`
Compute the name is an identifier.
`def shape_is_coherent(self) -> 'FieldSpec'`
Compute the shape is coherent.
- ▶ `class FieldPath`
One addressable value inside a compiled schema.
- ▶ `def validate_fields(fields: list[FieldSpec], *, where: str='output') -> None`
Reject duplicate names per level and runaway nesting depth.
- ▶ `def _scalar_annotation(kind: str, enum_values: list[str]) -> Any`
Internal helper for the scalar annotation step.
- ▶ `def _annotation_for(spec: FieldSpec, model_name: str, *, permit_none: bool) -> Any`
Internal helper for the annotation for step.
- ▶ `def _build_model(name: str, fields: list[FieldSpec]) -> Type[BaseModel]`
Build the model.
- ▶ `def _safe_model_name(name: str) -> str`
Internal helper for the safe model name step.
- ▶ `def build_response_model(fields: list[FieldSpec], *, model_name: str='StructuredOutput') -> Type[BaseModel]`
Compile the visual schema into a Pydantic model.
- ▶ `def json_schema(fields: list[FieldSpec], *, model_name: str='StructuredOutput') -> dict[str, Any]`
JSON Schema for the visual schema, via the compiled Pydantic model.
- ▶ `def field_paths(fields: list[FieldSpec], prefix: str='') -> list[FieldPath]`
Flatten a visual schema into typed dotted paths, parents included.
- ▶ `def _field_paths(fields: list[FieldSpec], prefix: str, *, parent_unavailable: bool) -> list[FieldPath]`
Internal helper for the field paths step.
- ▶ `def parse_fields(raw: Any) -> list[FieldSpec]`
Coerce YAML/JSON rows into FieldSpec, with a clear error on bad shape.
- ▶ `def describe_schema(fields: list[FieldSpec]) -> str`
A compact, human-readable contract used in prompts and the About tab.

app/runtime/hitl.py — 272 lines

HITL resume path.

- ▶ `class HITLResumeError`
Exception raised for the HITLResumeError case.
- ▶ `class HITLResumeConflict`
Another worker already claimed this paused checkpoint.
- ▶ `def _find_rejection(state: dict) -> dict | None`
Any node output carrying decision == 'reject' means a HITL gate was rejected.
- ▶ `def _resume_output(checkpoint: dict[str, Any] | None, final_state: dict[str, Any]) -> dict[str, Any] | None`
The same projection a fresh 'run_workflow' completion computes
- ▶ `async def resume_workflow(run_id: str, decision: dict[str, Any]) -> dict[str, Any]`
Resume a paused workflow with the user's decision payload.
- ▶ `def _validate_saved_decision(checkpoint: dict[str, Any], decision: dict[str, Any]) -> None`
Validate the saved decision.
- ▶ `async def resume_workflow_durable(run_id: str, decision: dict[str, Any], *, services: dict[str, Any] | None=None, session_id: str | None=None, actor: str='system') -> dict[str, Any]`
Resume in memory when possible, otherwise rebuild from Mongo.

app/runtime/loader.py — 36 lines

Loader module.

- ▶ `def load_workflow(path: str | Path) -> WorkflowSpec`
Load the workflow.
- ▶ `def load_workflow_from_string(yaml_text: str) -> WorkflowSpec`
Load the workflow from string.

app/runtime/logic_preflight.py — 2113 lines

Zero-token validation of authored business logic.

- ▶ `class FieldIndex`
Every value the workflow can produce, keyed by its reference path.
 - `def add(self, node_id: str, path: FieldPath) -> None`
Add the result.
 - `def get(self, reference: str) -> FieldPath | None`
Return the result.
 - `def node_of(self, reference: str) -> str | None`
Compute the node of.
 - `def paths_for(self, node_id: str) -> list[FieldPath]`
Compute the paths for.
- ▶ `def _register_builders() -> None`
Populate the typed-output builders lazily.
- ▶ `def _declared_value(parsed: Any, name: str) -> Any`
Internal helper for the declared value step.
- ▶ `def _infer_kind(value: Any) -> str`
Internal helper for the infer kind step.
- ▶ `def _json_schema_kind(schema: dict[str, Any]) -> str`
Internal helper for the json schema kind step.
- ▶ `def _json_schema_paths(schema: dict[str, Any], prefix: str) -> list[FieldPath]`
Flatten a JSON-Schema object's `properties` into typed dotted paths.
- ▶ `def _static_mcp_tool_registries() -> dict[str, dict[str, dict[str, Any]]]`
Per-server `TOOLS\_BY\_NAME` registries for the built-in MCP connectors.
- ▶ `def _static_mcp_tool_definition(server_id: Any, tool_name: Any) -> dict[str, Any] | None`
Internal helper for the static mcp tool definition step.
- ▶ `def _static_mcp_output_schema(server_id: Any, tool_name: Any) -> dict[str, Any] | None`
The declared output_schema for a tool this platform ships itself.
- ▶ `def _static_mcp_required_arguments(server_id: Any, tool_name: Any, arguments: dict[str, Any]) -> list[str] | None`
Argument names the tool's own static schema requires but `arguments`
- ▶ `def _static_subprocess_child_spec(config: dict[str, Any]) -> WorkflowSpec | None`
Load the referenced workflow's spec purely to read its shape (declared
- ▶ `def _shallow_child_node_paths(child_spec: WorkflowSpec, node_id: str) -> list[FieldPath]`
One node's typed output paths, one level deep, deliberately NOT a
- ▶ `def _transform_kind(operation: str) -> str`
Internal helper for the transform kind step.
- ▶ `def _kind_from_annotation(annotation: Any) -> tuple[str, bool]`
Best-effort (kind, nullable) for a declared output_schema field.
- ▶ `def _declared_output_paths(klass: Any) -> list[FieldPath]`
Typed paths for a node's own output_schema fields.
- ▶ `def build_field_index(spec: WorkflowSpec) -> FieldIndex`
Typed paths for every node in the workflow.
- ▶ `def validate_business_logic(spec: WorkflowSpec, *, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None=None) -> None`
Run every logic check. `issue` is preflight's own issue recorder.

- ▶ `def _check_conditional_join_reachability(spec: WorkflowSpec, issue: Callable[..., None]) -> None`
Two conditional dispatches into one shared step form an unsatisfiable join.
- ▶ `def _reaching_branches(edge: Any, node_id: str, forward: dict[str, set[str]]) -> set[str]`
Which of this router's branches can reach `node\_id`.
- ▶ `def _reachable_from(start: str, forward: dict[str, set[str]]) -> set[str]`
Internal helper for the reachable from step.
- ▶ `def _edge_targets(edge: Any) -> list[str]`
Internal helper for the edge targets step.
- ▶ `def _check_decision(node: NodeSpec, config: dict[str, Any], index: FieldIndex, nodes: dict[str, NodeSpec], issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None) -> None`
Check the decision.
- ▶ `def _check_router(node: NodeSpec, config: dict[str, Any], index: FieldIndex, nodes: dict[str, NodeSpec], issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None) -> None`
Check the router.
- ▶ `def _check_condition(condition: Condition, *, node: NodeSpec, index: FieldIndex, nodes: dict[str, NodeSpec], path: str, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None, label: str) -> None`
Check the condition.
- ▶ `def _check_threshold(condition: Condition, described: FieldPath, value: float, node: NodeSpec, path: str, issue: Callable[..., None], label: str) -> None`
Catch a threshold that can never do anything.
- ▶ `def _numeric_bounds(described: FieldPath) -> tuple[float | None, float | None] | None`
Bounds for a numeric field, when the schema declares them.
- ▶ `def _resolve_reference(reference: str, *, node: NodeSpec, index: FieldIndex, nodes: dict[str, NodeSpec], path: str, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None, label: str) -> FieldPath | None`
Resolve a rule/route reference, reporting what is wrong with it.
- ▶ `def _check_ai_task(node: NodeSpec, config: dict[str, Any], issue: Callable[..., None]) -> None`
Contract sanity on a visually built schema.
- ▶ `def _check_transform_agent(node: NodeSpec, config: dict[str, Any], issue: Callable[..., None]) -> None`
Same contract sanity as `\_check\_ai\_task`, for a new-style TransformAgent
- ▶ `def _check_output_field_enums(node: NodeSpec, fields: list[FieldSpec], issue: Callable[..., None]) -> None`
Check the output field enums.
- ▶ `def _check_external_actions(spec: WorkflowSpec, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None) -> None`
An outward-facing action with nothing between it and the model.
- ▶ `def _check_external_action(node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]) -> None`
A write or external_action call with nothing between it and the model.
- ▶ `def _check_subprocess_agents(spec: WorkflowSpec, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] | None) -> None`
Everything about a Subprocess step preflight can prove without
- ▶ `def _load_subprocess_refs(name: str) -> set[str]`
Workflow names a *saved* workflow's own SubprocessAgent nodes
- ▶ `def _find_subprocess_cycle(entry_refs: set[str]) -> list[str] | None`
Iterative cycle detection over the on-disk subprocess call graph.
- ▶ `def _check_sql_query_agents(spec: WorkflowSpec, issue: Callable[..., None]) -> None`
A literal `{...}` inside the `sql` field text would be substituted
- ▶ `def _check_python_snippet_agents(spec: WorkflowSpec, issue: Callable[..., None]) -> None`
Everything decidable about a Python snippet without running it:
- ▶ `def _check_mcp_agent(node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]) -> None`
MCPAgent chooses its own tools at runtime, so — unlike MCPToolAgent —

- ▶ `def _check_mcp_tool(node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]) -> None`
Validate an MCP tool step without contacting the server.
- ▶ `def unreachable_branch_targets(spec: WorkflowSpec, reachable: Iterable[str]) -> list[str]`
Router branch targets no run can reach. Complements the graph

app/runtime/node_events.py — 66 lines

Helpers for live event emission in the workflow runtime.

- ▶ `def sanitize_preview(value: Any) -> str`
Short, safe preview for the Cockpit card. Strips sensitive keys, truncates strings,
- ▶ `def is_graph_interrupt(exc: BaseException) -> bool`
LangGraph's interrupt class has moved between versions. Match by name
- ▶ `def interrupt_payload(exc: BaseException) -> Any`
Recover the actual value passed to `"langgraph.types.interrupt(...)"`.

app/runtime/pipeline_executor.py — 346 lines

Runs one pipeline stage at a time.

- ▶ `class PipelineExecutionError`
Raised for pipeline-level problems the caller must fix before running
- ▶ `def load_stage_workflow(stage: PipelineStageSpec) -> tuple[WorkflowSpec, str]`
Load the stage workflow.
- ▶ `def resolve_input_source(name: str, stage: PipelineStageSpec, pipeline_input_names: set[str], prior_stage_node_ids: list[tuple[str, set[str]]]) -> tuple[str, str | None]`
Where would this stage input come from? Used by both preflight (to
- ▶ `def _coerce_for_target(value: Any, input_spec: WorkflowInputSpec) -> Any`
Adapt an auto-matched or mapped value to the shape a target input
- ▶ `def materialize_stage_inputs(stage: PipelineStageSpec, stage_spec: WorkflowSpec, pipeline_inputs: dict[str, Any], stage_outputs_by_id: dict[str, dict[str, Any]]) -> dict[str, Any]`
Compute the real input values for one stage, right before running it.
- ▶ `def _pending_stage_records(spec: PipelineSpec) -> list[dict[str, Any]]`
Internal helper for the pending stage records step.
- ▶ `async def _collect_prior_stage_outputs(db, session: str, pipeline_doc: dict[str, Any], upto_index: int) -> dict[str, dict[str, Any]]`
Gather every completed prior stage's node outputs, by stage id.
- ▶ `async def _run_stage(*, pipeline_spec: PipelineSpec, pipeline_run_id: str, stage_index: int, session: str, services: dict[str, Any], pipeline_inputs: dict[str, Any], stage_outputs_by_id: dict[str, dict[str, Any]], collection_id: str, run_id: str | None=None) -> dict[str, Any]`
Run the stage.
- ▶ `async def run_pipeline(*, pipeline_spec: PipelineSpec, pipeline_yaml: str, pipeline_run_id: str | None, pipeline_inputs: dict[str, Any], session: str, services: dict[str, Any], collection_id: str='default', stage_run_id: str | None=None) -> dict[str, Any]`
Start a new pipeline run: create its record and run stage 0.
- ▶ `async def advance_pipeline(*, pipeline_run_id: str, session: str, services: dict[str, Any], collection_id: str='default', stage_run_id: str | None=None) -> dict[str, Any]`
Run the next stage of a pipeline that is gated awaiting approval.

app/runtime/pipeline_loader.py — 54 lines

Pipeline loader module.

- ▶ `def stage_workflow_path(workflow_name: str) -> Path`
Compute the stage workflow path.
- ▶ `def load_pipeline(path: str | Path) -> PipelineSpec`
Load the pipeline.

- ▶ `def load_pipeline_from_string(yaml_text: str) -> PipelineSpec`
Load the pipeline from string.

`app/runtime/pipeline_preflight.py` — 265 lines

Zero-token pipeline validation — never calls a node or LLM.

- ▶ `class PipelinePreflightReport`
Pydantic model defining the PipelinePreflightReport shape.
`def errors(self) -> list[PreflightIssue]`
The errors.
`def refresh(self) -> 'PipelinePreflightReport'`
Refresh the result.
- ▶ `class PipelinePreflightError`
Exception raised for the PipelinePreflightError case.
- ▶ `def _issue(report: PipelinePreflightReport, code: str, message: str, *, severity: PreflightSeverity=PreflightSeverity.ERROR, stage_id: str | None=None, suggestion: str | None=None) -> None`
Internal helper for the issue step.
- ▶ `def preflight_pipeline_yaml(yaml_text: str, *, provided_inputs: dict[str, Any] | None=None) -> PipelinePreflightReport`
Compute the preflight pipeline yaml.
- ▶ `def _add_check(report: PipelinePreflightReport, name: str, before: int, detail: str) -> None`
Add the check.
- ▶ `def require_pipeline_preflight(report: PipelinePreflightReport) -> None`
Compute the require pipeline preflight.

`app/runtime/pipeline_schema.py` — 111 lines

A Pipeline chains saved workflows so one workflow's node outputs become

- ▶ `class PipelineStageSpec`
Pydantic model defining the PipelineStageSpec shape.
`def id_is_not_reserved(self) -> 'PipelineStageSpec'`
Compute the id is not reserved.
- ▶ `class PipelineSpec`
The stable, use-case-neutral pipeline contract.
`def validate_stages(self) -> 'PipelineSpec'`
Validate the stages.
- ▶ `class PipelineStageResult`
One stage's linkage to its underlying workflow run, as tracked by a
- ▶ `class PipelineRunState`
In-memory shape of one pipeline run, mirrored in Mongo.

`app/runtime/preflight.py` — 2981 lines

Zero-token workflow validation.

- ▶ `class DuplicateYamlKeyError`
Raised when YAML silently would have overwritten a duplicate key.
- ▶ `class _UniqueKeyLoader`
Provides the UniqueKeyLoader behaviour.
- ▶ `class PreflightSeverity`
Enumeration of PreflightSeverity values.
- ▶ `class PreflightIssue`
Pydantic model defining the PreflightIssue shape.
- ▶ `class PreflightCheck`
Pydantic model defining the PreflightCheck shape.

- ▶ `class WorkflowPreflightReport`
Pydantic model defining the WorkflowPreflightReport shape.
 - `def errors(self) -> list[PreflightIssue]`
The errors.
 - `def warnings(self) -> list[PreflightIssue]`
The warnings.
 - `def refresh(self) -> 'WorkflowPreflightReport'`
Refresh the result.
- ▶ `class WorkflowPreflightError`
Carries the complete report when execution is blocked.
- ▶ `def _construct_unique_mapping(loader: _UniqueKeyLoader, node: yaml.MappingNode, deep: bool=False) -> dict[Any, Any]`
Internal helper for the construct unique mapping step.
- ▶ `def _issue(report: WorkflowPreflightReport, code: str, message: str, *, severity: PreflightSeverity=PreflightSeverity.ERROR, path: str | None=None, node_id: str | None=None, suggestion: str | None=None) -> None`
Internal helper for the issue step.
- ▶ `def _add_check(report: WorkflowPreflightReport, name: str, before: int, detail: str) -> None`
Add the check.
- ▶ `def _load_unique_yaml(yaml_text: str) -> Any`
Load the unique yaml.
- ▶ `def _parse_spec(yaml_text: str, report: WorkflowPreflightReport) -> WorkflowSpec | None`
Parse the spec.
- ▶ `def _validate_registry(report: WorkflowPreflightReport) -> None`
Validate the registry.
- ▶ `def _unknown_config_fields(node_spec: NodeSpec, node_class: type[NodeType]) -> list[str]`
Internal helper for the unknown config fields step.
- ▶ `def _validated_node_config(node_spec: NodeSpec) -> dict[str, Any]`
Internal helper for the validated node config step.
- ▶ `def _iter_model_values(config: Any, path: str='config') -> Iterable[tuple[str, str]]`
Internal helper for the iter model values step.
- ▶ `def _required_services_for_node(node: NodeSpec) -> set[str]`
Delegates to the node type's own ``required_services()`` override (see
- ▶ `def _validate_nodes(spec: WorkflowSpec, report: WorkflowPreflightReport) -> None`
Validate the nodes.
- ▶ `def _edge_targets(edge: EdgeSpec) -> list[str]`
Internal helper for the edge targets step.
- ▶ `def _adjacency(spec: WorkflowSpec) -> tuple[dict[str, set[str]], dict[str, set[str]]]`
Internal helper for the adjacency step.
- ▶ `def _reachable(start: str, graph: dict[str, set[str]]) -> set[str]`
Internal helper for the reachable step.
- ▶ `def _topological_order(reachable: set[str], forward: dict[str, set[str]], reverse: dict[str, set[str]]) -> list[str]`
Kahn's-algorithm topological sort restricted to ``reachable`` Nodes
- ▶ `def _guaranteed_before(entry: str, forward: dict[str, set[str]], reverse: dict[str, set[str]]) -> dict[str, set[str]]`
For each node reachable from entry, the set of nodes GUARANTEED to
- ▶ `def _has_cycle(graph: dict[str, set[str]]) -> bool`
Return whether cycle.
- ▶ `def _validate_router_edges(spec: WorkflowSpec, report: WorkflowPreflightReport) -> None`
Validate the router edges.
- ▶ `def _validate_typed_router(source: NodeSpec, config: dict[str, Any], mode: str, edge: EdgeSpec, edge_path: str, report: WorkflowPreflightReport) -> None`
Branch/route agreement for the Builder's visual router modes.
- ▶ `def _multi_router_ids(spec: WorkflowSpec) -> set[str]`
RouterAgent nodes configured for Multi-Route (selection="multi") —

- ▶ `def _plain_targets(spec: WorkflowSpec) -> dict[str, list[str]]`
Every target reached by a plain (non-HITL, non-router) edge, mapped
- ▶ `def _exclusive_branch_groups(spec: WorkflowSpec, forward: dict[str, set[str]]) -> list[list[set[str]]]`
For each SINGLE-selection router (condition+branches) edge with 2+
- ▶ `def _mutually_exclusive(a: str, b: str, exclusive_groups: list[list[set[str]]]) -> bool`
True if `a` and `b` are exclusively reachable via two DIFFERENT
- ▶ `def _validate_fanin_reachability(spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]]) -> None`
A plain-edge fan-in target (2+ declared predecessors combined into
- ▶ `def _multi_route_branch_sets(spec: WorkflowSpec, forward: dict[str, set[str]]) -> list[tuple[str, dict[str, set[str]]]]`
Per Multi-Route router edge: branch label -> nodes reachable ONLY via
- ▶ `def _selection_deps(node_id: str, branch_sets: list[tuple[str, dict[str, set[str]]]]) -> frozenset[tuple[str, str]]`
Every (multi_router_id, branch_label) this node's execution is
- ▶ `def _validate_multiroute_andjoin(spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]]) -> None`
A fan-in target whose declared predecessors depend on DIFFERENT
- ▶ `def _validate_start_end_edges(spec: WorkflowSpec, report: WorkflowPreflightReport) -> None`
Start represents the beginning of execution and End its terminus —
- ▶ `def _validate_start_fields(spec: WorkflowSpec, report: WorkflowPreflightReport) -> None`
Validates one Start node's own field/file_field list. Mirrors
- ▶ `def _validate_graph(spec: WorkflowSpec, report: WorkflowPreflightReport) -> tuple[dict[str, set[str]], dict[str, set[str]]]`
Validate the graph.
- ▶ `def _iter_strings(value: Any, path: str) -> Iterable[tuple[str, str]]`
Internal helper for the iter strings step.
- ▶ `def _annotation_permits_none(annotation: Any) -> bool`
True if a declared field type allows None (`X | None`/Optional[X]).
- ▶ `def _validate_template_output_path(spec: WorkflowSpec, current_node: NodeSpec, reference: str, report: WorkflowPreflightReport, path: str, forward: dict[str, set[str]], guaranteed: dict[str, set[str]]) -> None`
Validate the template output path.
- ▶ `def _matches_declared_field(reference: str, declared: set[str]) -> bool`
Does a dotted reference match one of a node's declared output paths?
- ▶ `def _validate_templates(spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]], reverse: dict[str, set[str]]) -> None`
Validate the templates.
- ▶ `def _validate_inputs(spec: WorkflowSpec, provided_inputs: dict[str, Any] | None, report: WorkflowPreflightReport) -> None`
Validate the inputs.
- ▶ `def _compile_dry_run(spec: WorkflowSpec, services: dict[str, Any] | None, report: WorkflowPreflightReport) -> None`
Internal helper for the compile dry run step.
- ▶ `def _validate_guided_experience(spec: WorkflowSpec, report: WorkflowPreflightReport) -> None`
Validate authored Guided Run copy without penalising legacy workflows.
- ▶ `def preflight_workflow_yaml(yaml_text: str, *, provided_inputs: dict[str, Any] | None=None, services: dict[str, Any] | None=None, compile_graph: bool=True) -> WorkflowPreflightReport`
Run all deterministic checks. This function never calls a node or LLM.
- ▶ `def _validate_business_logic(spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]], reverse: dict[str, set[str]]) -> None`
Check visually authored rules, routes and schemas (zero tokens).
- ▶ `def preflight_workflow_spec(spec: WorkflowSpec, *, provided_inputs: dict[str, Any] | None=None, services: dict[str, Any] | None=None, compile_graph: bool=False) -> WorkflowPreflightReport`
Validate an already parsed spec without changing or running it.

```

▶ async def _probe_services(spec: WorkflowSpec, services: dict[str, Any], report:
WorkflowPreflightReport, *, require_run_history: bool, owner_scope_id: str | None=None) -> None
    Probe the services.
▶ def _probe_integration_nodes(spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport)
-> None
    Zero-token: checks the referenced connection exists on the service —
▶ def _probe_web_search_nodes(spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport) ->
None
    Zero-token: WebSearchService.resolve_provider does a settings check
▶ def _probe_image_generation_nodes(spec: WorkflowSpec, service: Any, report:
WorkflowPreflightReport) -> None
    Zero-token: OpenAllImageGenerationService.available() checks settings
▶ def _probe_kimi_vision_nodes(spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport)
-> None
    Zero-token: KimiVisionService.available() checks settings only —
▶ async def _probe_rag_resource_nodes(spec: WorkflowSpec, repository: Any, report:
WorkflowPreflightReport, *, owner_scope_id: str) -> None
    Confirm every referenced Knowledge resource exists and is usable.
▶ async def _probe_workflow_model_access(spec: WorkflowSpec, llm: Any, report:
WorkflowPreflightReport) -> None
    Confirm provider/project access without making a generation request.
▶ async def preflight_workflow_for_run(yaml_text: str, *, provided_inputs: dict[str, Any] | None,
services: dict[str, Any], probe_services: bool=True, require_run_history: bool=True,
owner_scope_id: str | None=None) -> WorkflowPreflightReport
    Strict API gate used immediately before a new or retried run.
▶ def require_preflight(report: WorkflowPreflightReport) -> None
    Compute the require preflight.

```

app/runtime/rules.py — 886 lines

Deterministic business-rule evaluation.

```

▶ class Condition
    One leaf test: ``field operator value``.
    def value_present_when_needed(self) -> 'Condition'
        Compute the value present when needed.
▶ class ConditionGroup
    AND / OR / NOT over conditions and nested groups (§10).
    def group_is_usable(self) -> 'ConditionGroup'
        Group the is usable.
▶ class Action
    What a matched rule does.
    def _rename_legacy_append(cls, value: Any) -> Any
        `merge` is `append`'s new name — the accumulate-into-a-list
    def value_matches_operation(self) -> 'Action'
        Compute the value matches operation.
▶ class Rule
    One IF/THEN rule.
    def rule_is_decidable(self) -> 'Rule'
        Compute the rule is decidable.
▶ class ConditionTrace
    Why one condition passed or failed — rendered verbatim in the UI.
▶ class GroupTrace
    Pydantic model defining the GroupTrace shape.
▶ class RuleTrace
    Pydantic model defining the RuleTrace shape.
▶ class RuleEvaluation

```

Result of evaluating a rule set: the facts, plus the full reasoning.

```
def explanation_lines(self) -> list[str]
```

Flat, human-readable "matched rules" list for the run trace (§24).

- ▶ `def operators_for_type(field_type: str) -> tuple[str, ...]`
Compute the operators for type.
- ▶ `def resolve_path(context: dict[str, Any], path: str) -> Any`
Read a dotted path out of workflow state, returning ``\_MISSING`` sentinel
- ▶ `def path_exists(context: dict[str, Any], path: str) -> bool`
Compute the path exists.
- ▶ `def _resolve(context: dict[str, Any], path: str) -> Any`
Resolve the result.
- ▶ `def _leaf_traces(node: GroupTrace | ConditionTrace | None) -> Iterable[ConditionTrace]`
Internal helper for the leaf traces step.
- ▶ `def _as_number(value: Any) -> float | None`
Internal helper for the as number step.
- ▶ `def _is_empty(value: Any) -> bool`
Return whether empty.
- ▶ `def _contains(haystack: Any, needle: Any) -> bool`
Internal helper for the contains step.
- ▶ `def _boolish(value: Any) -> bool | None`
Read a value as a boolean, or None if it isn't one.
- ▶ `def _equals(actual: Any, expected: Any) -> bool`
Internal helper for the equals step.
- ▶ `def _truthy(value: Any) -> bool`
Internal helper for the truthy step.
- ▶ `def evaluate_condition(condition: Condition, context: dict[str, Any]) -> ConditionTrace`
Compute the evaluate condition.
- ▶ `def _previewable(value: Any) -> Any`
Keep traces small — they are stored in run history and sent over SSE.
- ▶ `def _summarise(condition: Condition, actual: Any, missing: bool, matched: bool) -> str`
Internal helper for the summarise step.
- ▶ `def evaluate_group(group: ConditionGroup, context: dict[str, Any]) -> GroupTrace`
Compute the evaluate group.
- ▶ `def evaluate_rules(rules: list[Rule], context: dict[str, Any], *, initial: dict[str, Any] | None=None) -> RuleEvaluation`
Evaluate every rule in order and accumulate the facts they set.
- ▶ `def _apply_action(action: Action, values: dict[str, Any], context: dict[str, Any]) -> dict[str, Any]`
Write one fact. Action values may themselves reference workflow state.
- ▶ `def collect_condition_fields(group: ConditionGroup | None) -> list[str]`
Every field path a group reads — used by preflight to check references.
- ▶ `def collect_conditions(group: ConditionGroup | None) -> list[Condition]`
Every leaf condition in a group, for type checking in preflight.

app/runtime/schema.py — 558 lines

Schema module.

- ▶ `class WorkflowFileRef`
Stable object-storage reference passed through workflow state.
- ▶ `class WorkflowInputSpec`
Pydantic model defining the WorkflowInputSpec shape.
 - `def validate_file_options(self) -> 'WorkflowInputSpec'`
Validate the file options.
 - `def effective_max_files(self, platform_limit: int) -> int`
Compute the effective max files.
- ▶ `class ModelRoutingPolicy`

- Automatic-selection policy consulted by the deterministic ModelRouter
- ▶ `class GuidedStageSpec`
Optional business-language grouping used by Guided Run.
- ▶ `class WorkflowExperienceSpec`
Workflow-level settings for the non-technical runtime surface.
- ▶ `class NodeExperienceSpec`
Optional user-facing copy and handoff metadata for one node.
- ▶ `class LibraryDurationRange`
Pydantic model defining the LibraryDurationRange shape.
- ▶ `class LibraryHumanReviews`
Pydantic model defining the LibraryHumanReviews shape.
- ▶ `class LibraryEvidencePolicy`
Pydantic model defining the LibraryEvidencePolicy shape.
- ▶ `class LibraryMetadataSpec`
Optional catalog/presentation metadata for the Workflow Library.
- ▶ `class NodeSpec`
One node instance in a workflow.
 - `def selected_model_must_be_allowed(self) -> 'NodeSpec'`
Compute the selected model must be allowed.
 - `def data_protection_mode_must_be_valid(self) -> 'NodeSpec'`
Compute the data protection mode must be valid.
 - `def effective_config(self) -> dict[str, Any]`
Compute the effective config.
- ▶ `class StaticVariable`
Pydantic model defining the StaticVariable shape.
- ▶ `class WorkflowOutputNode`
Pydantic model defining the WorkflowOutputNode shape.
- ▶ `class WorkflowOutputSpec`
Pydantic model defining the WorkflowOutputSpec shape.
- ▶ `class EdgeSpec`
Edges support simple, fan-out, and conditional routing.
- ▶ `class WorkflowSpec`
The stable, use-case-neutral workflow contract.
 - `def validate_graph_references(self) -> 'WorkflowSpec'`
Validate the graph references.
 - `def derive_inputs_from_start_node(self) -> 'WorkflowSpec'`
A Start node (app/nodes/start.py) is the single place a workflow

`app/runtime/sleep_guard.py` — 77 lines

macOS-only guard against idle sleep while a workflow is executing.

- ▶ `def _enabled() -> bool`
Internal helper for the enabled step.
- ▶ `async def acquire() -> None`
Block idle sleep for the duration of one workflow run.
- ▶ `async def release() -> None`
Release this run's hold; sleep is allowed again once none remain.

`app/runtime/snippet_client.py` — 122 lines

App-side client for the snippet-runner sidecar (app/runtime/snippet_daemon.py).

- ▶ `class SnippetRunnerUnavailable`
The sidecar could not be reached at all — refused connection, no
- ▶ `class SnippetRunnerClient`
Provides the SnippetRunnerClient behaviour.


```

    async def run(self, code: str, inputs: dict[str, Any], *, timeout_seconds: float, memory_mb:
int) -> dict[str, Any]
    Run the result.
    async def probe(self) -> None
    Preflight health check (SNIPPET_RUNNER_UNAVAILABLE) — a real,
    async def _call(self, request: dict[str, Any], timeout_seconds: float) -> dict[str, Any]
    Internal helper for the call step.

```

app/runtime/snippet_daemon.py — 221 lines

The snippet-runner sidecar's own entrypoint.

- ▶ `def _socket_path() -> str`
Internal helper for the socket path step.
- ▶ `def _clamped(request: dict[str, Any]) -> SnippetRequest`
Never trust the client's own limits past the sidecar's own ceiling —
- ▶ `def _rlimit_preexec(memory_mb: int, timeout_seconds: float)`
Runs in the child, after fork, before exec — this is real isolation,
- ▶ `async def _execute(request: SnippetRequest) -> dict[str, Any]`
Execute the result.
- ▶ `async def _handle(reader: asyncio.StreamReader, writer: asyncio.StreamWriter) -> None`
Handle the result.
- ▶ `async def main() -> None`
Compute the main.

app/runtime/snippet_protocol.py — 151 lines

Shared shape between the app and the snippet-runner sidecar.

- ▶ `class SnippetRequest`
Provides the SnippetRequest behaviour.
- ▶ `class SnippetResponse`
Provides the SnippetResponse behaviour.
- ▶ `async def read_message(reader: asyncio.StreamReader, limit: int=MAX_MESSAGE_BYTES) -> bytes`
Read one EOF-terminated message, accumulating across as many socket

app/runtime/state.py — 71 lines

Shared mutable state that flows through every LangGraph node.

- ▶ `class WorkflowState`
Provides the WorkflowState behaviour.
- ▶ `def merge_node_outputs(a: dict[str, Any], b: dict[str, Any]) -> dict[str, Any]`
Reducer for node_outputs.

app/runtime/templating.py — 228 lines

Resolve `{{node_id.field.subfield}}` expressions inside node configs.

- ▶ `def _lookup(path: str, state: dict) -> Any`
Walk a dotted path against workflow state.
- ▶ `def _resolve_reference(match: 're.Match[str]', state: dict) -> Any`
Resolve one matched reference, honouring the optional marker.
- ▶ `def _as_text(value: Any) -> str`
Render a substituted value inside a larger string.
- ▶ `def resolve(value: Any, state: dict) -> Any`
Recursively walk a config value and replace template strings.
- ▶ `def _union_members(annotation: Any) -> list[Any]`

Internal helper for the union members step.

- ▶ `def _accepts_none(annotation: Any) -> bool`
Whether a value of None satisfies this annotation.
- ▶ `def _element_annotation(annotation: Any, container: type, position: int) -> Any`
The declared type of a dict value / list item, or `_UNKNOWN`.
- ▶ `def prune_absent(config: dict[str, Any], schema: Any) -> dict[str, Any]`
Drop config values that resolved to None but cannot be None.

security (17 modules)

app/security/__init__.py — 0 lines

(no summary)

app/security/audit.py — 127 lines

Audit log writer + reader.

- ▶ `def _require_session(session_id: str) -> str`
The isolation boundary, enforced in ONE place.
- ▶ `def summarize_payload(data: dict[str, Any] | None) -> dict[str, Any]`
Record SHAPE, not CONTENT.
- ▶ `async def write_audit_event(db, run_id: str, session_id: str, node_id: str, event_type: str, actor: str='system', payload: dict[str, Any] | None=None) -> None`
Write the audit event.
- ▶ `async def read_audit_events(db, session_id: str, run_id: str | None=None, limit: int=200) -> list[dict[str, Any]]`
The ONLY read path. session_id is always AND-ed into the query.

app/security/dependencies.py — 110 lines

Dependencies module.

- ▶ `class CurrentUser`
Provides the CurrentUser behaviour.
- ▶ `def _user_from_token(token: str | None) -> CurrentUser`
Internal helper for the user from token step.
- ▶ `async def get_current_user(token: str | None=Depends(oauth2_scheme), access_token: str | None=Cookie(default=None)) -> CurrentUser`
Return the current user.
- ▶ `async def get_current_user_cookie_or_header(token: str | None=Depends(oauth2_scheme), access_token: str | None=Cookie(default=None)) -> CurrentUser`
Header first (API clients), then HttpOnly cookie (browser EventSource).
- ▶ `def require_permission(permission: str)`
Compute the require permission.

app/security/entity_ner.py — 92 lines

Lazy-loaded spaCy NER safety net — PERSON/ORG spans not already known.

- ▶ `class EntityMatch`
Provides the EntityMatch behaviour.
- ▶ `def _load_pipeline()`
Load the pipeline.
- ▶ `def extract_entities(text: str) -> list[EntityMatch]`
PERSON/ORG spans via spaCy. Synchronous/CPU-bound — callers on the

app/security/entity_protection_errors.py — 34 lines

Fail-closed exception hierarchy for confidential entity protection.

- ▶ `class EntityProtectionError`
Base class for every fail-closed entity-protection error.
- ▶ `class EntityVaultUnavailableError`

The Mongo-backed mapping vault could not be reached (down/timeout).

- ▶ `class EntityScanUnavailableError`
The spaCy NER safety-net pass crashed or could not run.
- ▶ `class VaultKeyMisconfiguredError`
entity_vault_master_key is missing, a placeholder, or too short.
- ▶ `class UnresolvedEntityRiskError`
A residual entity candidate could not be confidently tokenized.
- ▶ `class ResponseLeakDetectedError`
A registered real entity value appeared verbatim in a model response.

`app/security/entity_registry.py` — 154 lines

Explicit per-scope entity registry — the primary detection mechanism.

- ▶ `class EntityRegistry`
Provides the EntityRegistry behaviour.
`async def register(self, *, session_id: str, collection_id: str, entity_type: str, value: str, source: str='manual') -> str`
Register the result.
`async def list_entities(self, *, session_id: str, collection_id: str) -> list[dict[str, str]]`
List the entities.
`async def delete(self, *, session_id: str, collection_id: str, entity_type: str, value: str) -> bool`
Delete the result.
`async def sync_from_partners(self, *, session_id: str, collection_id: str, graph: Any) -> list[str]`
Register every partner on a finalized ProposalGraph.

`app/security/entity_tokenizer.py` — 402 lines

Entity tokenization orchestration — the Phase 1 core guarantee.

- ▶ `class ProcessingMode`
Enumeration of ProcessingMode values.
- ▶ `class TokenizeResult`
Provides the TokenizeResult behaviour.
- ▶ `class DetokenizeResult`
Provides the DetokenizeResult behaviour.
- ▶ `class _Span`
Provides the Span behaviour.
- ▶ `class EntityTokenizerService`
Provides the EntityTokenizerService behaviour.
`def registry(self) -> EntityRegistry`
Exposed for the manual-registration API and proposal-graph sync.
`async def ensure_indexes(self) -> None`
Ensure the indexes.
`async def tokenize(self, text: str, *, session_id: str, collection_id: str='default', mode: ProcessingMode=ProcessingMode.PSEUDONYMISED) -> TokenizeResult`
Tokenize the result.
`async def _detect_spans(self, text: str, *, session_id: str, collection_id: str) -> list[_Span]`
Detect the spans.
`async def detokenize(self, value: Any, *, session_id: str, collection_id: str='default') -> DetokenizeResult`
Recursively walks str/dict/list, restoring placeholders in every
`async def _detokenize_text(self, text: str, *, session_id: str, collection_id: str) -> DetokenizeResult`
Detokenize the text.

- ▶ `def _overlaps(start: int, end: int, claimed: list[tuple[int, int]]) -> bool`
Internal helper for the overlaps step.
- ▶ `def _boundary_pattern(value: str) -> re.Pattern[str]`
Internal helper for the boundary pattern step.

app/security/entity_vault.py — 388 lines

Encrypted, reversible placeholder<->original-value vault.

- ▶ `class EntityVault`
Mongo-backed envelope-encrypted mapping vault, scoped per (session, collection).
 - `def _mappings(self)`
The mappings.
 - `def _counters(self)`
The counters.
 - `async def ensure_indexes(self) -> None`
Ensure the indexes.
 - `async def get_or_create_placeholder(self, *, session_id: str, collection_id: str, entity_type: str, real_value: str, source: str) -> str`
Stable placeholder for one entity within a scope.
 - `async def list_scope_entities(self, *, session_id: str, collection_id: str) -> list[dict[str, str]]`
Decrypted known entities for a scope — feeds the registry-first
 - `async def resolve_placeholders(self, *, session_id: str, collection_id: str, placeholders: set[str]) -> dict[str, str]`
Batch placeholder -> real value lookup, for detokenizing a response.
 - `async def delete_entity(self, *, session_id: str, collection_id: str, entity_type: str, value: str) -> bool`
Delete the entity.
- ▶ `def _load_kek() -> bytes`
Derive the vault's Key Encryption Key from settings, failing closed.
- ▶ `def _normalize(value: str) -> str`
Normalize the result.
- ▶ `def entity_hash(entity_type: str, value: str) -> str`
Deterministic match key for one (type, normalized value) pair.
- ▶ `def _mapping_id(session_id: str, collection_id: str, ehash: str) -> str`
Internal helper for the mapping id step.
- ▶ `def _counter_id(session_id: str, collection_id: str, entity_type: str) -> str`
Internal helper for the counter id step.
- ▶ `def _expiry() -> datetime`
Internal helper for the expiry step.
- ▶ `def _seal(real_value: str, *, session_id: str, collection_id: str, placeholder: str) -> dict[str, bytes]`
Internal helper for the seal step.
- ▶ `def _unseal(doc: dict[str, Any], *, session_id: str, collection_id: str, placeholder: str) -> str`
Internal helper for the unseal step.
- ▶ `async def ensure_indexes(db: Any) -> None`
Module-level convenience matching this codebase's ``ensure\_\*\_indexes`` convention.

app/security/guardrails.py — 252 lines

Deterministic input/output guardrails for workflow execution.

- ▶ `class GuardrailViolation`
Raised when a request or generated output violates a hard guardrail.
- ▶ `class GuardrailFinding`
Provides the GuardrailFinding behaviour.

- ▶ `class GuardrailResult`
Provides the GuardrailResult behaviour.
- ▶ `def _redact_matches(text: str, patterns: tuple[tuple[str, re.Pattern[str]], ...], *, path: str) -> tuple[str, list[GuardrailFinding]]`
Redact the matches.
- ▶ `def _walk(value: Any, *, path: str, transform) -> tuple[Any, list[GuardrailFinding]]`
Internal helper for the walk step.
- ▶ `def check_workflow_inputs(value: Any) -> GuardrailResult`
Block injection and apply the configured PII policy to direct inputs.
- ▶ `def check_generated_output(value: Any) -> GuardrailResult`
Remove credentials/private keys from model and tool output.

app/security/jwt_handler.py — 69 lines

Jwt handler module.

- ▶ `def create_access_token(data: dict[str, Any]) -> str`
Create the access token.
- ▶ `def decode_token(token: str) -> dict[str, Any]`
Decode the token.

app/security/llm_policy.py — 269 lines

Data classification + OPA policy enforcement for direct provider calls.

- ▶ `class ProviderComplianceMetadata`
Provides the ProviderComplianceMetadata behaviour.
- ▶ `class PolicyDecision`
Provides the PolicyDecision behaviour.
- ▶ `class OpaClient`
Minimal OPA REST client — mirrors omniroute/src/lib/policy/opa/client.ts exactly
`async def evaluate(self, policy_input: dict) -> PolicyDecision`
Compute the evaluate.
- ▶ `def processing_mode_to_data_class(processing_mode: str | None) -> DataClass`
Maps Eurskem's per-node ProcessingMode onto the classification lattice.
- ▶ `def effective_data_class(declared: DataClass, *, entities_detected: bool) -> DataClass`
max(declared, detected) — detecting a real entity in the text implies at least
- ▶ `def provider_and_backend_for_gateway(gateway_class_name: str) -> tuple[str, Backend]`
Compute the provider and backend for gateway.
- ▶ `def _parse_csv_env(name: str) -> frozenset[str]`
Parse the csv env.
- ▶ `def get_provider_compliance_metadata(provider: str | None) -> ProviderComplianceMetadata`
Operator-configured ZDR/data-collection facts — a business/legal fact about Eurskem's
- ▶ `def get_default_opa_client() -> OpaClient`
Return the default opa client.
- ▶ `async def enforce_policy(*, workspace_id: str, data_class: DataClass, gateway_class_name: str, model: str, capabilities: list[str] | None=None, opa_client: OpaClient | None=None) -> PolicyDecision`
Evaluates the routing policy for one candidate (provider, model) pair. Raises

app/security/middleware.py — 239 lines

HTTP request identity, limits, security headers, and Redis rate limiting.

- ▶ `class RequestContextMiddleware`
Provides the RequestContextMiddleware behaviour.
`async def dispatch(self, request: Request, call_next: Callable) -> Response`
Dispatch the result.

- ▶ `class RequestSizeLimitMiddleware`
Provides the `RequestSizeLimitMiddleware` behaviour.
`async def dispatch(self, request: Request, call_next: Callable) -> Response`
Dispatch the result.
- ▶ `class RedisRateLimitMiddleware`
Provides the `RedisRateLimitMiddleware` behaviour.
`async def dispatch(self, request: Request, call_next: Callable) -> Response`
Dispatch the result.
- ▶ `def _rate_limit_subject(request: Request) -> str`
Internal helper for the rate limit subject step.
- ▶ `def _route_label(request: Request) -> str`
Internal helper for the route label step.

`app/security/passwords.py` — 41 lines

Argon2 password hashing helpers for local production authentication.

- ▶ `def hash_password(password: str) -> str`
Hash the password.
- ▶ `def verify_password(password: str, encoded: str | None) -> bool`
Verify the password.

`app/security/presidio_client.py` — 117 lines

Presidio Analyzer client — the PII/entity detection safety-net tier.

- ▶ `class PresidioMatch`
Provides the `PresidioMatch` behaviour.
- ▶ `def _map_entity_type(presidio_type: str) -> str`
Map the entity type.
- ▶ `async def analyze(text: str, *, language: str='en', score_threshold: float=0.5, client: httpx.AsyncClient | None=None) -> list[PresidioMatch]`
Calls POST {presidio_analyzer_url}/analyze. Raises `EntityScanUnavailableError` on any

`app/security/rbac.py` — 34 lines

Rbac module.

- ▶ `class Role`
Enumeration of Role values.
- ▶ `def has_permission(role: Role, permission: str) -> bool`
Return whether permission.

`app/security/sso_stub.py` — 27 lines

Microsoft SSO stub.

- ▶ `def stub_login(username: str, role: str, session_id: str | None=None) -> str`
Issue a JWT as if Azure AD had authenticated the user.

`app/security/users.py` — 105 lines

Mongo-backed local users used by the IONOS production deployment.

- ▶ `async def ensure_user_indexes(db: Any) -> None`
Ensure the user indexes.
- ▶ `def validate_username(username: str) -> str`
Validate the username.

- ▶ `async def upsert_local_user(db: Any, *, username: str, password: str, role: Role, active: bool=True) -> None`
Upsert the local user.
- ▶ `async def authenticate_local_user(db: Any, *, username: str, password: str) -> dict[str, Any] | None`
Authenticate the local user.

storage (2 modules)

app/storage/__init__.py — 0 lines

(no summary)

app/storage/minio_client.py — 272 lines

S3-compatible object storage client.

- ▶ **class ObjectStore**
S3-compatible object store wrapper.
 - def ensure_bucket(self, bucket: str | None=None) -> None**
Create the bucket if it doesn't exist. Safe to call repeatedly.
 - def put_file(self, local_path: Path, key: str | None=None, content_type: str | None=None, extra_metadata: dict[str, str] | None=None) -> str**
Upload a local file. Returns the object key it was stored under.
 - def put_bytes(self, data: bytes, key: str, content_type: str | None=None) -> str**
Upload raw bytes under a given key. Used for generated artifacts.
 - def get_bytes(self, key: str) -> bytes**
Download an object's contents. Raises ClientError on 404.
 - def object_exists(self, key: str) -> bool**
Cheap existence check using HEAD. Used by the pipeline for idempotency.
 - def list_objects(self, prefix: str) -> list[dict[str, Any]]**
List objects under a key prefix. Returns [{key, size, last_modified}].
 - def presigned_url(self, key: str, expires_seconds: int=3600) -> str**
Generate a temporary URL that grants read access to one object.
- ▶ **def content_hash(path: Path, chunk_size: int=65536) -> str**
Compute SHA256 hash of file contents.
- ▶ **def key_for_path(path: Path) -> str**
The object key a file should be stored under: 'sha256:<hash>.<ext>'.
- ▶ **def knowledge_key_for_path(path: Path, owner_scope_id: str) -> str**
Content-addressed key for a Knowledge Studio upload, scoped by owner.
- ▶ **def get_object_store() -> ObjectStore**
Lazily-constructed module-level store. Use this from app code.

tools (10 modules)

app/tools/content_length.py — 23 lines

Character-count helper for the proposal-size guardrails.

- ▶ `def narrative_char_count(content: str) -> int`
Return `content`'s length with embedded base64 image data URLs excluded.

app/tools/database_lookup.py — 218 lines

Bounded clients for documented public database APIs.

- ▶ `class DatabaseResponse`
Provides the DatabaseResponse behaviour.
- ▶ `class PublicDatabaseLookupService`
Rate-limited, allow-listed public database transport.
 - `async def close(self) -> None`
Close the result.
 - `async def query_eurostat(self, query: StructuredDatasetQuery, *, timeout_seconds: float=45.0, max_response_bytes: int=20 * 1024 * 1024) -> DatabaseResponse`
Query the eurostat.
 - `async def _client(self, timeout_seconds: float) -> httpx.AsyncClient`
Internal helper for the client step.
 - `async def _rate_limit(self) -> None`
Internal helper for the rate limit step.
- ▶ `def _pairs_for_audit(pairs: list[tuple[str, str]]) -> dict[str, Any]`
Internal helper for the pairs for audit step.
- ▶ `def get_database_lookup_service() -> PublicDatabaseLookupService`
Return the database lookup service.

app/tools/docx_proposal_rendering.py — 1966 lines

Secure Horizon proposal HTML/Markdown to editable DOCX rendering.

- ▶ `class ProposalDocxRenderResult`
Provides the ProposalDocxRenderResult behaviour.
- ▶ `class _InlineStyle`
Provides the InlineStyle behaviour.
- ▶ `class _TablePlacement`
Provides the TablePlacement behaviour.
- ▶ `class _BuildState`
Provides the BuildState behaviour.
- ▶ `class _HorizonDocxBuilder`
Provides the HorizonDocxBuilder behaviour.
 - `def _configure_document(self) -> None`
Configure the document.
 - `def _configure_styles(self) -> None`
Configure the styles.
 - `def _configure_footnote_styles(self, styles, normal) -> None`
Configure the footnote styles.
 - `def _configure_header_footer(self, section) -> None`
Configure the header footer.
 - `def add_cover(self, evidence_blockers: list[str]) -> None`
Add the cover.
 - `def add_toc(self, entries: list[dict[str, Any]]) -> None`

```

Add the toc.
def add_fragment(self, fragment: str) -> None
Add the fragment.
def _render_block(self, node, *, list_level: int=0) -> None
Render the block.
def _append_text_with_footnotes(self, paragraph, text: str, style: _InlineStyle) -> None
Append the text with footnotes.
def _append_inline(self, paragraph, container, style: _InlineStyle | None=None) -> None
Append the inline.
def _render_list(self, list_tag: Tag, *, list_level: int) -> None
Render the list.
def _table_placements(self, table_tag: Tag) -> tuple[list[_TablePlacement], int, int]
Internal helper for the table placements step.
def _render_table(self, table_tag: Tag) -> None
Render the table.
def _placeholder_text(self, figure: Tag) -> str
Internal helper for the placeholder text step.
def _render_placeholder(self, prompt: str) -> None
Render the placeholder.
def _decode_data_image(self, source: str) -> bytes
Decode the data image.
def _render_image(self, image: Tag, caption: Tag | None=None) -> None
Render the image.
def _render_figure(self, figure: Tag) -> None
Render the figure.
def add_bibliography(self, citation_registry: list[dict[str, Any]]) -> None
Add the bibliography.
def add_evidence_annex(self, evidence_qa: dict[str, Any], evidence_blockers: list[str]) -> None
Add the evidence annex.
def save(self) -> bytes
Save the result.
▶ def _set_run_font(run, *, name: str='Arial', size: float | None=None, color: str | None=None,
bold: bool | None=None, italic: bool | None=None) -> None
Set the run font.
▶ def _set_shading(element, fill: str) -> None
Set the shading.
▶ def _set_run_shading(run, fill: str) -> None
Set the run shading.
▶ def _append_styled_text(paragraph, text: str, style: '_InlineStyle') -> None
Append the styled text.
▶ def _set_paragraph_borders(paragraph, *, left: tuple[str, int, int] | None=None, bottom:
tuple[str, int, int] | None=None) -> None
Set the paragraph borders.
▶ def _set_cell_shading(cell, fill: str) -> None
Set the cell shading.
▶ def _set_cell_margins(cell, margins: dict[str, int] | None=None) -> None
Set the cell margins.
▶ def _set_cell_width(cell, width_dxa: int) -> None
Set the cell width.
▶ def _set_repeat_header(row) -> None
Set the repeat header.
▶ def _set_table_borders(table, *, style: str='single', color: str=_BORDER, size: int=4) -> None
Set the table borders.
▶ def _set_table_geometry(table, widths_dxa: list[int], *, indent_dxa: int=_TABLE_INDENT_DXA) ->
None
Apply fixed Word table geometry with matching grid and cell widths.

```

```

▶ def _column_widths(rows: list[list[str]], column_count: int, *, total_width:
int=_TABLE_WIDTH_DXA) -> list[int]
    Internal helper for the column widths step.
▶ def _set_update_fields(doc: DocxDocument) -> None
    Set the update fields.
▶ def _add_field(paragraph, instruction: str, fallback: str='') -> None
    Add the field.
▶ def _add_toc_field(paragraph) -> None
    Add the toc field.
▶ def _add_external_hyperlink(paragraph, text: str, url: str, *, bold: bool=False, italic:
bool=False) -> None
    Add the external hyperlink.
▶ def _bookmark_name(name: str, bookmark_id: int) -> str
    Internal helper for the bookmark name step.
▶ def _add_internal_hyperlink(paragraph, text: str, anchor: str, *, bookmark_id: int, bold:
bool=False, size: float=10) -> None
    Add the internal hyperlink.
▶ def _add_bookmark(paragraph, name: str, bookmark_id: int) -> None
    Add the bookmark.
▶ def _add_footnote_reference(paragraph, footnote_id: int) -> None
    Add the footnote reference.
▶ def _footnote_body_text(number: int, citation: dict[str, Any]) -> str
    Internal helper for the footnote body text step.
▶ def _build_footnotes_part_xml(footnote_bodies: dict[int, str]) -> bytes
    Build the footnotes part xml.
▶ def _attach_footnotes_part(doc: DocxDocument, footnote_bodies: dict[int, str]) -> None
    Internal helper for the attach footnotes part step.
▶ def _toc_with_pdf_pages(entries: list[dict[str, Any]], pdf: bytes) -> list[dict[str, Any]]
    Attach deterministic page numbers from the matching PDF layout.
▶ def _set_alt_text(inline_shape, alt_text: str) -> None
    Set the alt text.
▶ def _normalise_metadata(metadata: dict[str, Any]) -> dict[str, Any]
    Internal helper for the normalise metadata step.
▶ def render_horizon_proposal_docx(*, content: str, content_format: Literal['markdown', 'html'],
metadata: dict[str, Any], citation_registry: list[dict[str, Any]] | None=None, evidence_qa:
dict[str, Any] | None=None, evidence_blockers: list[str] | None=None, include_toc: bool=True,
include_bibliography: bool=True, include_evidence_annex: bool=False, page_limit: int | None=45,
max_embedded_image_bytes: int=10000000, enable_footnotes: bool=False) -> ProposalDocxRenderResult
    Create an editable Horizon Part B DOCX from sanitised HTML/Markdown.

```

app/tools/excel_io.py — 25 lines

Read tables from an Excel file. Pure file I/O — no node logic.

```

▶ def read_tables_from_xlsx(file_bytes: bytes) -> dict[str, list[list[Any]]]
    Return a dict of {sheet_name: rows} where rows is a list of lists.

```

app/tools/image_io.py — 235 lines

OpenAI image-generation service used by workflow nodes.

```

▶ class GeneratedImage
    Provides the GeneratedImage behaviour.
▶ class OpenAIImageGenerationService
    Provides the OpenAIImageGenerationService behaviour.
    def available(self) -> bool
        Compute the available.

```

```

    async def generate(self, prompt: str, *, size: str='auto', quality: str='auto', output_format:
str='png', model: OpenAIImageModel | None=None) -> GeneratedImage
    Generate the result.
    ▶ def _image_request_kwargs(*, model: str, prompt: str, size: str, quality: str, output_format:
str) -> dict[str, Any]
        Internal helper for the image request kwargs step.
    ▶ def _model_unavailable(exc: BaseException) -> bool
        Internal helper for the model unavailable step.
    ▶ def get_image_generation_service() -> OpenAIImageGenerationService
        Return the image generation service.

```

app/tools/pdf_io.py — 57 lines

PDF read + render utilities.

```

    ▶ def extract_text_from_pdf(file_bytes: bytes) -> list[dict[str, Any]]
        Return a list of {page_num, text} for each page.
    ▶ def render_proposal_pdf(sections: dict[str, str], template: str, client_name: str,
proposal_title: str) -> bytes
        Render an HTML proposal, then convert to PDF bytes via WeasyPrint.

```

app/tools/powerpoint_io.py — 36 lines

Build a proposal .pptx from a dict of {section: text}.

```

    ▶ def build_proposal_pptx(sections: dict[str, str], title: str, client_name: str) -> bytes
        Title slide + one content slide per section. Returns .pptx bytes.

```

app/tools/proposal_rendering.py — 328 lines

Secure, proposal-aware HTML/Markdown to PDF rendering.

```

    ▶ class ProposalRenderResult
        Provides the ProposalRenderResult behaviour.
    ▶ def _markdown() -> MarkdownIt
        Internal helper for the markdown step.
    ▶ def _slug(value: str) -> str
        Internal helper for the slug step.
    ▶ def _unique_id(base: str, used: set[str]) -> str
        Internal helper for the unique id step.
    ▶ def _sanitise_fragment(value: str, content_format: str) -> str
        Internal helper for the sanitise fragment step.
    ▶ def _toc(fragment: str) -> list[dict[str, Any]]
        Internal helper for the toc step.
    ▶ def _safe_url_fetcher(url: str, *args: Any, **kwargs: Any)
        Allow embedded images and renderer-owned local assets only.
    ▶ def render_horizon_proposal(*, content: str, content_format: Literal['markdown', 'html'],
metadata: dict[str, Any], citation_registry: list[dict[str, Any]] | None=None, evidence_qa:
dict[str, Any] | None=None, evidence_blockers: list[str] | None=None, include_toc: bool=True,
include_bibliography: bool=True, include_evidence_annex: bool=False, page_limit: int | None=45) ->
ProposalRenderResult
        Render a Horizon Part B proposal and return immutable output metadata.

```

app/tools/vision_io.py — 168 lines

Kimi K3 vision service for image understanding.

```

    ▶ class VisionAnalysis

```

```

    Provides the VisionAnalysis behaviour.
    ▶ class KimiVisionService
        Send an object-storage image to Kimi as a base64 content part.
        def available(self) -> bool
            Compute the available.
        async def analyze(self, image_bytes: bytes, *, content_type: str, prompt: str, model:
KimiVisionModel='kimi-k3', max_completion_tokens: int=8192) -> VisionAnalysis
            Compute the analyze.
    ▶ def get_kimi_vision_service() -> KimiVisionService
        Return the kimi vision service.

```

app/tools/web_io.py — 588 lines

Live web-search providers with an auditable fallback policy.

```

    ▶ class WebResult
        Provides the WebResult behaviour.
    ▶ class WebSearchResponse
        Provides the WebSearchResponse behaviour.
    ▶ class TavilyQuotaExhausted
        Tavily cannot serve the request because quota/rate is exhausted.
    ▶ class WebSearchService
        Provides the WebSearchService behaviour.
        def available_providers(self) -> set[str]
            Compute the available providers.
        def resolve_provider(self, requested: SearchProvider) -> str
            Resolve Auto without making a provider request.
        async def search(self, query: str, *, provider: SearchProvider='auto', top_k: int=8,
fallback_to_openai: bool=True) -> WebSearchResponse
            Search the result.
        def _select_provider(self, requested: SearchProvider) -> str
            Select the provider.
        async def _search_tavily(self, query: str, top_k: int) -> list[WebResult]
            Search the tavily.
        async def _search_openai(self, query: str, top_k: int) -> WebSearchResponse
            Search the openai.
        async def _search_kimi(self, query: str, top_k: int) -> WebSearchResponse
            Search the kimi.
        def _stub_results(query: str, top_k: int) -> list[WebResult]
            Internal helper for the stub results step.
    ▶ def _rank_score(index: int, count: int) -> float
        Rank the score.
    ▶ def _url_citations(value: Any) -> list[dict[str, str]]
        Internal helper for the url citations step.
    ▶ def _response_usage(response: Any) -> tuple[int, int]
        Internal helper for the response usage step.
    ▶ def get_web_search_service() -> WebSearchService
        Return the web search service.
    ▶ async def search_web(query: str, top_k: int=8, provider: SearchProvider='auto') -> list[dict[str,
Any]]
        MCP-compatible entry point returning JSON-safe candidate records.

```

workflow (14 modules)

app/workflow/builder_store.py — 347 lines

Durable, file-backed state used by the Workflow Builder.

```
► class WorkflowBuilderStore
    Own autosave drafts and immutable versions for one workflows root.
    def validate_name(name: str) -> str
        Validate the name.
    def validate_version_id(version_id: str) -> str
        Validate the version id.
    def workflow_path(self, name: str) -> Path
        Compute the workflow path.
    def draft_path(self, name: str) -> Path
        Draft the path.
    def versions_dir(self, name: str) -> Path
        Compute the versions dir.
    def save_draft(self, name: str, yaml_text: str, *, canvas: dict[str, Any] | None=None) -> dict[str, Any]
        Save the draft.
    def read_draft(self, name: str) -> dict[str, Any] | None
        Read the draft.
    def delete_draft(self, name: str) -> bool
        Delete the draft.
    def record_version(self, name: str, yaml_text: str) -> str
        Record the version.
    def save_workflow(self, name: str, yaml_text: str) -> str
        Save the workflow.
    def get_version(self, name: str, version_id: str) -> str
        Return the version.
    def list_versions(self, name: str) -> list[dict[str, Any]]
        List the versions.
    def restore_version(self, name: str, version_id: str) -> tuple[str, str]
        Restore the version.
    def delete_workflow(self, name: str) -> bool
        Remove the executable YAML together with every Builder-owned
► def _utc_now() -> str
    Internal helper for the utc now step.
► def _digest(text: str) -> str
    Internal helper for the digest step.
► def _atomic_write(path: Path, text: str) -> None
    Internal helper for the atomic write step.
```

app/workflow/business_projection.py — 14 lines

Business View's projection — see app/workflow/business_view/.

app/workflow/capability_selection.py — 183 lines

Deterministic, zero-token capability → node-type shortlisting for workflow

```
► def _split_pascal_case(name: str) -> str
    "RouterAgent" -> "router agent"; "MCPToolAgent" -> "mcp tool agent" —
► def _tokens(text: str) -> set[str]
```

Internal helper for the tokens step.

- ▶ `def _entry_text(entry: dict[str, Any]) -> str`
Internal helper for the entry text step.
- ▶ `def select_candidate_node_types(prompt: str, manifest: list[dict[str, Any]], *, max_candidates: int=12) -> list[str]`
Return the type names the generator's system prompt should describe:
- ▶ `def select_generation_model(prompt: str, manifest: list[dict[str, Any]]) -> str`
Deterministic complexity → one of two model tiers, no LLM call.

app/workflow/claim_verifications.py — 76 lines

Durable storage for claim-verification results (see

- ▶ `def _key(run_id: str, record_id: str) -> str`
Internal helper for the key step.
- ▶ `async def ensure_indexes(db) -> None`
Ensure the indexes.
- ▶ `async def get_verifications(db, run_id: str) -> dict[str, dict[str, Any]]`
Return the verifications.
- ▶ `async def save_verification(db, run_id: str, record_id: str, result: dict[str, Any]) -> None`
Save the verification.

app/workflow/fact_corrections.py — 201 lines

Editing an extracted fact on a run, and marking what it fed as stale.

- ▶ `def stale_decisions_for(field: str) -> tuple[str, ...]`
Compute the stale decisions for.
- ▶ `def _condition_paths(config: dict[str, Any]) -> list[str]`
Every state path a router/decision node's configuration reads.
- ▶ `def _group_paths(group: Any) -> list[str]`
Group the paths.
- ▶ `def _field_of(path: str, node_id: str) -> str | None`
`understand\_message.parsed.pump\_model` → `pump\_model`, for this node only.
- ▶ `def derive_dependencies(spec: Any, node_id: str) -> dict[str, tuple[str, ...]]`
extraction field → the decision nodes that read it, from the spec itself.
- ▶ `def payload_key_for(output: Any) -> str`
Which sub-object of an extraction node's output holds its fields.
- ▶ `async def apply_fact_correction(db: Any, *, run_id: str, session_id: str, field: str, value: Any, node_id: str | None=None, payload_key: str='result', stale_decisions: tuple[str, ...] | list[str] | None=None, allowed_fields: frozenset[str] | set[str] | None=None) -> dict[str, Any]`
Overwrite one extracted field on a run and mark what it fed as stale.

app/workflow/file_inputs.py — 419 lines

Workflow file-input contracts, validation, and object-storage helpers.

- ▶ `class WorkflowFileInputError`
Raised when an upload or workflow file reference is invalid.
- ▶ `def scope_token(session_id: str) -> str`
Return a non-reversible, path-safe tenant/session token.
- ▶ `def workflow_input_prefix(session_id: str) -> str`
Compute the workflow input prefix.
- ▶ `def safe_filename(filename: str | None) -> str`
Strip paths/control characters while retaining a readable name.
- ▶ `def extension_for(filename: str) -> str`
Compute the extension for.
- ▶ `def category_for_extension(extension: str) -> str`

Compute the category for extension.

- `def content_type_for(filename: str, extension: str) -> str`
Compute the content type for.
- `def accepted_by_spec(ref: WorkflowFileRef, spec: WorkflowInputSpec) -> bool`
Accept category names or literal extensions in workflow YAML.
- `async def store_upload(upload: UploadFile, *, session_id: str, object_store: Any, remaining_total_bytes: int | None=None) -> WorkflowFileRef`
Stream one multipart upload to disk, enforce limits, then store it.
- `async def validate_workflow_inputs(specs: dict[str, WorkflowInputSpec], values: dict[str, Any], *, session_id: str, object_store: Any | None) -> dict[str, Any]`
Validate and normalize declared workflow file inputs before execution.
- `async def validate_workflow_file_reference(raw_ref: WorkflowFileRef | dict[str, Any], *, session_id: str, object_store: Any | None, require_parseable_text: bool=False) -> WorkflowFileRef`
Validate one file reference before a download, extraction, or HITL edit.
- `async def extract_workflow_file_text(raw_ref: WorkflowFileRef | dict[str, Any], *, session_id: str, object_store: Any | None, max_chars: int=1000000) -> dict[str, Any]`
Extract one scoped upload into plain text for the HITL editor.
- `async def record_uploaded_files(db: Any | None, *, session_id: str, refs: list[WorkflowFileRef]) -> None`
Persist upload metadata for audit/cleanup without storing file bytes.

app/workflow/library_metadata.py — 180 lines

Derives Workflow Library catalog metadata and readiness from a WorkflowSpec.

- `def _humanize(identifier: str) -> str`
Internal helper for the humanize step.
- `def _terminal_node_types(spec: WorkflowSpec) -> list[str]`
Internal helper for the terminal node types step.
- `def infer_output_types(spec: WorkflowSpec) -> list[str]`
Compute the infer output types.
- `def _default_human_review_count(spec: WorkflowSpec) -> int`
Internal helper for the default human review count step.
- `def library_summary(spec: WorkflowSpec) -> dict[str, Any]`
The Library card/list-facing metadata dict for one workflow.
- `def readiness_summary(report: WorkflowPreflightReport) -> dict[str, Any]`
Maps a structural preflight report to the Library's three-state

app/workflow/orchestration.py — 470 lines

Shared execution bookkeeping for a single workflow run.

- `class BackgroundRunManager`
Own detached run tasks with a Redis lease shared by all workers.
 - `def launch(self, coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] | None=None) -> None`
Launch the result.
 - `async def _run_owned(self, coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool, services: dict[str, Any] | None=None) -> None`
Run the owned.
 - `async def close(self) -> None`
Cancel owned work and let run_and_finalize persist interruption.
- `async def start_new_run_record(db: Any, *, run_id: str, session: str, spec: WorkflowSpec, workflow_yaml: str, inputs: dict[str, Any], collection_id: str, retry_of_run_id: str | None=None, attempt: int=1, pipeline_run_id: str | None=None, pipeline_name: str | None=None, stage_id: str | None=None, stage_index: int | None=None, total_stages: int | None=None) -> None`
Create the durable "running" record and checkpoint before execution.

- ▶ `async def record_run_failure(db: Any, *, run_id: str, session: str, error: BaseException | str, services: dict[str, Any] | None=None) -> dict[str, Any]`
Persist an exception raised while executing/resuming a run.
- ▶ `async def finalize_run_result(result: dict[str, Any], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] | None=None) -> dict[str, Any]`
Persist a terminal or re-paused outcome. Node hooks have already
- ▶ `async def run_and_finalize(coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] | None=None) -> dict[str, Any]`
Await one execution coroutine and persist its outcome end to end.
- ▶ `async def _reconcile_pipeline_stage(db: Any, run_id: str, session: str) -> None`
Best-effort: if this run_id is a pipeline stage, sync the pipeline.
- ▶ `async def _reconcile_subprocess_callback(db: Any, run_id: str, session: str, services: dict[str, Any] | None=*, status: str, output: Any, node_outputs: dict[str, Any], error: str | None) -> None`
Best-effort: if this run_id is a Subprocess node's child, deliver its

app/workflow/pipeline_history.py — 283 lines

Durable pipeline-run tracking.

- ▶ `def _require_session(session_id: str) -> str`
Internal helper for the require session step.
- ▶ `async def ensure_pipeline_indexes(db) -> None`
Ensure the pipeline indexes.
- ▶ `async def create_pipeline_run(db, *, pipeline_run_id: str, session_id: str, pipeline_name: str, pipeline_yaml: str, pipeline_inputs: dict[str, Any], stages: list[dict[str, Any]]) -> None`
Create the pipeline record before its first stage starts.
- ▶ `async def record_stage_launch(db, *, pipeline_run_id: str, session_id: str, stage_index: int, run_id: str) -> None`
Link a stage to the run_id about to execute it.
- ▶ `async def get_pipeline_run(db, session_id: str, pipeline_run_id: str) -> dict[str, Any] | None`
Return the pipeline run.
- ▶ `async def list_pipeline_runs(db, session_id: str, limit: int=50) -> list[dict[str, Any]]`
List the pipeline runs.
- ▶ `async def find_active_pipeline_stage(db, *, run_id: str, session_id: str) -> dict[str, Any] | None`
Return the pipeline_runs doc if run_id is a stage of a non-terminal
- ▶ `async def abandon_pipeline(db, *, pipeline_run_id: str, session_id: str) -> bool`
Manually move a stuck pipeline out of "running"/"gated" so its stage
- ▶ `def _stage_outcome_status(run_status: str) -> str`
Map an underlying workflow run's status onto a pipeline stage status.
- ▶ `async def reconcile_stage_completion(db, *, run_id: str, session_id: str) -> None`
Sync pipeline state after ANY run finalizes, whether via the pipeline's

app/workflow/preflight_stats.py — 162 lines

One record per preflight-producing call (manual validate, generate-from-

- ▶ `async def ensure_indexes(db: Any) -> None`
Ensure the indexes.
- ▶ `async def record_attempt(db: Any, *, source: AttemptSource, workflow_name: str | None, success: bool, error_codes: list[str], initial_error_codes: list[str] | None=None, deterministic_fixes_applied: int=0, llm_attempts: int=0, total_attempts: int=1) -> None`
Record the attempt.
- ▶ `async def preflight_stats(db: Any, *, since: datetime | None=None, sample_limit: int=2000) -> dict[str, Any]`
Compute the preflight stats.

app/workflow/run_chat_store.py — 83 lines

Durable storage for "Ask AI about this run" conversations.

- ▶ `def _require_session(session_id: str) -> str`
Internal helper for the require session step.
- ▶ `async def ensure_run_chat_indexes(db) -> None`
Ensure the run chat indexes.
- ▶ `async def get_run_chat_turns(db, *, session_id: str, run_id: str) -> list[dict[str, Any]]`
Return the run chat turns.
- ▶ `async def append_run_chat_turn(db, *, session_id: str, run_id: str, question: str, answer: str, model: str) -> list[dict[str, Any]]`
Append one Q&A turn and return the full updated turn list.

app/workflow/run_history.py — 1446 lines

Durable workflow-run history.

- ▶ `def _is_document_size_error(exc: Exception) -> bool`
Return whether document size error.
- ▶ `async def _externalize_if_large(db, *, run_id: str, node_id: str, field: str, value: Any, force: bool=False) -> Any`
Return `value` unchanged, or a small GridFS pointer if it's too big.
- ▶ `async def _inflate_value(db, value: Any) -> Any`
Reverse `\_externalize\_if\_large`: fetch and decode a GridFS pointer.
- ▶ `async def _inflate_run_document(db, doc: dict[str, Any] | None) -> dict[str, Any] | None`
Transparently resolve GridFS pointers so callers see the original shape.
- ▶ `def _require_session(session_id: str) -> str`
Enforce the same isolation boundary on every history operation.
- ▶ `def _node_key(node_id: str) -> str`
Return a Mongo-safe map key while preserving node_id in the value.
- ▶ `def redact_for_history(value: Any) -> Any`
Recursively remove prompts and credentials from operator-visible data.
- ▶ `def build_node_input(state: dict[str, Any], resolved_config: dict[str, Any]) -> dict[str, Any]`
Build the inspectable input for one node without duplicating all outputs.
- ▶ `async def ensure_indexes(db) -> None`
Create indexes used by run detail and newest-first history views.
- ▶ `async def upsert_run(db, run_id: str, session_id: str, workflow_name: str | None=None, status: str | None=None, node_types: dict[str, str] | None=None, inputs: dict[str, Any] | None=None, variables: dict[str, Any] | None=None, outputs: dict[str, Any] | None=None, workflow_yaml: str | None=None, started_at: float | None=None, ended_at: float | None=None, node_count: int | None=None, completed_node_count: int | None=None, retry_of_run_id: str | None=None, attempt: int | None=None, reused_node_count: int | None=None, reused_nodes: list[str] | None=None, error: str | None=None, pipeline_run_id: str | None=None, pipeline_name: str | None=None, stage_id: str | None=None, stage_index: int | None=None, total_stages: int | None=None) -> None`
Create or patch one run without erasing fields omitted by the caller.
- ▶ `async def record_node_started(db, *, run_id: str, session_id: str, node_id: str, type_name: str, node_input: dict[str, Any], started_at: float) -> None`
Mark one node active and persist its effective, redacted input.
- ▶ `async def record_node_completed(db, *, run_id: str, session_id: str, node_id: str, output: dict[str, Any], model_selections: list[dict[str, Any]] | None=None, ended_at: float, duration_s: float) -> None`
Persist a full node output as soon as that node completes.
- ▶ `async def record_node_reused(db, *, run_id: str, session_id: str, node_id: str, type_name: str, output: dict[str, Any], source_run_id: str, ended_at: float, duration_s: float) -> None`
Persist a node result replayed from a previous failed attempt.
- ▶ `async def record_node_paused(db, *, run_id: str, session_id: str, node_id: str, paused_at: float, pause_kind: str='hitl_gate') -> None`

Mark a node and the overall run as paused.

- ▶ `async def request_pause(db, *, run_id: str, session_id: str) -> bool`
Ask a running workflow to pause at its next node boundary.
- ▶ `async def clear_pause_request(db, *, run_id: str, session_id: str) -> None`
Clear the pause request.
- ▶ `async def is_pause_requested(db, *, run_id: str, session_id: str) -> bool`
Cheap per-node check — the executor calls this before every node.
- ▶ `async def record_node_failed(db, *, run_id: str, session_id: str, node_id: str, type_name: str, error: str, error_type: str | None=None, error_traceback: str | None=None, model_selections: list[dict[str, Any]] | None=None, ended_at: float, duration_s: float) -> None`
Persist node attribution and partial run state on failure.
- ▶ `async def initialize_run_checkpoint(db, *, run_id: str, session_id: str, workflow_yaml: str, inputs: dict[str, Any], collection_id: str, retry_of_run_id: str | None=None) -> None`
Create the private checkpoint used for zero-token node replay.
- ▶ `async def record_checkpoint_node_completed(db, *, run_id: str, session_id: str, node_id: str, output: dict[str, Any], extra_state: dict[str, Any]) -> None`
Save the exact replayable result immediately after node completion.
- ▶ `async def record_checkpoint_paused(db, *, run_id: str, session_id: str, node_id: str, pause_context: Any, pause_kind: str='hitl_gate') -> None`
Persist enough information to resume a paused run after process restart.
- ▶ `async def record_checkpoint_approval(db, *, run_id: str, session_id: str, node_id: str, decision: dict[str, Any], actor: str) -> bool`
Append an immutable human decision before resuming execution.
- ▶ `async def mark_checkpoint_status(db, *, run_id: str, session_id: str, status: str) -> None`
Mark the checkpoint status.
- ▶ `async def get_retry_checkpoint(db, session_id: str, run_id: str) -> dict[str, Any] | None`
Return a session-scoped private checkpoint for the retry endpoint.
- ▶ `async def get_resume_checkpoint(db, session_id: str, run_id: str) -> dict[str, Any] | None`
Return a restart-safe checkpoint only when it is currently paused.
- ▶ `def _as_aware_utc(value: datetime) -> datetime`
Internal helper for the as aware utc step.
- ▶ `def _process_is_alive(pid: int | None) -> bool`
Best-effort same-host liveness check for a run's owning process.
- ▶ `async def _reconcile_if_stale(db, doc: dict[str, Any] | None) -> dict[str, Any] | None`
Flip an orphaned "running" run to "failed" the moment it is read.
- ▶ `async def get_run(db, session_id: str, run_id: str) -> dict[str, Any] | None`
Return one full run if it belongs to the caller's session.
- ▶ `async def list_runs(db, session_id: str, limit: int=50) -> list[dict[str, Any]]`
Return light run summaries, newest first, including active runs.
- ▶ `async def workflow_stats(db, session_id: str, workflow_name: str, *, sample_limit: int=200) -> dict[str, Any]`
Library "Runs and performance" tab: success rate, median duration, and
- ▶ `async def _delete_run_blobs(db, *, run_id: str) -> None`
Remove every GridFS blob externalized for this run (see
- ▶ `async def delete_run(db, *, run_id: str, session_id: str) -> bool`
Permanently remove a run's history, checkpoint, and externalized blobs.
- ▶ `async def cleanup_stale_runs(db) -> list[str]`
Periodic sweep (see the background loop started in app/main.py) that

app/workflow/subprocess_callback.py — 207 lines

Deliver a finished child run's result to its waiting Subprocess parent.

- ▶ `def _select_result(launch: dict[str, Any], *, output: Any, node_outputs: dict[str, Any]) -> Any`
Project the child's result the way the SubprocessConfig author asked
- ▶ `async def deliver_by_token(db: Any, services: dict[str, Any], token: str, *, status: str, output: Any, node_outputs: dict[str, Any], error: str | None) -> dict[str, Any]`

Deliver the by token.

- ▶ `async def deliver_by_child_run_id(db: Any, services: dict[str, Any], child_run_id: str, *, status: str, output: Any, node_outputs: dict[str, Any], error: str | None) -> None`
Best-effort: the overwhelming majority of runs are not a Subprocess

`app/workflow/subprocess_launches.py` — 171 lines

Subprocess launch correlation — the webhook side of the Subprocess node.

- ▶ `async def ensure_indexes(db: Any) -> None`
Ensure the indexes.
- ▶ `async def find_by_launch_key(db: Any, launch_key: str) -> dict[str, Any] | None`
Find the by launch key.
- ▶ `async def reserve_launch(db: Any, *, launch_key: str, parent_run_id: str, parent_node_id: str, parent_session_id: str, child_run_id: str, child_workflow: str, result_from: str, result_node: str | None, timeout_seconds: float) -> tuple[dict[str, Any], bool]`
Claim `launch\_key` for one child launch. Returns (doc, created).
- ▶ `async def find_by_child_run_id(db: Any, child_run_id: str) -> dict[str, Any] | None`
Find the by child run id.
- ▶ `async def find_by_token(db: Any, token: str) -> dict[str, Any] | None`
Find the by token.
- ▶ `async def deliver(db: Any, *, token: str, decision: dict[str, Any]) -> bool`
Atomically mark the launch "delivered", once.

workflow.business_view (18 modules)

`app/workflow/business_view/__init__.py` — 46 lines

Business View — the surface a business employee works on, not a run viewer.

`app/workflow/business_view/actions.py` — 390 lines

Typed, permission-checked Business View actions.

```
► class ActionContext
    Everything action availability depends on.
    def can_act(self) -> bool
        Whether this person may change the work item at all.
    def rerun_mode(self) -> str | None
        Which existing rerun primitive fits this run's state.
► class ActionFactory
    Builds the actions valid for one run, for one person, right now.
    def pause(self) -> BusinessAction | None
        Pause the result.
    def resume(self) -> BusinessAction | None
        Resume the result.
    def stop(self) -> BusinessAction | None
        Stop the result.
    def recheck(self, *, label: str | None=None) -> BusinessAction | None
        Compute the recheck.
    def approve(self) -> BusinessAction | None
        Compute the approve.
    def assign(self, *, suggested: str | None=None) -> BusinessAction | None
        Compute the assign.
    def edit_fact(self, field: str, *, label: str | None=None, emphasis: str='secondary') ->
BusinessAction | None
        Compute the edit fact.
    def explain(self, *, target: str='decision') -> BusinessAction
        Compute the explain.
    def draft_clarification(self, *, topic: str, label: str='Ask customer') -> BusinessAction |
None
        Draft the clarification.
    def add_note(self) -> BusinessAction | None
        Add the note.
    def route_override(self, *, current: str | None) -> BusinessAction | None
        Compute the route override.
    def review_attachment(self, *, file_key: str, name: str) -> BusinessAction
        Compute the review attachment.
    def lookup_record(self, *, kind: str, reference: str, tool: str, server_id: str, argument: str)
-> BusinessAction | None
        Read a named record back out of the system of record.
    def open_record(self, *, kind: str, reference: str) -> BusinessAction
        Compute the open record.
    def technical_details(self, activity_id: str, *, label: str='Technical details') ->
BusinessAction
        Compute the technical details.
    def ask_ai(self, *, question: str, label: str | None=None) -> BusinessAction
        Compute the ask ai.
```

app/workflow/business_view/activities.py — 617 lines

Business activity aggregation — the replacement for per-node event spam.

- ▶ class _ActivityDef
Provides the ActivityDef behaviour.
- ▶ class _Group
Provides the Group behaviour.
- ▶ def _stage_definitions(run: RunView) -> dict[str, tuple[str, int]]
Declared experience stages → (display name, order).
- ▶ def classify(node: NodeView, stages: dict[str, tuple[str, int]]) -> str
Which business activity this node contributes to.
- ▶ def group_nodes(run: RunView) -> list[_Group]
Bucket every node that ran into ordered activity groups.
- ▶ def _group_status(nodes: list[NodeView]) -> str
Group the status.
- ▶ def _group_kind(group: _Group) -> ActivityKind
Group the kind.
- ▶ def _source_label(source: BusinessSource, ai: AIModelUsage | None) -> str
A source label that names the model when — and only when — AI ran it.
- ▶ def system_check_fact(node: NodeView) -> BusinessFact
One external lookup, as the business outcome of that lookup.
- ▶ def router_fact(finding: RouterFinding, run: RunView) -> BusinessFact
One deterministic check, in the words of the thing it checked.
- ▶ def handoff_facts(node: NodeView) -> list[BusinessFact]
Team/owner/customer facts out of a terminal handoff note.
- ▶ def _technical(group: _Group, ai: AIModelUsage | None) -> TechnicalActivityDetail
References into the technical layer — deliberately no payloads (§5, §60).
- ▶ def _summary_for(group: _Group, facts: list[BusinessFact]) -> str | None
Internal helper for the summary for step.
- ▶ def build_activities(run: RunView, factory: ActionFactory) -> list[BusinessActivityView]
Every business activity this run has actually performed.
- ▶ def active_activity(activities: list[BusinessActivityView]) -> BusinessActivityView | None
Compute the active activity.
- ▶ def running_headline(activity_id: str) -> str | None
Compute the running headline.
- ▶ def outcome_note(run: RunView) -> tuple[NodeView, dict[str, str]] | None
The last terminal handoff note this run produced, if any.
- ▶ def find_understanding_node(run: RunView) -> NodeView | None
The AI node that interpreted the incoming request.
- ▶ def extraction_payload(node: NodeView | None) -> dict[str, Any]
The structured fields an extraction node produced.

app/workflow/business_view/attention.py — 326 lines

The Attention Center — what needs a person, and how they can resolve it.

- ▶ def _field_for(text: str, known: set[str]) -> str | None
Which extracted field this free-text gap refers to, if any.
- ▶ def _title_for(text: str, field: str | None) -> str
A short business title for one gap.
- ▶ def _resolution_actions(*, text: str, field: str | None, factory: ActionFactory, attachments: list[BusinessAttachment], records: list[BusinessRelatedRecord], editable: set[str]) -> tuple[list[BusinessAction], str]
Every way this gap can actually be closed, most direct first.
- ▶ def _missing_information(run: RunView, payload: dict[str, Any]) -> list[str]
Everything the workflow itself reported as materially missing.

- ▶ `def best_action(item: BusinessAttentionItem) -> BusinessAction | None`
The action to recommend for this item — evidence first (§28).
- ▶ `def build_attention(run: RunView, payload: dict[str, Any], factory: ActionFactory, *, attachments: list[BusinessAttachment], records: list[BusinessRelatedRecord], editable: set[str], rule_linked: set[str] | None=None) -> list[BusinessAttentionItem]`
Everything on this work item that wants a person, worst first.
- ▶ `def blocking_items(items: list[BusinessAttentionItem]) -> list[BusinessAttentionItem]`
Compute the blocking items.

`app/workflow/business_view/common.py` — 393 lines

Shared, dependency-free helpers for the Business View projection.

- ▶ `def humanize_identifier(value: str) -> str`
``rfq_inside_sales_queue` → `Rfq inside sales queue`.`
- ▶ `def field_label(key: str) -> str`
Business label for an extraction/record field name.
- ▶ `def lookup_label(node_id: str) -> str`
``get_sales_order` → `Sales order`; `emergency_customer_lookup` → `Emergency customer`.`
- ▶ `def humanize_route(value: str) -> str`
``INSIDE_SALES_QUEUE` → `Inside sales queue`; `RFQ` stays `RFQ`.`
- ▶ `def humanize_case_title(value: str) -> str`
``STANDARD RFQ - KEY ACCOUNT` → `Standard RFQ — key account`.`
- ▶ `def is_empty_value(value: Any) -> bool`
Whether a value means "the workflow found nothing here".
- ▶ `def format_value(value: Any, *, key: str='') -> str`
A business-readable rendering of one extracted value.
- ▶ `def compact_text(value: Any, max_length: int=220) -> str | None`
Whitespace-collapsed, length-capped text, or None for anything else.
- ▶ `def timestamp_iso(value: Any) -> str | None`
Normalise the several time shapes run history stores into one ISO string.
- ▶ `def duration_ms(started: Any, ended: Any) -> int | None`
Elapsed milliseconds between two epoch-second timestamps, if both exist.
- ▶ `def sentence(text: str) -> str`
Capitalise and full-stop a fragment so headlines read as prose.
- ▶ `def parse_handoff_note(text: Any) -> dict[str, str]`
Read the ``CASE: ...`` / ``Key: value`` handoff note these workflows emit.
- ▶ `def title_case_team(value: str) -> str`
Trim a handoff team string to the part a status headline can carry.

`app/workflow/business_view/context.py` — 159 lines

Business context: the attachments and related records this work item touches.

- ▶ `def available_tools(run: RunView) -> dict[str, str]`
``tool name` → server id` for every MCP tool this run's workflow declares.`
- ▶ `def _file_refs(value: Any) -> list[dict[str, Any]]`
Every ``WorkflowFileRef``-shaped dict inside a run input value.
- ▶ `def build_attachments(run: RunView, factory: ActionFactory) -> list[BusinessAttachment]`
Real uploaded files on this run, each with a way to open it (§36).
- ▶ `def build_related_records(run: RunView, payload: dict[str, Any], factory: ActionFactory) -> list[BusinessRelatedRecord]`
Records in other systems this request names (§37).
- ▶ `def customer_name(run: RunView, payload: dict[str, Any]) -> str | None`
The customer this work item is about, verified where possible.

app/workflow/business_view/decision.py — 246 lines

The handling decision, and the facts and rules behind it (§19, §20).

- ▶ `def _decision_agent_view(run: RunView) -> tuple[dict, list[BusinessRule], list[BusinessFact]] | None`
Decisions, matched rules and decision facts from a `DecisionAgent`.
- ▶ `def _agent_headline(decisions: dict) -> str | None`
The outcome a `DecisionAgent`'s named fields amount to.
- ▶ `def _router_rules(run: RunView) -> list[BusinessRule]`
One rule entry per router that ran, in the author's own wording.
- ▶ `def build_decision(run: RunView, factory: ActionFactory, handling_facts: list[BusinessFact]) -> BusinessDecisionView | None`
The handling decision, or None when this run has not made one yet.
- ▶ `def handling_facts(run: RunView) -> list[BusinessFact]`
The supporting "✓ RFQ / ✓ Standard complexity / ✓ No safety issue" facts.

app/workflow/business_view/dispatch.py — 271 lines

Executing a typed Business View action.

- ▶ `class BusinessActionError`
A typed action that cannot be performed as asked.
- ▶ `class ClarificationDraft`
A question for the customer, drafted and never sent.
- ▶ `async def _draft_clarification(*, llm: Any, topic: str, context: dict[str, Any]) -> dict[str, Any]`
Draft the clarification.
- ▶ `async def _lookup_record(*, mcp: Any, params: dict[str, Any], run_id: str, session_id: str, role: str) -> dict[str, Any]`
Internal helper for the lookup record step.
- ▶ `async def dispatch_business_action(*, action_type: str, params: dict[str, Any], run_id: str, session_id: str, username: str, role: str, db: Any, services: dict[str, Any], context: dict[str, Any] | None=None) -> dict[str, Any]`
Perform one typed action and return what the UI should show.

app/workflow/business_view/explanation.py — 146 lines

«Why?» — the facts and rules behind a decision, optionally in plainer words.

- ▶ `class DecisionExplanation`
Pydantic model defining the DecisionExplanation shape.
- ▶ `class ExplanationView`
What the Why? panel renders.
- ▶ `def deterministic_explanation(decision: BusinessDecisionView) -> ExplanationView`
Facts and rules, exactly as the run recorded them.
- ▶ `def validate(explanation: DecisionExplanation, view: ExplanationView) -> str | None`
None when every citation resolves, otherwise why it was rejected.
- ▶ `async def explain(llm: Any, decision: BusinessDecisionView, *, model: str=EXPLANATION_MODEL) -> ExplanationView`
The Why? panel — plainer wording when it validates, evidence always.

app/workflow/business_view/models.py — 459 lines

The Business View contract — the shapes a business user's screen is built from.

- ▶ `class BusinessSource`
Where a displayed result actually came from.
- ▶ `class BusinessActionType`

The closed set of typed platform actions a Business View button may invoke.

- ▶ `class BusinessAction`
One button. Typed, permission-checked, and valid for the current state.
- ▶ `class BusinessFact`
One business-language value with its provenance.
- ▶ `class AIModelUsage`
Which model actually ran, alongside what was asked for (§22–§24).
- ▶ `class BusinessRule`
One deterministic rule that fired, in the author's own words.
- ▶ `class TechnicalNodeRef`
A pointer into the technical layer — never the technical payload itself.
- ▶ `class TechnicalActivityDetail`
What "Technical details" reveals for one business activity (§47).
- ▶ `class BusinessActivityView`
Several technical nodes, presented as the one thing the business did.
- ▶ `class BusinessAttentionItem`
One thing a person should look at, with the ways to resolve it (§6, §7).
- ▶ `class BusinessStatusView`
The authoritative business status (§12, §13).
- ▶ `class BusinessUnderstanding`
"What I understood" as business fields, never as JSON (§4).
- ▶ `class BusinessDecisionView`
The handling decision, made visually important (§19, §20).
- ▶ `class BusinessNextStep`
"What happens next" — always present, blocked or not (§30).
- ▶ `class BusinessRelatedRecord`
A record in another system this work item refers to (§35, §37).
- ▶ `class BusinessAttachment`
A real file input on this run — never an attachment merely mentioned in
- ▶ `class BusinessTimelineEntry`
One business-meaningful moment.
- ▶ `class BusinessWorkItem`
Pydantic model defining the BusinessWorkItem shape.
- ▶ `class BusinessRequiredUserAction`
A pending human gate, kept as its own field because the HITL review
- ▶ `class BusinessProcess`
Pydantic model defining the BusinessProcess shape.
- ▶ `class BusinessProjection`
Everything one Work Item screen needs, in business language (§45).

`app/workflow/business_view/narrator.py` — 245 lines

BusinessStatusNarrator — turning validated state into concise business words.

- ▶ `class NarrationInput`
Everything the narrator is allowed to see. Nothing else is sent.
- ▶ `class Narration`
Pydantic model defining the Narration shape.
- ▶ `def narration_input(projection: BusinessProjection) -> NarrationInput`
The bounded, structured input contract (§14).
- ▶ `def deterministic_narration(projection: BusinessProjection) -> Narration`
The narration the screen uses when no model is involved (§16).
- ▶ `def _claims(text: str) -> set[str]`
Internal helper for the claims step.
- ▶ `def validate(narration: Narration, source: NarrationInput) -> str | None`
None when the narration is usable, otherwise why it was rejected.

```

▶ async def narrate(llm: Any, projection: BusinessProjection, *, model: str=NARRATION_MODEL) ->
tuple[Narration, str, str | None]
    Return (narration, source, model) — falling back rather than failing.
▶ def apply(projection: BusinessProjection, narration: Narration, *, source: str, model: str |
None) -> None
    Fold a narration into the projection's status, in place.

```

app/workflow/business_view/projection.py — 304 lines

Assembling one Work Item screen (§45).

```

▶ def _work_item_title(customer: str | None, work_type: str, process_name: str) -> str
    "BASF RFQ" rather than "Pump Manufacturer Multilingual Customer Case Routing".
▶ def _request_type(payload: dict[str, Any]) -> str
    Internal helper for the request type step.
▶ def _required_user_actions(run) -> list[BusinessRequiredUserAction]
    Internal helper for the required user actions step.
▶ def _suggested_questions(*, status_code: str, decision, attention, records) -> list[str]
    Prompt chips that change with the work item's state (§31).
▶ def _dedupe(actions: list[BusinessAction], *, by_label: bool=False) -> list[BusinessAction]
    Drop repeats. Two buttons reading the same is a bug, whatever their ids.
▶ def build_business_projection(run_doc: dict[str, Any], *, workflow_spec: WorkflowSpec | None,
gate: dict[str, Any] | None=None, cost_entries: list[dict[str, Any]] | None=None, role: Role |
str=Role.CONULTANT) -> BusinessProjection
    Reshape one run into everything its Business View screen needs.

```

app/workflow/business_view/routers.py — 185 lines

Reading a deterministic router as a business finding.

```

▶ class RouterFinding
    One router's answer, in business terms where the subject is recoverable.
    def route_label(self) -> str
        The route label.
▶ def _condition_paths(config: dict[str, Any]) -> list[str]
    Every state path this router's configuration tests, in declared order.
▶ def _group_paths(group: Any) -> list[str]
    Recursively collect 'field' paths out of a structured condition group.
▶ def _common_subject(paths: list[str]) -> str | None
    Internal helper for the common subject step.
▶ def read_router(node: NodeView, run: RunView) -> RouterFinding | None
    Turn one executed router node into a business finding, or None.
▶ def _outputs_of(run: RunView) -> dict[str, Any]
    Internal helper for the outputs of step.
▶ def router_findings(run: RunView) -> list[RouterFinding]
    Every router that actually ran, in execution order.

```

app/workflow/business_view/runstate.py — 397 lines

One normalised view of a run, shared by every part of the projection.

```

▶ class NodeView
    One workflow node as the Business View needs to see it.
    def ran(self) -> bool
        The ran.
    def succeeded(self) -> bool
        The succeeded.
    def source(self) -> BusinessSource

```

```

    The source.
def status_label(self) -> str
    The status label.
def duration_ms(self) -> int | None
    The duration ms.
def started_iso(self) -> str | None
    The started iso.
def ended_iso(self) -> str | None
    The ended iso.
def output_dict(self) -> dict[str, Any]
    Compute the output dict.
▶ class RunView
    The whole run, normalised. Built once per projection.
    def node(self, node_id: str) -> NodeView | None
        Compute the node.
    def ran_nodes(self) -> list[NodeView]
        Compute the ran nodes.
    def of_kind(self, *kinds: str) -> list[NodeView]
        Compute the of kind.
    def of_type(self, *type_names: str) -> list[NodeView]
        Compute the of type.
    def is_paused(self) -> bool
        The is paused.
    def awaits_approval(self) -> bool
        The awaits approval.
    def is_finished(self) -> bool
        The is finished.
▶ def _execution_kinds() -> dict[str, str]
    type_name → execution_kind, read from the node registry itself.
▶ def execution_kind_of(type_name: str) -> str
    Compute the execution kind of.
▶ def _node_status(node_id: str, run_doc: dict[str, Any]) -> str
    Business status for one node, from the durable record.
▶ def _display_name(node_id: str, spec: NodeSpec | None) -> str
    Internal helper for the display name step.
▶ def build_run_view(run_doc: dict[str, Any], *, workflow_spec: WorkflowSpec | None, gate:
dict[str, Any] | None=None, cost_entries: list[dict[str, Any]] | None=None) -> RunView
    Normalise a run document into the shape the projection reads.
▶ def model_usage(nodes: list[NodeView]) -> AIModelUsage | None
    Roll up the AI calls made by a set of nodes into one badge (§22–§24).

```

app/workflow/business_view/status.py — 247 lines

The authoritative business status, and what happens next.

```

▶ def _attention_count(items: list[BusinessAttentionItem]) -> int
    Internal helper for the attention count step.
▶ def build_status(run: RunView, *, activities: list[BusinessActivityView], attention:
list[BusinessAttentionItem], decision: BusinessDecisionView | None, understanding:
BusinessUnderstanding, customer: str | None) -> BusinessStatusView
    The one status line a business user reads first.
▶ def _status(code, headline, summary, tone, count, run) -> BusinessStatusView
    Internal helper for the status step.
▶ def build_next_step(run: RunView, *, status: BusinessStatusView, decision: BusinessDecisionView |
None, attention: list[BusinessAttentionItem], handoff_action: str | None, factory: ActionFactory)
-> BusinessNextStep
    "What happens next" — always answered, blocked or not (§30).

```

```

▶ def state_version(*, status: BusinessStatusView, decision: BusinessDecisionView | None,
attention: list[BusinessAttentionItem], activities: list[BusinessActivityView], understanding:
BusinessUnderstanding) -> str
    A stable hash of everything a business user would notice changing.

```

app/workflow/business_view/store.py — 163 lines

Durable state the Business View adds to a run: narration cache, notes, overrides.

```

▶ async def ensure_business_view_indexes(db: Any) -> None
    One lookup key, one TTL-free cache — cheap to rebuild, so no history.
▶ async def get_cached_narration(db: Any, *, run_id: str, session_id: str, state_version: str) ->
dict[str, Any] | None
    Return the cached narration.
▶ async def put_cached_narration(db: Any, *, run_id: str, session_id: str, state_version: str,
headline: str, summary: str, next_step: str, source: str, model: str | None) -> None
    Store the cached narration.
▶ async def _append_to_run(db: Any, *, run_id: str, session_id: str, field: str, record: dict[str,
Any]) -> dict[str, Any]
    Append the to run.
▶ async def add_note(db: Any, *, run_id: str, session_id: str, text: str, by: str) -> dict[str,
Any]
    Record something a person wants the next handler to know.
▶ async def add_route_override(db: Any, *, run_id: str, session_id: str, route: str, reason: str |
None, by: str) -> dict[str, Any]
    Record that a person sent this work item somewhere else.

```

app/workflow/business_view/timeline.py — 207 lines

The business timeline: what happened, once per thing that happened.

```

▶ def _marks(activity: BusinessActivityView) -> list[str]
    Internal helper for the marks step.
▶ def build_timeline(run: RunView, *, activities: list[BusinessActivityView], decision:
BusinessDecisionView | None, status: BusinessStatusView) -> list[BusinessTimelineEntry]
    Build the timeline.
▶ def happened(activities: list[BusinessActivityView], decision: BusinessDecisionView | None) ->
list[str]
    The compact "What happened" checklist shown before the full history (§18).

```

app/workflow/business_view/understanding.py — 141 lines

«What I understood» as business fields — the replacement for the JSON dump.

```

▶ def _ordered(keys: list[str]) -> list[str]
    Business-priority order first, then whatever the workflow also extracted.
▶ def editable_fields(payload: dict[str, Any]) -> set[str]
    Which extracted fields a person may correct in place.
▶ def build_understanding(run: RunView, factory: ActionFactory) -> tuple[BusinessUnderstanding,
NodeView | None, dict[str, Any]]
    The understanding block, plus the node and payload other parts reuse.
▶ def correction_target(run: RunView) -> tuple[str, str, set[str]] | None
    (node_id, payload key, editable fields) for this run's extraction node.

```